

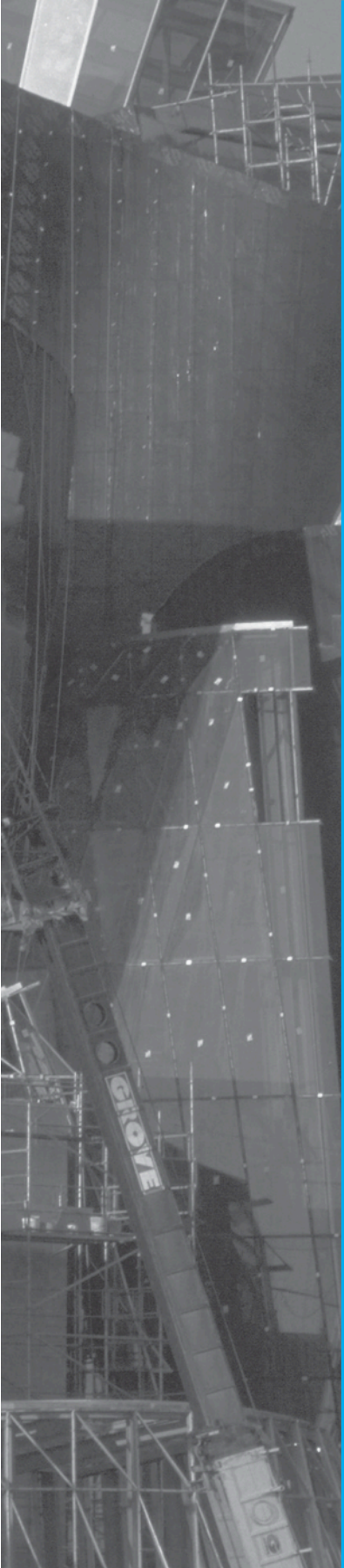
SOMMERVILLE



SOFTWARE ENGINEERING



9



4

Requirements engineering

Objectives

The objective of this chapter is to introduce software requirements and to discuss the processes involved in discovering, documenting checking, and managing these requirements. When you have read the chapter you will:

- understand the differences between business, functional, and non-functional requirements;
- understand how requirements may be organized in a software requirements document;
- understand the stages of the requirements engineering process and the main elicitation techniques;
- know the checks and techniques used to validate requirements;
- understand why requirements change and how requirements change is managed.

Contents

- 4.1 Functional and non-functional requirements
- 4.2 The software requirements document
- 4.3 Requirements specification
- 4.4 Requirements engineering processes
- 4.5 Requirements elicitation and analysis
- 4.6 Requirements validation
- 4.7 Requirements management

The requirements for a system are the descriptions of the services that a software system should provide and the constraints under which it must operate. They define *what* the software should do to meet the needs of its users and other stakeholders, and they say nothing, or as little as possible, about *how* it will do it. The process of discovering, analyzing, documenting, and checking these services and constraints is called requirements engineering (RE).

Requirements engineering matters because every later activity depends on it. A design can be no better than the requirements it implements, and tests can only check what the requirements state. Errors in requirements are also the most expensive errors to correct. If a missing or wrong requirement is discovered after the system has been delivered, the design, the code, the tests, and the documentation may all have to change. The same error found while the requirements are being written costs only a revision of a few sentences. This is the reason for the attention that this chapter gives to validating requirements and managing requirements change.

It helps to separate two levels of description:

1. *User requirements* are statements, in natural language plus diagrams, of what services the system is expected to provide to its users and the constraints under which it must operate. They are written for customers and managers who are not interested in the details of the implementation.
2. *System requirements* are more detailed descriptions of the software system's functions, services, and operational constraints. The system requirements document (sometimes called a functional specification) defines exactly what is to be implemented, and it may be part of the contract between the buyer of the system and its developers.

Different levels of requirement are needed because they are read by different people. A client manager wants to check that the system will support the business; a software developer needs to know precisely what each function must do with each input.

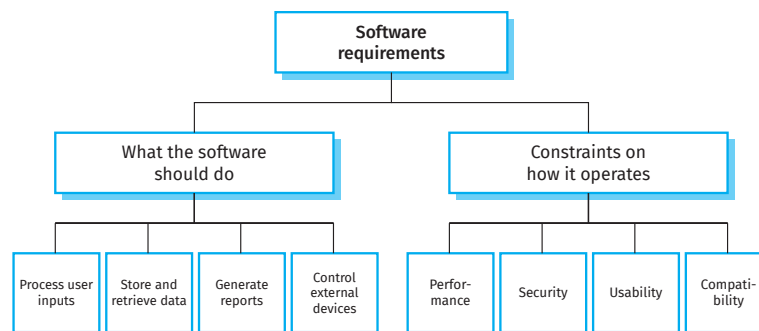
The people whose needs the requirements must meet are the system *stakeholders*: anyone who is affected by the system or who has a legitimate interest in it. *Direct* stakeholders use the system or operate it themselves, such as end-users, operators, and administrators. *Indirect* stakeholders do not use it but are affected by its results or have an influence over its requirements, such as managers, customers of the organization, regulators, and auditors.

4.1 Functional and non-functional requirements

Software requirements can be divided into two groups (Figure 4.1). The first group states what the software should do: it should process user inputs, store and retrieve data, generate reports, or control external devices. The second group states the

constraints under which it does these things: how fast it must respond, how secure it must be, how easy it must be to use, and which other systems it must work with.

Figure 4.1 Business, functional, and non-functional requirements



Both groups sit inside a third, higher-level kind of requirement. *Business requirements* are the high-level goals and objectives that a system must achieve to support an organization's strategic business needs. They explain *why* the system is being built and what impact it should have on the business, for example 'reduce the time taken to settle an insurance claim from ten days to two'. Business requirements are not implemented directly; they are refined into the functional and non-functional requirements that together achieve them. Every functional requirement should therefore be traceable to a business requirement. A function that supports no business goal is a candidate for removal.

The two main categories of software requirement are:

1. *Functional requirements* These are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations. In some cases, the functional requirements may also explicitly state what the system should not do.
2. *Non-functional requirements* These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole rather than to individual system features or services.

In reality, the distinction between the two is not as clear-cut as these definitions suggest. A user requirement concerned with security, such as a statement limiting access to authorized users, looks like a non-functional requirement. When it is developed in more detail, however, it generates requirements that are clearly functional, such as the need to provide a login facility and to record who accessed each record. Requirements are not independent, and one requirement often generates or constrains others.

4.1.1 Functional requirements

The functional requirements for a system describe what the system should do. They depend on the type of software being developed, the expected users of the software, and the general approach taken by the organization when writing requirements. Functional requirements focus on user tasks, data processing, and system features such as calculations and data manipulation. They are derived directly from user needs and business objectives. Examples are:

1. The system shall generate a monthly sales report for each branch.
2. A user shall be able to search for a customer by customer ID.
3. The system shall send an e-mail confirmation to the customer when an order is dispatched.

Functional requirements are essential. Without them the system does not do its basic job, and a system that is missing a required function has failed, however well it performs the functions it does have. They are checked by functional testing (unit, system, and acceptance testing) that shows each feature working as intended.

In principle, the functional requirements specification of a system should be both complete and consistent. *Completeness* means that all services required by the user should be defined. *Consistency* means that requirements should not have contradictory definitions. In practice, for large, complex systems, it is practically impossible to achieve both, partly because of the size of the system and partly because stakeholders have different, and often inconsistent, needs. Inconsistencies may not be obvious when the requirements are first specified; they are discovered during validation (Section 4.6) or even after the system has been delivered.

4.1.2 Non-functional requirements

Non-functional requirements, as the name suggests, are requirements that are not directly concerned with the specific services delivered by the system to its users. They define *how* the system must perform: its quality attributes and operational limits, such as reliability, response time, security, storage, and scalability. They are derived from technical constraints, budgets, regulations, or the need to work with other systems. Examples are:

1. The system shall display any report within 3 seconds of the request.
2. The database shall support 10,000 concurrent users.
3. All payment data shall be encrypted when stored and when transmitted.

Non-functional requirements are often more critical than individual functional requirements. Users can usually find ways to work around a system function that does

not quite meet their needs, but failing to meet a non-functional requirement can make the whole system unusable. If an aircraft system does not meet its reliability requirements, it will not be certified as safe; if a web store takes a minute to load each page, customers will leave. The system may work, but it is effectively useless. Non-functional requirements are checked by performance testing, security audits, and by measuring the delivered system against benchmarks, for example 'load time < 2 s'. This is why they should be stated quantitatively wherever possible: 'the system should be fast' cannot be tested, whereas 'the system shall respond within 2 seconds for 95% of requests' can.

Figure 4.2 is a classification of non-functional requirements. You can see from this diagram that non-functional requirements may come from required characteristics of the software (product requirements), from the organization developing the software (organizational requirements), or from external sources:

1. *Product requirements* These requirements specify or constrain the behavior of the software. Examples include performance requirements on how fast the system must execute and how much memory it requires (efficiency), reliability requirements that set out the acceptable failure rate (dependability), security requirements, and usability requirements.
2. *Organizational requirements* These are broad system requirements derived from policies and procedures in the customer's and developer's organizations. Examples include operational process requirements that define how the system will be used, development process requirements that specify the programming language, the development environment, or process standards to be used, and environmental requirements that specify the operating environment of the system.
3. *External requirements* This broad heading covers all requirements that are derived from factors external to the system and its development process. These may include regulatory requirements that set out what must be done for the system to be approved for use by a regulator, such as a central bank; legislative requirements that must be followed to ensure that the system operates within the law; and ethical requirements that ensure that the system will be acceptable to its users and the general public.

The difference between the two kinds of requirement can be summarized by asking what each one describes. A functional requirement names a behavior: it can be demonstrated by giving the system an input and observing the output. A non-functional requirement names a property: it is observed by measuring the system over many inputs, many users, or a long period of time.

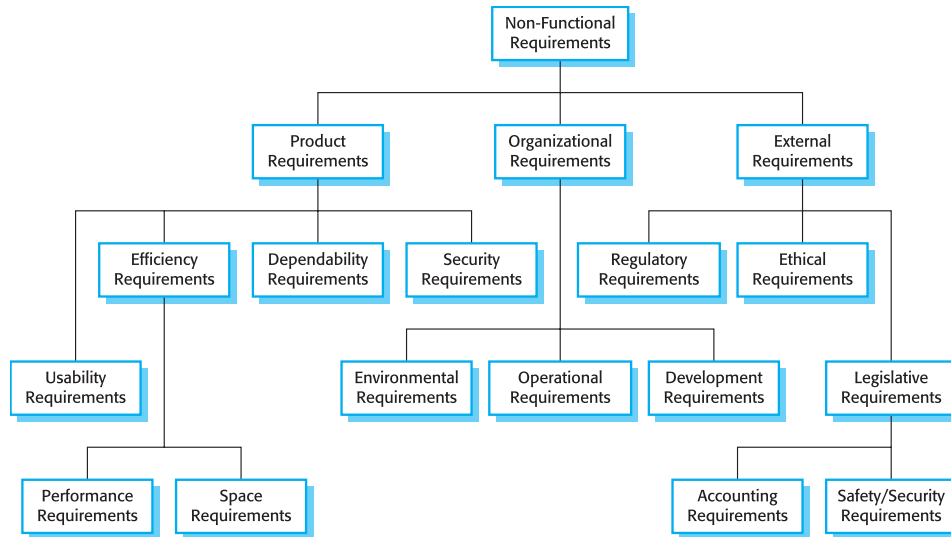


Figure 4.2 Types of non-functional requirement

Example 4.1 Classifying the requirements for an antivirus product

A company has asked for antivirus software with the following requirements: (1) core detection capabilities, using signature-based, heuristic, and machine-learning detection; (2) real-time monitoring and protection of both the file system and web traffic; (3) automatic malware removal; (4) low resource consumption; (5) daily updates from the vendor's server; (6) an attractive user interface; (7) code security; (8) compliance with legal requirements. Classify each requirement.

Solution A requirement is functional if it names a service that the product performs on an input, and non-functional if it names a property or constraint of the product as a whole.

1. *Detection* (signature, heuristic, and machine-learning) is functional. Each technique is a distinct service: given a file, it reports whether the file is malicious.
2. *Real-time monitoring and protection* is functional: the product must intercept file and web events and scan them. It carries a performance constraint (scanning must not noticeably delay the user), which should be stated separately.
3. *Automatic malware removal* is functional: quarantine or delete an infected file and repair what it changed.
4. *Low resource consumption* is a non-functional product requirement of the efficiency type (performance and space). It must be made measurable, for example 'CPU use below 5% and memory below 150 MB during background scanning'.

5. *Daily updates from the server* is functional (download and install signature and model updates) with a dependability constraint on the update service.
6. *An attractive user interface* is a non-functional product requirement of the usability type. As written it cannot be tested and should be restated, for example 'a first-time user can start a full scan within 30 seconds without help'.
7. *Code security* is a non-functional product requirement of the security type: the product itself must resist tampering, for example by signing its update packages and protecting its own processes.
8. *Legal considerations* is a non-functional external requirement of the legislative type: data protection law on the handling of scanned user files, and licensing of any third-party detection engines.

Answer Requirements 1, 2, 3, and 5 are functional. Requirements 4 (efficiency), 6 (usability), and 7 (security) are non-functional product requirements, and requirement 8 is a non-functional external (legislative) requirement. Requirements 4 and 6 must be rewritten in measurable form before they can be verified. ■

Example 4.2 Requirement types for a university ERP system

A university plans an enterprise resource planning (ERP) system to streamline its operations: admissions, course registration, examinations and grades, fees, and staff administration. Explain and identify (i) the user requirements, (ii) the system requirements, (iii) the direct and indirect stakeholders, (iv) two functional requirements, and (v) two non-functional requirements.

Solution (i) *User requirements* are high-level statements, in the language of the users, of the services the ERP must provide: 'Students shall be able to register for courses and see their grades online'; 'Staff shall be able to record fee payments and issue receipts'; 'Faculty shall be able to enter marks for the courses they teach'.

(ii) *System requirements* refine each user requirement into a precise statement of what is to be implemented: 'The system shall allow a student to register for a course only if the prerequisites are satisfied, the seat limit has not been reached, and the registration window is open; it shall record the registration with a timestamp and e-mail a confirmation.' Quality attributes such as speed or availability are not user or system requirements in themselves; they are non-functional requirements, which appear at both levels.

(iii) *Direct stakeholders* are students, faculty, administrative staff (admissions, examinations, accounts), and the IT staff who operate the system. *Indirect stakeholders* are the university management, parents, regulators and accreditation bodies, external auditors, and banks that process fee payments.

(iv) *Functional requirements*: FR1, a student shall be able to register for the courses offered in the coming semester; FR2, the system shall compute each student's grade point average from the marks entered and generate a grade sheet.

(v) *Non-functional requirements*: NFR1 (performance), the system shall respond to 95% of requests within 3 seconds with 5,000 concurrent users during registration week; NFR2 (security), students shall be able to view only their own records, and all access to marks shall be logged.

Answer User requirements: natural-language service statements for students, faculty, and staff. System requirements: detailed, implementable refinements of them. Direct stakeholders: students, faculty, administrative and IT staff; indirect: management, parents, regulators, auditors, banks. FRs: course registration and grade computation. NFRs: measurable performance and security. ■

Example 4.3 Functional and non-functional requirements from a scenario

Identify the functional and non-functional requirements for each system. (a) An online examination system in which students register, log in, attend examinations, and view results; faculty create question papers, schedule examinations, and evaluate answers; and the system must support many simultaneous users, evaluate accurately, process results quickly, and maintain data integrity. (b) A project management tool for teams distributed across time zones and continents, for planning projects, assigning tasks, tracking progress, and collaborating (three functional and two non-functional requirements). (c) An online event-ticketing platform on which users find events, buy tickets, and enter the venue with a QR code, and organizers create and manage events (five functional requirements and one non-functional requirement for each of security, usability, reliability, recoverability, and portability).

Solution Functional requirements are found from the verbs in the scenario (what each actor does with the system); non-functional requirements are found from the qualities it must have, which are then made measurable.

(a) *Functional*: students register and log in; take an examination within its scheduled time; view their results. Faculty create question papers from a question bank; schedule examinations; evaluate answers, with objective questions marked automatically. The system publishes the results. *Non-functional*: *performance and scalability*, 2,000 students can take an examination at once with page responses under 2 seconds; *accuracy*, automatic marking agrees with the answer key in 100% of objective questions; *timeliness*, results are available within 24 hours of the examination; *integrity and security*, answers are saved every 30 seconds and cannot be changed after submission, and question papers are encrypted until the start time.

(b) *Functional*: create a project plan with milestones and dependencies; assign tasks to members with due dates; update and track task status on a shared board. *Non-functional*: *availability*, 99.9% uptime, because some team is always working; *usability across locales*, all times shown in each user's local time zone and the interface usable in the team's languages.

(c) *Functional*: register and log in; search events by location, date, and category; buy tickets with a card or digital wallet; receive a confirmation with a QR code; organizers create events and track ticket sales. *Non-functional*: *security*, payment data

is encrypted in transit and never stored in full; *usability*, a ticket can be bought in at most three steps on a phone; *reliability*, 99.9% availability and no double-selling of a seat during peak sales; *recoverability*, data is backed up hourly and service is restored within one hour of a failure; *portability*, the platform works on current desktop and mobile browsers and on Android and iOS.

Answer Each answer lists the actions of each actor as functional requirements and states each quality as a measurable non-functional requirement: performance, accuracy, timeliness, and integrity for (a); availability and locale usability for (b); security, usability, reliability, recoverability, and portability for (c). ■

4.2 The software requirements document

The software requirements document, usually called the software requirements specification or SRS, is an official statement of what the system developers should implement. It should include both the user requirements for a system and a detailed specification of the system requirements. The process of requirements specification produces this structured document. It defines the data, functional, and behavioral elements of the system unambiguously, using models where necessary to show information flow, events, states, and constraints.

Requirements documents are essential when an outside contractor is developing the software, and they are useful whenever a system is large, long-lived, or critical. Agile methods argue that requirements change so rapidly that a detailed document is out of date as soon as it is written, and they use lighter-weight approaches (Section 4.7.3). Even then, a short document that sets out the business and dependability requirements is valuable, because requirements that apply to the system as a whole are easily forgotten when attention is on the next increment.

A requirements document has a diverse set of readers, from the senior managers of the customer organization who pay for the system to the engineers who develop, test, and maintain it. For this reason it should cover a broad range of topics. The subjects that an SRS must address are:

1. *Functionality* What is the software supposed to do?
2. *External interfaces* How does the software interact with people, the system's hardware, other hardware, and other software?
3. *Performance* What is the speed, availability, response time, and recovery time of the software functions?
4. *Quality attributes* What are the reliability, maintainability, portability, and security considerations?
5. *Constraints* Are there required standards, policies, implementation languages, database integrity rules, resource limits, or operating environments?

6. *Safety* Are there hazards that the software must avoid or control?

Figure 4.3 shows one widely used organization of a requirements document, based on the IEEE standard for requirements documents. The standard is generic, and each organization adapts it to its own needs. The specific requirements in part 4 are normally the largest and most important part of the document, and they may be organized in different ways: by feature, by class of user, or by mode of operation.

Figure 4.3 The structure of a software requirements specification

Part	Description
1. Introduction	Purpose of the document and its intended readers; scope of the product; definitions, acronyms, and abbreviations; references; an overview of how the rest of the document is organized.
2. Overall description	Product perspective (how the system fits with other systems); a summary of product functions; user classes and their characteristics; the operating environment; design and implementation constraints; assumptions and dependencies.
3. External interface requirements	User interfaces, hardware interfaces, software interfaces, and communication interfaces.
4. System features (specific requirements)	The detailed functional requirements, organized by feature, by user class, or by mode of operation. This is normally the largest and most important part of the document.
5. Other non-functional requirements	Performance, safety, and security requirements; software quality attributes; business rules.
Supporting information	Glossary, analysis models, a list of items still to be decided, appendices (for example sample input and output formats), and an index.

4.2.1 Properties of a good specification

A requirements document is used as the basis for design, for testing, and often for a contract. It can only serve these purposes if its requirements have certain properties. A good SRS is:

1. *Correct* Every requirement stated is one that the software shall meet; nothing in it contradicts the real needs of the stakeholders.
2. *Unambiguous* Every requirement has only one interpretation. Natural language is inherently ambiguous, so terms are defined in a glossary and vague words such as ‘fast’, ‘user-friendly’, and ‘flexible’ are avoided.
3. *Complete* All significant requirements are included, together with the responses of the software to all realizable classes of input, both valid and invalid.
4. *Consistent* No subset of the requirements conflicts with another, for example one requirement that a report is printed daily and another that it is printed weekly.

5. *Ranked for importance and stability* Each requirement is marked as essential, desirable, or optional, and as stable or likely to change, so that effort and change control can be planned.
6. *Verifiable* For every requirement there is a finite, cost-effective process by which a person or a machine can check that the software meets it.
7. *Modifiable* Its structure and style allow changes to be made easily, completely, and consistently; each requirement is stated once and not repeated.
8. *Traceable* The origin of each requirement is clear, and each requirement can be referenced by later documents such as designs and test cases.

Requirements that fail these tests cause predictable problems. Requirements that are inconsistent, incomplete, not aligned with the business goals, not implementable, or incomprehensible lead to rework during development or to a delivered system that does not meet its users' needs. Good requirements, by contrast, are clear, necessary, complete, comprehensible, testable, and trackable from their source through to the code and tests that implement them.

4.3 Requirements specification

Requirements specification is the process of writing down the user and system requirements in a requirements document. Ideally, the user and system requirements should be clear, unambiguous, easy to understand, complete, and consistent. In practice this is difficult to achieve, because stakeholders interpret requirements in different ways and there are often inherent conflicts and inconsistencies in the requirements.

User requirements are almost always written in natural language, supplemented by simple diagrams and tables. System requirements may also be written in natural language, but other notations based on forms, graphical system models, or mathematical models can be used. Graphical models, which are described in Chapter 5, are most useful when you need to show how a state changes or when you need to describe a sequence of actions.

4.3.1 Natural language specification

Natural language is expressive, intuitive, and universal. It is also potentially vague and ambiguous, and its interpretation depends on the background of the reader. To minimize misunderstandings, the following simple guidelines are useful:

1. Invent a standard format and ensure that all requirement definitions adhere to it. Standardizing the format makes omissions less likely and requirements easier to check. Each requirement should be stated in a single sentence where possible.

2. Use language consistently to distinguish between mandatory and desirable requirements. Mandatory requirements use 'shall'; desirable requirements use 'should'.
3. Use text highlighting (bold or italic) to pick out key parts of the requirement.
4. Do not assume that readers understand technical software engineering language. Avoid jargon, abbreviations, and acronyms.
5. Whenever possible, associate a rationale with each requirement. The rationale explains why the requirement has been included, and it is particularly useful when requirements are changed, because it helps to decide what changes would be undesirable.

Example 4.4 Rewriting requirements so that they can be verified

The following statements were collected for an online store. Explain what is wrong with each and rewrite it. (a) 'The system should be easy to use.' (b) 'The site must be fast and must be available at all times.' (c) 'The system shall store customer details and should allow them to be changed.'

Solution (a) is not verifiable: 'easy to use' cannot be measured. A usability requirement must name a user group, a task, and a measure.

(b) combines two requirements in one sentence, and neither is measurable. 'Available at all times' is also unrealistic, since no system can promise 100% availability.

(c) is compound and mixes a mandatory verb with a desirable one, so it is unclear whether changing details is required.

Answer (a) *Usability* A first-time customer shall be able to find a product and complete a purchase in under 4 minutes without help, in at least 90% of trials.

(b) *Performance* Product pages shall load within 2 seconds for 95% of requests at a load of 2,000 concurrent users. *Availability* The store shall be available for at least 99.5% of each calendar month.

(c) R1: The system shall store the name, address, e-mail, and phone number of each registered customer. R2: A registered customer shall be able to change their stored details after logging in. ■

4.4 Requirements engineering processes

The requirements engineering process is the systematic process of discovering, analyzing, documenting, and checking the services that a system should provide and the constraints under which it must operate. Figure 4.4 shows its main activities and the documents they produce. A feasibility study produces a feasibility report; elicitation and analysis produce system models; specification produces the user and system requirements; and validation checks them before they are collected into the

requirements document. Management of requirements runs through all of these stages, because requirements change throughout.

The stages, their purposes, and the techniques used in each are as follows:

1. *Feasibility study* The purpose is to assess the technical, economic, and operational feasibility of the proposed system: can it be built with available technology, will its benefits outweigh its costs, and will it fit the way the organization works? Activities include market analysis, technology assessment, risk analysis, and cost-benefit analysis. The result is a recommendation on whether to continue.

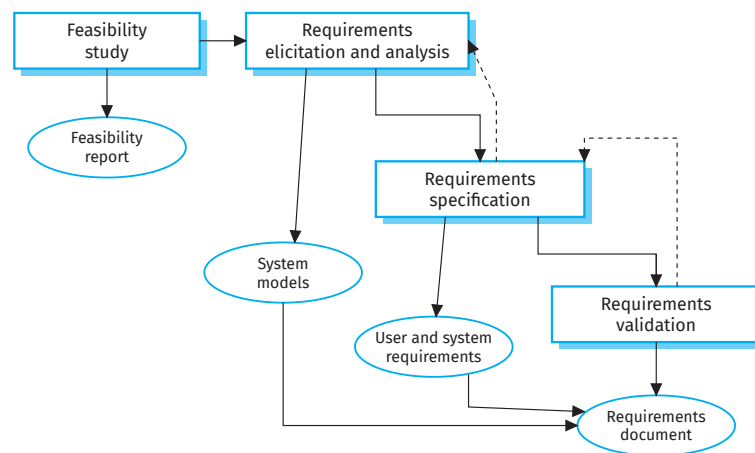


Figure 4.4 The requirements engineering process

2. *Requirements elicitation* The purpose is to gather information about the needs and expectations of stakeholders: users, customers, and domain experts. Techniques include interviews, surveys, observation, focus groups, workshops, and prototyping (Section 4.5).
3. *Requirements analysis* The purpose is to analyze the elicited information, identify conflicts, and prioritize requirements. Techniques include use case analysis (describing the interactions between users and the system), data-flow diagrams (visualizing the flow of data through the system), requirement decomposition (breaking complex requirements into smaller, manageable ones), and risk analysis (identifying potential risks and developing mitigation strategies).
4. *Requirements specification* The purpose is to document the requirements in a clear, concise, and unambiguous manner. The deliverable is the software requirements specification, which must include the functional requirements (what the system should do), the non-functional requirements (quality attributes such as performance, usability, and security), the user requirements (from the perspective of end-users), and the system requirements (from the perspective of the system developers).

4. *Requirements validation* The purpose is to ensure that the requirements are correct, complete, consistent, and feasible. Techniques include reviews and inspections, prototyping, and testing (Section 4.6).
5. *Requirements management* Throughout stages 1 to 5, requirements management tracks changes to requirements, manages conflicts, and ensures that the system remains aligned with the evolving needs of stakeholders. Its techniques are change control, version control, and traceability (Section 4.7).

The same work is sometimes described as seven overlapping tasks: *inception* (establishing a basic understanding of the problem, the stakeholders, and the nature of the solution), *elicitation*, *elaboration* (refining the information into an analysis model), *negotiation* (reconciling conflicting requirements), *specification*, *validation*, and *management*. The list is different, but the idea is the same: the requirements move from vague business needs to a precise, checked, and controlled description.

Example 4.5 Requirements engineering tasks for a health-monitoring system

A team is to build a monitoring system for elderly patients with chronic illnesses: real-time health monitoring, emergency alerts, medication reminders, and doctor–patient communication. The hospital management has given only broad objectives. Evaluate how the tasks of requirements engineering apply to this system.

1. *Inception* The team establishes the problem, the stakeholders (patients, carers, doctors, nurses, hospital management, emergency services, and regulators), and the scope: which illnesses and which vital signs are monitored.
2. *Elicitation* Interviews with doctors, observation of nurses on the wards, workshops with management, and prototypes of the reminder screens shown to patients reveal detailed needs, such as the heart-rate limits that trigger an alert.
3. *Elaboration* The information is refined into models: use cases (raise emergency alert), scenarios, and data models of patients, readings, and prescriptions.
4. *Negotiation* Conflicts are resolved and requirements prioritized. Doctors want frequent readings; patients want long battery life. Emergency alerts are given the highest priority.
5. *Specification* The agreed requirements are written into an SRS, with measurable non-functional requirements, such as ‘an emergency alert reaches the duty doctor within 10 seconds’, and privacy constraints.
6. *Validation* A review team of clinicians and engineers checks the SRS for correctness, completeness, consistency, and feasibility, and a prototype confirms that elderly users can respond to reminders.
7. *Management* Requirements are given identifiers and traced to design and tests, and changes, such as a new vital sign, pass through change control. ■

These activities are not carried out once in strict sequence. In practice, requirements engineering is an iterative process in which the activities are interleaved. The amount of time and effort devoted to each activity in each iteration depends on the stage of the overall process and on the type of system being developed. Early in the process, most effort is spent on understanding high-level business and non-functional requirements and the user requirements for the system. Later, in the outer rings of the spiral, more effort is devoted to eliciting and understanding the detailed system requirements.

4.5 Requirements elicitation and analysis

After an initial feasibility study, the next stage of the requirements engineering process is requirements elicitation and analysis. Software engineers work with customers and system end-users to find out about the application domain, what services the system should provide, the required performance of the system, hardware constraints, and so on. Elicitation has four objectives:

1. to understand user expectations;
2. to identify the functional and non-functional requirements;
3. to prevent miscommunication between the stakeholders and the development team; and
4. to ensure that the project deliverables align with business goals.

A process model of elicitation and analysis is shown in Figure 4.5. The activities are:

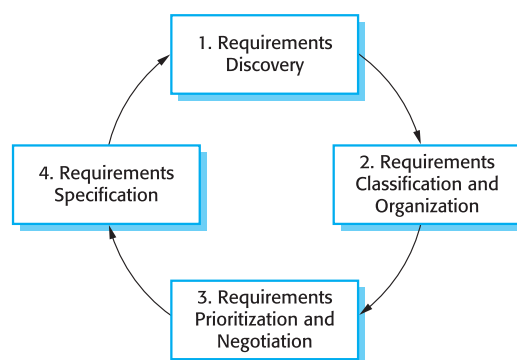


Figure 4.5 The requirements elicitation and analysis process

1. *Requirements discovery* This is the process of interacting with stakeholders of the system to discover their requirements. Domain requirements from stakeholders and documentation are also discovered during this activity.

2. *Requirements classification and organization* This activity takes the unstructured collection of requirements, groups related requirements, and organizes them into coherent clusters.
3. *Requirements prioritization and negotiation* Inevitably, when multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation.
4. *Requirements specification* The requirements are documented and input into the next round of the spiral.

Elicitation is difficult. Stakeholders often do not know what they want from a computer system except in the most general terms; they express requirements in their own terms and with implicit knowledge of their own work; different stakeholders have different requirements, which they may express in different ways; and the business environment may change during the analysis process.

4.5.1 Elicitation techniques

No single technique is sufficient for every system. The following techniques are commonly combined:

1. *Interviews* One-to-one or group discussions with stakeholders. *Closed* interviews use a predetermined list of questions; *open* interviews explore issues without a fixed agenda. Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system, but they are less effective for understanding domain requirements, which stakeholders find hard to articulate.
2. *Questionnaires and surveys* Written questions sent to many stakeholders. They reach large, dispersed user groups cheaply but give little opportunity to follow up an unexpected answer.
3. *Workshops and brainstorming* Collaborative sessions in which stakeholders and developers generate and refine requirements together. They are effective at surfacing conflicts early and building agreement.
4. *Prototyping* An early, partial version of the system, or a mock-up of its interface, is shown to users to obtain feedback. Users who cannot describe what they want can often say at once what is wrong with a prototype.
5. *Observation and ethnography* An analyst immerses themselves in the working environment and observes the day-to-day work. Observation reveals how people actually work, which is often different from how formal procedures say they work, and it uncovers requirements that derive from cooperation between people.

6. *Document analysis* Existing forms, reports, manuals, regulations, and the documentation of current systems are studied to extract requirements, particularly domain requirements.
7. *Introspection* The analyst imagines what they would want if they were the user. It is quick but reliable only when the analyst has deep domain knowledge, and its results must be checked with real users.
8. *Use cases and scenarios* Real-life examples of how the system will be used are written down or modeled. People find it easier to relate to concrete examples than to abstract descriptions, so scenarios are an effective way of discovering the details of interactions (Chapter 5).
9. *Storyboarding and walkthroughs* Sequences of sketches of the user interface are walked through with stakeholders to check the flow of a task.
10. *Facilitated application specification technique (FAST)* A structured meeting of customers and developers, run by a neutral facilitator with agreed rules and an agenda. Before the meeting each participant lists objects, services, constraints, and performance criteria; the lists are combined, conflicts are discussed, and a joint specification is drafted.

The choice depends on the stakeholders and the system. Interviews and document analysis are nearly always used; observation is valuable where work practices are complex or tacit; prototyping and storyboarding are most useful for interactive systems whose users have difficulty describing what they need.

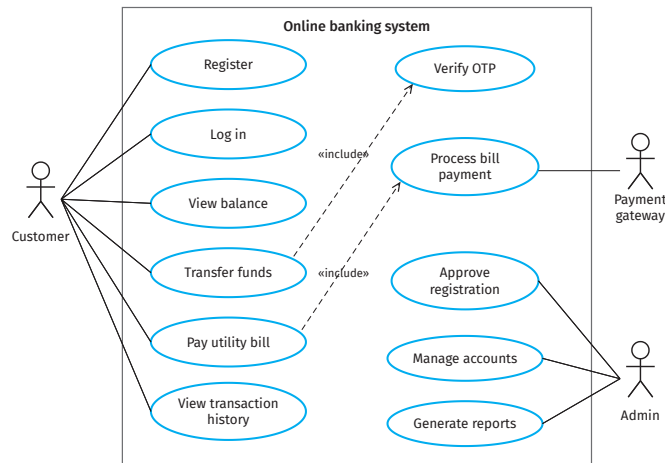
Example 4.6 Use cases of an online banking system

Customers of an online banking system register, log in, view their balance, transfer funds, pay utility bills through an external payment gateway, and view their transaction history; an administrator manages accounts, approves registrations, and generates reports. Draw a use case diagram.

Answer Using the notation of Section 5.2.1, the actors are the Customer and the Admin (primary actors) and the Payment gateway (a secondary actor, an external system) (Figure 4.6). The Customer is associated with Register, Log in, View balance, Transfer funds, Pay utility bill, and View transaction history; the Admin with Approve registration, Manage accounts, and Generate reports. Pay utility bill always «include»s Process bill payment, which is associated with the Payment gateway, and Transfer funds always «include»s Verify OTP. Log in is a precondition of the other use cases, not a step inside them, so it is not «include»d by each of them.



Figure 4.6 Use cases of an online banking system



Example 4.7 Eliciting requirements from a busy customer

A customer says they are too busy to meet the analyst. Identify two suitable elicitation techniques and justify them, with the process and an example.

1. *Document analysis* needs none of the customer's time at first. The analyst studies the customer's existing forms, reports, procedures, and the current system, and drafts the requirements from them. For an order-processing system, the order forms, invoices, and returns policy give the data fields, the business rules (discount limits, credit checks), and the reports required.
2. *Questionnaires* (or a short, structured e-mail interview) let the customer answer at a convenient time, in a few minutes. The analyst sends focused, closed questions derived from the document analysis ('Must an order over ₹50,000 be approved by a manager?'), and follows up only on the answers that are unclear.

Observation of the customer's staff at work, and a prototype sent for comment, are also suitable, because they use the time of the staff or only a few minutes of the customer's. The drafted requirements are then validated in one short review meeting, which is the only meeting the customer has to attend. ■

4.6 Requirements validation

Requirements validation is the process of checking that requirements actually define the system that the customer really wants. It overlaps with analysis, since it is concerned with finding problems with the requirements. Validation verifies that the software requirements are complete, consistent, and accurately reflect stakeholder needs, and it detects errors early in order to reduce costly rework. During validation, stakeholders systematically check the requirements model for proper reflection of information, function, and behavior. The specification is examined to ensure that:

1. all software requirements have been stated unambiguously;
2. inconsistencies, omissions, and errors have been detected and corrected; and
3. the work products conform to the standards established for the process, the project, and the product.

Requirements validation is important because errors in a requirements document can lead to extensive rework costs when these problems are discovered during development or after the system is in service. The cost of fixing a requirements problem by making a system change is usually much greater than repairing design or coding errors, because a change to the requirements usually means that the system design and implementation must also be changed, and the system must then be tested again.

4.6.1 Validation checks

During the requirements validation process, different types of checks should be carried out on the requirements in the requirements document:

1. *Validity checks* A user may think that a system is needed to perform certain functions. However, further thought and analysis may identify additional or different functions that are required. Validity checks ensure that the system performs the functions needed across the user community.
2. *Consistency checks* Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.
3. *Completeness checks* The requirements document should include requirements that define all intended functions and the constraints on them.
4. *Realism checks* Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented within the proposed budget and schedule.
5. *Verifiability* To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

4.6.2 Validation techniques

There are a number of requirements validation techniques that can be used individually or in conjunction with one another:

1. *Requirements reviews* A group of people from the contractor and the client carefully review the SRS document to identify errors, ambiguities, and inconsistencies. The systematic analysis ensures that the requirements are verifiable and traceable. Estimates of defect density guide how thorough a review must be: if earlier reviews of similar documents found around 0.6 errors per page, a review of a 50-page SRS that finds only three errors has probably not been thorough enough.
2. *Prototyping* An executable model of the system is demonstrated to end-users and customers. They can experiment with this model to see if it meets their real needs. The technique is particularly effective for collecting user feedback and for validating assumptions early.
3. *Test-case generation* Requirements must be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems: if a test is difficult or impossible to design, the requirement will be difficult to implement and should be reconsidered. Requirements-based testing also shows that the system performs as the customer expects and supports risk management.
4. *Automated consistency analysis* If the requirements are expressed as a system model in a structured or formal notation, tools can check the model automatically for conflicts, contradictions, and inconsistencies. This improves both the accuracy and the efficiency of validation.
5. *Walkthroughs* An informal review without a formally defined procedure. Walkthroughs are used early to check feasibility, to obtain opinions, and to reach agreement among stakeholders.
6. *Simulation* The behavior of the system is replicated in a realistic setting to check that the requirements meet their goals. Simulation is especially effective for complex and real-time systems whose behavior is hard to imagine from a document.

You should not underestimate the problems involved in requirements validation. It is difficult to show that a set of requirements does in fact meet a user's needs, because users must picture the system in operation and imagine how it would fit into their work. As a result, you rarely find all requirements problems during validation, and further changes to correct omissions and misunderstandings will be needed after the requirements document has been agreed upon.

Example 4.8 Validating the requirements for an online fashion store

StyleSphere is a start-up e-commerce platform selling sustainable and ethically sourced fashion to environmentally conscious young customers. Its application is struggling with customer acquisition, building trust, and managing its supply chain, and a new version is planned with many changed requirements (see Example 4.9). Is it necessary to apply requirements validation techniques before the new version is built? Justify your answer with a suitable example.

Solution Validation is necessary. The new requirements come from several sources at once (marketing, supply-chain partners, legal advisers, and customer feedback), so they are especially likely to conflict, to be incomplete, or to be unrealistic. Requirements are finalized only after validation: the specification must be examined to ensure that every requirement is stated unambiguously, that inconsistencies, omissions, and errors have been found and corrected, and that the document conforms to the project's standards.

Consider two proposed requirements: 'Every product page shall show live stock levels from all partner suppliers' and 'Every product page shall load within 2 seconds'. A consistency and realism check shows that they may conflict, since querying several supplier systems on every page view could make pages slow. A third requirement, 'Every product shall display a verified sustainability certificate', fails a realism check if some small suppliers have no certification.

The validation techniques apply as follows:

1. A *requirements review* by marketing, supply-chain, and legal staff finds the stock/performance conflict and the certification gap. It is resolved by caching supplier stock every 15 minutes and by labelling uncertified items differently.
2. A *prototype* of the new product page, shown to a sample of target customers, checks whether the trust features (impact score, reviews, supplier story) actually increase confidence before they are built.
3. *Test-case generation* for 'the impact score shall be accurate' shows that the requirement cannot be tested until the formula and its data sources are defined, so the requirement is rewritten.
4. *Automated consistency analysis* of the structured requirements checks that every changed requirement is traced to its source and does not contradict another requirement.

Answer Yes. Because the changed requirements come from many stakeholders, validation is needed to find conflicts (live supplier stock versus a 2-second page load), unrealistic requirements (certification for every product), and untestable requirements (an undefined 'accurate' impact score) before they are designed and coded. Reviews, prototyping, test-case generation, and automated consistency analysis are the techniques to use; errors removed at this stage are far cheaper than errors found after release. ■

4.7 Requirements management

The requirements for large software systems are always changing. One reason for this is that these systems are usually developed to address ‘wicked’ problems, that is, problems that cannot be completely defined. Because the problem cannot be fully defined, the software requirements are bound to be incomplete. During the software process, the stakeholders’ understanding of the problem is constantly changing. The system requirements must then also evolve to reflect this changed problem view.

Once a system has been installed and is regularly used, new requirements inevitably emerge. It is hard for users and system customers to anticipate what effects the new system will have on their business processes and the way that work is done. Once end-users have experience of a system, they will discover new needs and priorities.

4.7.1 Why requirements change

Requirements change for many reasons. The main *sources* of change, and the *factors* that drive each one, are:

1. *The business and technical environment* After installation the hardware may change, the system may have to interface with other systems, business priorities may change, and new legislation and regulations may be introduced. Competition, new markets, and pressure on revenue are the usual factors.
2. *Customers who are not the users* The people who pay for a system and the people who use it are often different. Customers impose requirements because of organizational and budgetary constraints, and these may conflict with end-user requirements; after delivery, new features may have to be added for user support if the system is to meet its goals.
3. *Diverse stakeholders* Large systems have many stakeholders with different, and often contradictory, requirements. The final requirements are inevitably a compromise, and the balance of support given to different users may have to change as experience grows.
4. *Improved understanding* As development proceeds, developers and stakeholders understand the problem better. Feedback from prototypes and from early releases reveals requirements that nobody had thought of.
5. *Other systems and processes* Requirements that depend on external systems, devices, or organizational processes change when those systems and processes change.

From an evolution perspective, requirements fall into two classes. *Enduring requirements* are relatively stable requirements that derive from the core activity of the organization; a hospital will always have doctors, nurses, and patients. *Volatile*

requirements are likely to change during the development process or after the system has been put into operation. There are four types of volatile requirement:

1. *Mutable requirements* These are requirements that change because of changes to the environment in which the organization is operating. For example, in hospital systems, the funding of patient care may change and so require different treatment information to be collected.
2. *Emergent requirements* These are requirements that emerge as the customer's understanding of the system develops during the system development. The design process may reveal new emergent requirements.
3. *Consequential requirements* These are requirements that result from the introduction of the computer system. Introducing the computer system may change the organization's processes and open up new ways of working that generate new system requirements.
4. *Compatibility requirements* These are requirements that depend on the particular systems or business processes within an organization. As these change, the compatibility requirements on the commissioned or delivered system may also have to evolve.

Example 4.9 Changing requirements for an online fashion store

StyleSphere is a start-up e-commerce platform focused on sustainable and ethically sourced fashion, aimed at environmentally conscious customers looking for unique, high-quality clothing. The company runs its own e-commerce application but is struggling with customer acquisition, building trust, and its supply chain. Identify the sources of changing requirements and the factors behind them, explain the types of volatile requirement, and state the modified requirements for the new version of the application.

Solution *Sources of change and their factors.* Each problem the business faces becomes a source of new or changed requirements:

1. *Business environment (customer acquisition).* Factors: competition from larger fashion platforms, falling revenue, and marketing costs. The niche (sustainable fashion) must be made visible, so content, referral, and social-sharing features are needed.
2. *Customers' understanding and priorities (trust).* Factors: customer feedback, abandoned carts, and poor reviews. Customers doubt sustainability claims, so the product pages must show evidence.
3. *Other systems and processes (supply chain).* Factors: new partner suppliers and couriers with their own systems; stock-outs and late deliveries.
4. *Legislation and regulation.* Factors: consumer-protection rules on environmental claims ('green claims'), data-protection law, and changes in tax rules for online

sales.

5. *Data and experience from the running system.* Factors: analytics showing where visitors leave the site; requests from users of the current version.

Types of volatile requirement in this system:

1. *Mutable:* rules on what may be claimed as 'sustainable' and on tax for online sales change with the regulatory environment, so the product-information and checkout requirements must change with them.
2. *Emergent:* analysis of why visitors do not buy shows that they need proof of ethical sourcing; a requirement for a verified impact score and supplier story emerges only as this understanding develops.
3. *Consequential:* once the new version introduces a take-back and resale scheme for used clothing, customers need a way to request collection and track credit for returned items, requirements that exist only because the new system exists.
4. *Compatibility:* stock and shipment requirements depend on the partner suppliers' inventory systems and the couriers' tracking services; when those change their interfaces, StyleSphere's requirements must change too.

The core catalogue, cart, and checkout requirements are enduring and are not expected to change.

Answer Modified requirements for the new version:

1. (*Acquisition*) The system shall provide editorial content pages and a referral scheme giving the referrer and the new customer store credit on the first order.
2. (*Trust*) Each product page shall show the supplier, the materials, a sustainability certificate where one exists, and verified-buyer reviews.
3. (*Trust*) The system shall calculate and display an impact score for each product from a published formula.
4. (*Supply chain*) The system shall import stock levels from each partner supplier's system at least every 15 minutes and shall show 'dispatches in n days' based on that stock.
5. (*Supply chain*) The system shall display courier tracking for every dispatched order.
6. (*Regulation*) Environmental claims shall be displayed only where supported by a stored certificate.
7. (*Consequential*) Customers shall be able to request collection of used StyleSphere items and track the resulting store credit.

Requirement 1 addresses acquisition; 2 and 3 address trust; 4 and 5 address the supply chain; 6 addresses regulation and also supports trust; 7 introduces a new service. By type, 6 is mutable, 3 is emergent, 7 is consequential, and 4 and 5 are compatibility requirements. ■

4.7.2 Requirements management planning and change management

Requirements management is the process of understanding and controlling changes to system requirements. You need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. Its three main techniques are:

1. *Change control* A defined process through which every proposed change is submitted, analyzed, approved or rejected, and implemented.
2. *Version control* Different versions of the requirements document are maintained, so that it is always clear which version is current and what changed between versions.
3. *Traceability* Links are maintained between each requirement and its source, other requirements, and later artifacts such as design documents and test cases. Traceability lets you see how a change ripples through the system.

Planning is an essential first stage in the requirements management process. During planning you decide on how requirements are uniquely identified so that they can be cross-referenced; on the change management process; on the traceability policies that define the relationships to be recorded; and on the tool support to be used, which may range from specialist requirements management systems to spreadsheets and simple databases.

Requirements change management (Figure 4.7) should be applied to all proposed changes to a system's requirements after the requirements document has been approved. The advantage of using a formal process is that all change proposals are treated consistently and changes to the requirements document are made in a controlled way. There are three principal stages:

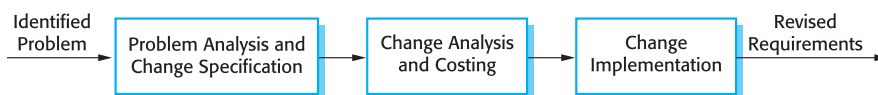


Figure 4.7
Requirements change management

1. *Problem analysis and change specification* The process starts with an identified requirements problem or, sometimes, with a specific change proposal. The problem or proposal is analyzed to check that it is valid, and the result is fed back to the person who asked for the change.

2. *Change analysis and costing* The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost of making the change is estimated in terms of modifications to the requirements document and, if appropriate, to the system design and implementation. A decision is then made whether or not to proceed.
3. *Change implementation* The requirements document and, where necessary, the system design and implementation are modified. The document should be organized so that changes can be made without extensive rewriting.

If a new requirement has to be urgently implemented, there is always a temptation to change the system and then retrospectively modify the requirements document. You should try to avoid this, as it almost inevitably leads to the requirements specification and the system implementation getting out of step.

4.7.3 Requirements in agile development

Agile processes take a different approach. Requirements are managed as small, negotiable items, often user stories, in a prioritized backlog that is continuously refined, implemented in short iterations, and validated through frequent stakeholder feedback. Instead of a single fixed SRS, agile development treats requirements as evolving assets that are detailed ‘just in time’ for implementation. Figure 4.8 shows the cycle.

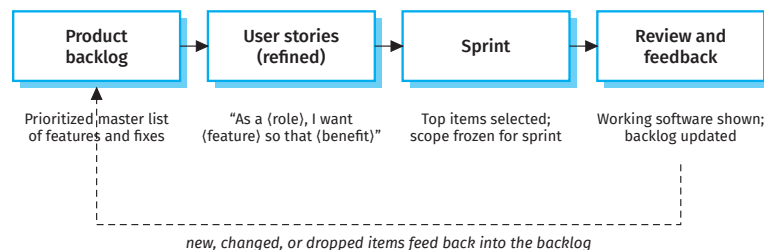


Figure 4.8
Requirements
engineering in agile
development

1. *Product backlog* The backlog is the master list containing all requests, features, and fixes. The rule is simple: if it is not on the list, it does not get built. The most important items (high value or high risk) are moved to the top; the bottom of the list remains vague and can change at any time.
2. *User stories* Requirements are written on small cards in the form ‘As a ⟨user role⟩, I want ⟨feature⟩ so that ⟨benefit⟩’. This keeps the focus on who needs a feature and why, rather than on how to code it. Each story is given acceptance criteria that say when it is done.
3. *Sprints* The whole list is not built at once; it is sliced into short, fixed time periods. In sprint planning, the team picks the top items from the backlog that

they can finish in one sprint. Once a sprint starts, new items are generally not added to it (the sprint is *frozen*), which lets the team concentrate.

4. *Feedback loop* At the end of each sprint the team shows the working software to the customer. The customer may say ‘this button is too small’ or ‘I don’t actually need this feature any more’. The backlog is immediately updated with this feedback: new ideas are added and unwanted items removed before the next sprint begins.

Large requirements are first recorded as *epics*. An epic is a high-level goal or large feature that spans several sprints; it is broad and deliberately vague, and it is refined later into a number of user stories. A user story is small enough to be completed in a single sprint, is detailed (with acceptance criteria), and belongs to an epic. In this way the epic is the parent and its stories are the children.

When a user proposes a requirements change in an agile process, the change does not go through a formal change management process. Rather, the user adds it to the backlog and prioritizes it, and it is scheduled into a later sprint if its priority is high enough.

Example 4.10 Splitting an epic into user stories

A library wants its members to be able to reserve books online. Write this as an epic and split it into user stories with acceptance criteria.

Solution The epic states the goal: ‘As a library member, I want to reserve books online so that I do not have to visit the library to check availability.’ It is too large for one sprint, because it involves searching, reserving, notification, and cancellation, so it is split into stories that can each be finished and demonstrated in a sprint.

1. As a member, I want to search the catalogue by title or author so that I can find a book. *Accepted when* results appear within 2 seconds and show availability.
2. As a member, I want to reserve a book that is on loan so that I get it when it is returned. *Accepted when* a reservation is recorded and shown in ‘My reservations’.
3. As a member, I want to be notified when a reserved book is ready so that I can collect it. *Accepted when* an e-mail is sent within 10 minutes of the book being returned.
4. As a member, I want to cancel a reservation so that the book goes to the next person. *Accepted when* the reservation disappears and the next member in the queue is promoted. ■

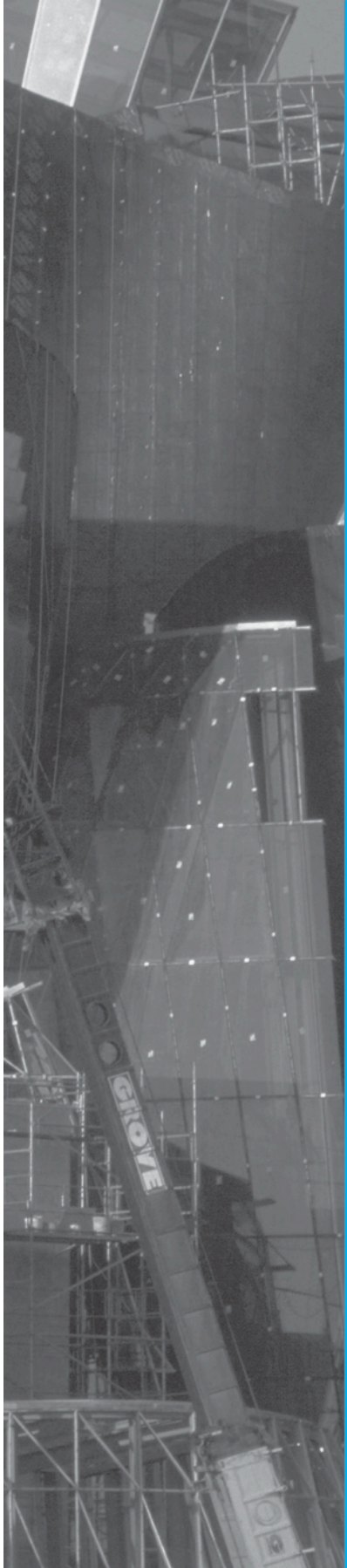
Example 4.11 User stories for a language-learning app

A language-learning app lets learners browse and enrol in courses, track their progress, take quizzes, and review completed courses. Write user stories with acceptance criteria for these features.

1. As a learner, I want to browse the courses offered next semester so that I can choose ones that fit my goals and schedule. *Accepted when* the list shows each course's title, level, instructor, schedule, and free seats.
2. As a learner, I want to enrol in the courses I select so that my place is secured. *Accepted when* I can review my selection before confirming, and a full course cannot be selected.
3. As a learner, I want to see my progress so that I stay motivated. *Accepted when* a dashboard shows lessons completed, quiz scores, and words learned.
4. As a learner, I want to set a goal with a target date so that I can work towards it. *Accepted when* the dashboard shows my progress towards the goal.
5. As a learner, I want to take a proficiency quiz so that I know which skills to improve. *Accepted when* the result reports a score for vocabulary, grammar, listening, and reading.
6. As a learner, I want to rate and review a course I have completed so that other learners can benefit. *Accepted when* I am prompted on completion and my rating appears on the course page. ■

KEY POINTS

- Requirements for a software system set out what the system should do and define constraints on its operation. Business requirements state why the system is needed; functional requirements state what it must do; non-functional requirements state how well it must do it.
- Non-functional requirements may be product, organizational, or external requirements. They often apply to the system as a whole and should be stated in measurable terms.
- The software requirements document is an agreed statement of the system requirements. A good specification is correct, unambiguous, complete, consistent, ranked, verifiable, modifiable, and traceable.
- The requirements engineering process includes a feasibility study, requirements elicitation and analysis, requirements specification, requirements validation, and requirements management. In practice these activities are interleaved in a spiral.
- Elicitation techniques include interviews, surveys, workshops, prototyping, observation, document analysis, introspection, scenarios, storyboarding, and FAST.
- Requirements validation checks requirements for validity, consistency, completeness, realism, and verifiability, using reviews, prototyping, test-case generation, automated consistency analysis, walkthroughs, and simulation.
- Requirements change because of changes in the business environment, conflicts between stakeholders, and improved understanding. Volatile requirements may be mutable, emergent, consequential, or compatibility requirements.
- Requirements management uses change control, version control, and traceability. Agile processes manage requirements as a prioritized backlog of user stories grouped into epics.



5

System modeling

Objectives

The aim of this chapter is to introduce the types of system model that are developed as part of the requirements engineering and system design processes. When you have read the chapter, you will

- understand why graphical models are used and the perspective of context, interaction, structure, and behavior from which a system may be modeled;
- be able to draw context diagrams and leveled data-flow diagrams that are balanced;
- know the UML diagram types and be able to draw use case, sequence, communication, class, activity, and state diagrams;
- understand how the scenario-based, class-based, behavioral, and flow-oriented elements together make up a requirements model.

Contents

- 5.1 Context models
- 5.2 Interaction models
- 5.3 Structural models
- 5.4 Behavioral models
- 5.5 Building a requirements model

System modeling is the process of creating abstract, visual representations of a system to understand, define, and communicate its requirements before implementation. Each model presents a different view or perspective of that system, and the models act as a bridge between vague user needs and precise technical specifications. They ensure that stakeholders and developers share a common understanding of what is to be built.

Models make structure, behavior, and interactions easier to understand than text alone; they clarify vague requirements and reduce miscommunication; they give developers, clients, and testers a common visual language; they support traceability and the generation of test cases and documentation; and they expose missing requirements and design flaws early, when they are cheap to fix. Above all, a model is an abstraction: it deliberately leaves out detail so that the engineer can concentrate on one perspective at a time.

A system may be modeled from five perspectives:

1. A *context* perspective, which models the environment of the system and its interactions with external entities.
2. An *interaction* perspective, which models how the system interacts with its users and how its components communicate.
3. A *structural* perspective, which models the organization of the system's components and their relationships.
4. A *behavioral* perspective, which models how the system behaves in response to inputs and events.
5. A *data* perspective, which models the structure and organization of the data within the system.

Three families of notation are used. Data-flow diagrams (DFDs) model context and the flow of data; entity-relationship (ER) diagrams model the structure of data (entities, their attributes, and the relationships between them); and the Unified Modeling Language (UML) covers all five perspectives. The UML is a standardized, general-purpose modeling language used to visualize, specify, construct, and document the artifacts of a software-intensive system. It is essentially a blueprint language for software. A UML model is more than a drawing: it is a collection of diagrams and model elements that together form a formal representation of the system's architecture, structure, and behavior, so that developers, technical writers, and business analysts can each understand the system at the level of detail relevant to them.

UML diagrams fall into two categories (Figure 5.1). *Structural diagrams* show the static elements of a system and their relationships, irrespective of time. They represent the 'nouns' of the system (classes, objects, components) and are used for planning and documenting architecture. *Behavioral diagrams* show the dynamic behavior of the system and its interactions over time. They represent the 'verbs' (actions and events) and are used to analyze user interactions and run-time behavior.

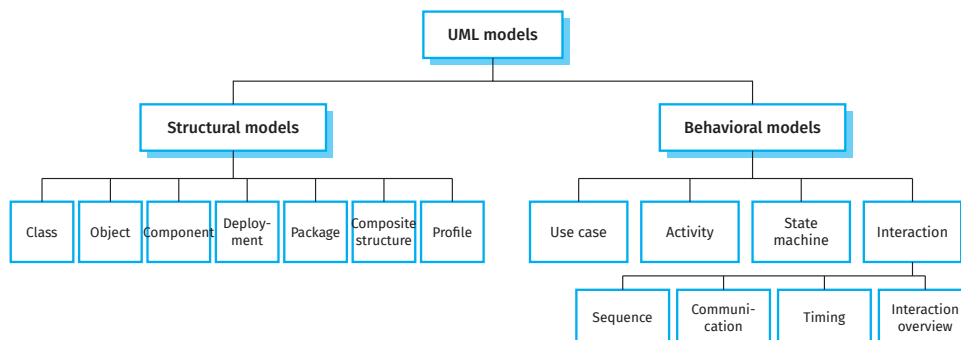


Figure 5.1 Structural and behavioral diagrams of the UML

The structural diagrams are: the *class diagram*, which shows classes, their attributes and operations, and their relationships; the *object diagram*, a snapshot of instances and their links at one point in time; the *component diagram*, which shows how the system is divided into modular components and their dependencies; the *deployment diagram*, which shows hardware nodes and the software artifacts deployed on them; the *package diagram*, which organizes model elements into logical groups to manage complexity; and the *composite structure diagram*, which shows the internal parts and ports of a class or component.

The behavioral diagrams are: the *use case diagram*, which captures high-level functional requirements as interactions between actors and use cases; the *activity diagram*, a flowchart of the steps of a business process or operation; the *state machine diagram*, which models the life cycle of a single object as states and the events that cause transitions between them; and four *interaction diagrams*. Of these, the *sequence diagram* shows how objects interact in chronological order; the *communication diagram* (called a collaboration diagram in earlier versions of the UML) shows the same information but emphasizes the links between objects rather than timing; the *interaction overview diagram* combines activity and sequence diagrams to give a high-level view of the flow of control; and the *timing diagram* shows the precise timing of state changes along a linear time axis.

5.1 Context models

At an early stage in the specification of a system, you should decide on the system boundaries. This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment. A context model is a visual representation that models the system as a single process interacting with external entities. It defines the system boundary, its scope, and its major inputs and outputs, without showing any internal detail.

The usual form of context model is the *context diagram*, which is the top level (level 0) of a set of data-flow diagrams. A DFD visually represents the flow of data through a system, using standardized symbols for processes, data stores, external

entities, and data flows to show inputs, outputs, transformations, and storage without describing implementation. The four elements are shown in Figure 5.2:

1. *External entities* are sources or sinks of data outside the system: people, organizations, or other systems. They are drawn as rectangles.
2. *Processes* transform incoming data into outgoing data. They are drawn as circles (bubbles), numbered, and named with a verb phrase such as ‘Validate order’.
3. *Data flows* show the movement of data and are drawn as labeled arrows.
4. *Data stores* are repositories where data is held between processes. They are drawn as open-ended rectangles.

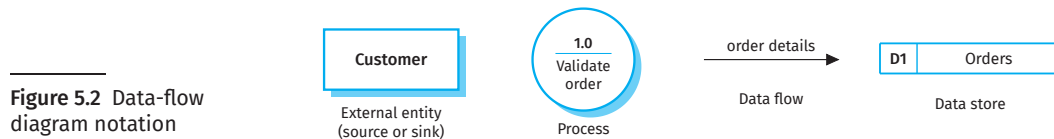


Figure 5.2 Data-flow diagram notation

In the context diagram the whole system is a single process. Figure 5.3 is the context diagram of a sales order system. It shows at once who uses the system (managers, employees, and customers) and what data crosses the boundary in each direction. In requirements engineering this gives a broad, stakeholder-friendly view that is used to clarify requirements, identify the interfaces that must be specified, and prevent scope creep: any proposed function that does not correspond to one of the flows on the context diagram is outside the agreed scope.

A context diagram says nothing about how the system works internally; its processes are revealed by decomposing it into lower-level DFDs (Section 5.4.1).

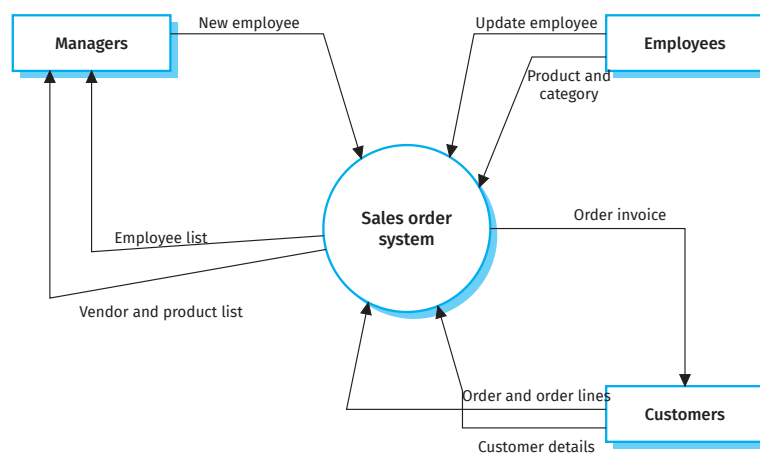


Figure 5.3 Context diagram (level-0 DFD) of a sales order system

5.2 Interaction models

Interaction models show how users interact with the system, how the system interacts with other systems, and how its components interact. Modeling user interaction helps to identify user requirements; modeling component interaction shows whether a proposed structure can deliver the required performance and dependability.

5.2.1 Use case modeling

A use case diagram illustrates how users (actors) interact with a system to achieve specific goals. It captures:

1. *Actors*, the roles that interact with the system, such as users or external systems, drawn as stick figures.
2. *Use cases*, the functions or actions that the system performs for an actor, such as ‘Borrow book’, drawn as ellipses.
3. *Relationships*, which show how actors and use cases are connected: associations, includes, and extends.

A use case diagram focuses on user goals, the system boundary, and high-level functionality without technical detail, and so it is used early to clarify requirements. Two relationships between use cases are important:

1. *«include»* is a mandatory dependency: the base use case cannot complete without executing the included use case, which is always performed as part of the base use case’s workflow. It is drawn as a dashed arrow labeled *«include»* pointing from the base use case to the included use case.
2. *«extend»* is a conditional dependency: the extending use case executes only if a particular condition is met during the base use case, and the base use case can complete successfully without it. It is drawn as a dashed arrow labeled *«extend»* pointing from the extending use case to the base use case, and the condition is documented.

Figure 5.4 shows the use cases of a library system. A member searches for, borrows, and returns books and views their account; a librarian adds books and manages members. Borrowing a book always includes checking that the book is available and that the member’s account is in good standing, so these are *«include»* relationships. A fine is generated only when a book is returned late, so ‘Generate fine’ extends ‘Return book’.

Each use case should also be documented in more detail, as a text description (actors, preconditions, main and alternative flows, postconditions) or as a sequence diagram.

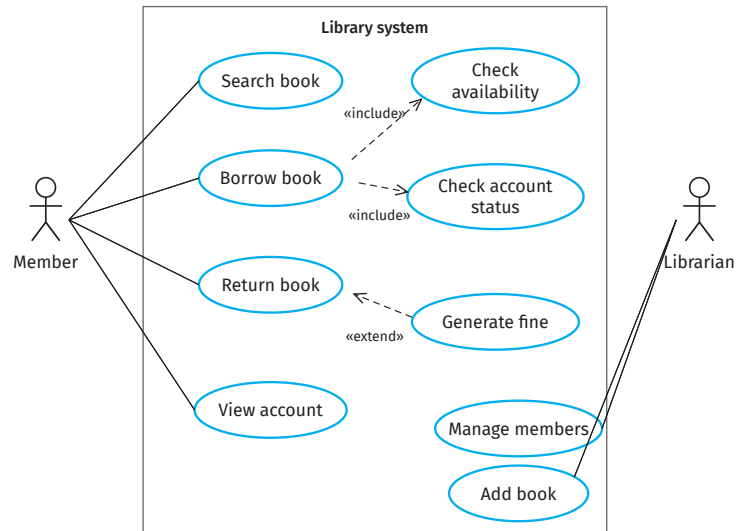


Figure 5.4 Use cases of a library system

5.2.2 Sequence diagrams

A sequence diagram is an interaction diagram that shows how objects interact with each other over time. It emphasizes the chronological order of the messages exchanged between objects to achieve a specific behavior, and it is used to model workflows, processes, and interactions between components. Its elements are:

1. *Lifelines*, which represent the objects or participants in the interaction. Each is drawn as a named box with a vertical dashed line below it.
2. *Messages*, drawn as arrows from one lifeline to another, labeled with the operation or signal and usually numbered.
3. *Activation bars*, narrow rectangles on a lifeline showing when an object is actively processing a message.
4. *Return messages*, optional dashed arrows that show a response or return value.
5. *Combined fragments*, boxes that represent conditional logic, loops, or parallel execution. The operator in the top-left corner is, for example, *alt* (alternatives, each with a guard), *opt* (an optional part executed only if its guard is true), or *loop*.
6. *Actors*, the external entities that take part in the interaction.

Time runs from top to bottom. Figure 5.8 shows a student member borrowing a book. The member searches, the library checks availability with the book object, and the result is displayed. If the book is available, the *opt* fragment is executed: the

member confirms, the book is reserved, a loan is created, and the book's status is updated.

Example 5.1 A sequence model for payment processing

A food-delivery service lets customers order from restaurants through a mobile app. Draw a sequence model for the payment step, from placing the order to confirming payment, including the case in which the bank rejects the payment.

Solution The participants are the Customer (an actor), the App, the Order System, the Payment Gateway, the Bank Server, and the Order Database. The order is saved first with status PENDING so that a failed payment does not lose the customer's cart. The gateway asks the bank to authorize the amount and returns the result to the order system. The two outcomes are alternatives, so they are placed in an *alt* fragment with the guards [approved] and [rejected] (Figure 5.5).

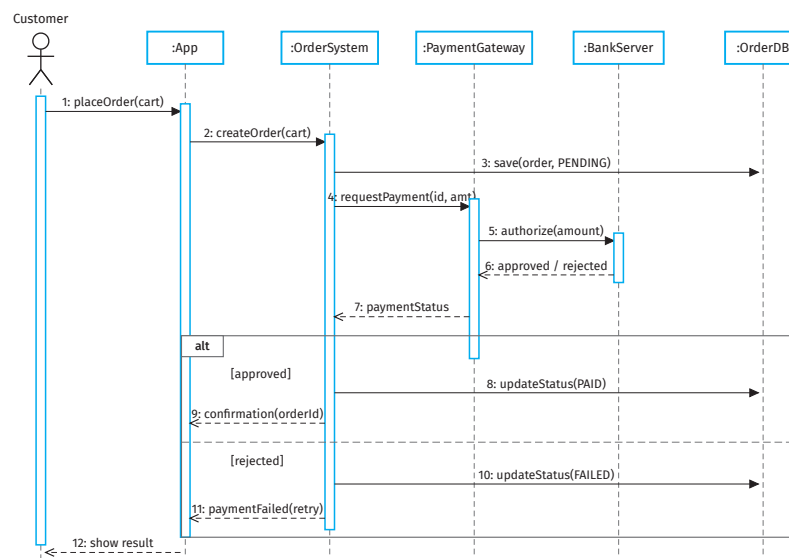


Figure 5.5 Sequence diagram for payment processing in a food-delivery system

Answer The message sequence is: 1 placeOrder (Customer→App); 2 createOrder (App→Order System); 3 save order as PENDING (Order System→Order DB); 4 requestPayment (Order System→Payment Gateway); 5 authorize (Gateway→Bank); 6 approved or rejected (Bank→Gateway); 7 payment status (Gateway→Order System); then, in the *alt* fragment, either 8 update status to PAID and 9 confirm to the app, or 10 update status to FAILED and 11 report failure so that the customer can retry; finally 12 the app shows the result to the customer. ■

Example 5.2 Use cases and a booking sequence for a ticket reservation system

A ticket reservation system for a public transport network integrates real-time seat availability, fare calculation, passenger details, and payment. The fare depends on the distance, the ticket type (adult, child, senior), and peak-hour pricing. (a) Model the interaction between users and the system. (b) Model the ticket booking scenario.

Solution (a) The interaction between users and the system is modeled by a use case diagram (Figure 5.6). The actors are the Passenger, the Admin who maintains the fare rules, and the external Payment gateway. Booking always includes calculating the fare and making the payment. Concessions and peak-hour surcharges apply only under a condition, so they extend ‘Calculate fare’.

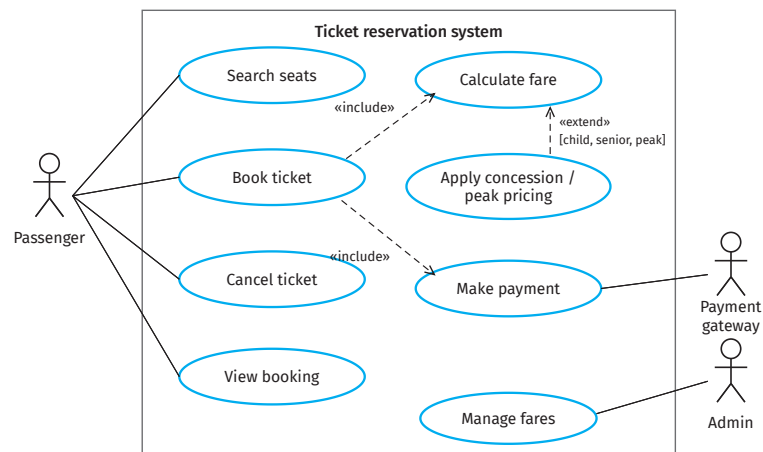


Figure 5.6 Use cases of a ticket reservation system

(b) A scenario, which is an ordered set of interactions, is modeled by a sequence diagram (Figure 5.7). Every fare parameter appears as an argument of the fare message (distance, ticket type, and time of travel), and the peak-hour surcharge is placed in an *opt* fragment. The seat is held while the fare is calculated and paid for, and is confirmed only after payment succeeds.

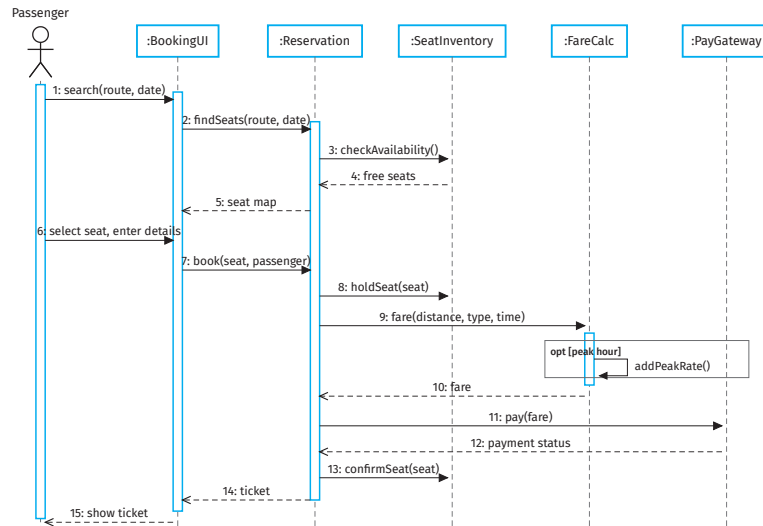


Figure 5.7 Sequence diagram for booking a ticket

Answer (a) A use case diagram with actors Passenger, Admin, and Payment gateway, and use cases Search seats, Book ticket («include» Calculate fare and Make payment), Cancel ticket, View booking, and Manage fares, with ‘Apply concession / peak pricing’ «extend» ‘Calculate fare’. (b) A sequence diagram whose messages run: search, find seats, check availability, select seat and enter passenger details, hold seat, fare(distance, type, time) with an optional peak-hour surcharge, pay, confirm seat, and issue the ticket. ■

5.2.3 Communication diagrams

A communication diagram (also known as a collaboration diagram) is another type of interaction diagram. It emphasizes the relationships between objects and shows how messages are exchanged within a particular context. Unlike a sequence diagram, it does not show time as a separate dimension; instead it shows the structural organization of the objects and their connections. Its elements are:

1. *Objects and links* Objects are rectangles labeled with their class names, and links are lines connecting objects that show associations between them.
2. *Messages* Each message is written beside a link with an arrow and a sequence number that gives the order of the interactions, for example ‘1: searchBook’, ‘2: checkAvailability’. The label describes the action taking place.

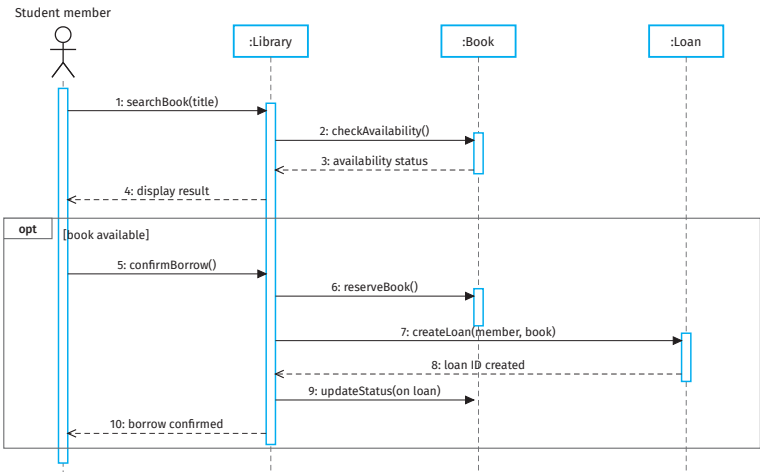


Figure 5.8 Sequence diagram for borrowing a book

Figure 5.9 is the communication diagram for the same borrowing interaction as Figure 5.8. The two diagrams contain the same information, and tools can generate one from the other. A sequence diagram is better when the order of messages is what matters; a communication diagram is better when you want to see which objects must be linked.

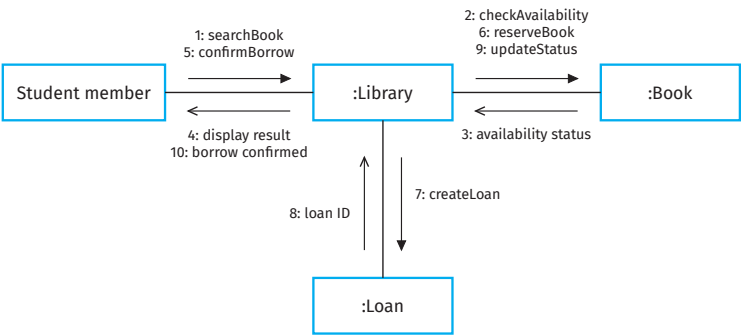


Figure 5.9 Communication diagram for borrowing a book

5.3 Structural models

Structural models show the organization of a system in terms of its components and their relationships.

5.3.1 Class diagrams

A class diagram is a structural diagram that visualizes the static design of a system. It shows classes with their attributes and operations, the relationships between classes, and the constraints and multiplicities of those relationships (for example, one-to-many). It serves as a blueprint for an object-oriented system and clarifies how its parts relate before implementation.

A class is drawn as a rectangle with three compartments: the name, the attributes, and the operations. Visibility is shown by a prefix: + for public and – for private. Six kinds of relationship are used (Figure 5.10):

1. *Association* A general relationship between classes, meaning that objects of one class are linked to objects of the other; drawn as a solid line. Example: Student enrolls in Course.
2. *Aggregation* A whole–part relationship in which the parts can exist independently of the whole; drawn with a hollow diamond at the whole. Example: a Department has Professors, who continue to exist if the department is closed.
3. *Composition* A strong whole–part relationship in which the parts cannot exist without the whole; drawn with a filled diamond at the whole. Example: a House has Rooms; if the house is destroyed, so are its rooms.
4. *Generalization* An ‘is-a’ relationship in which a subclass inherits from a superclass; drawn as a solid line with a hollow triangle pointing to the superclass. Example: ElectricCar is a Car.
5. *Dependency* A temporary ‘uses’ relationship in which one class uses another, for example as a parameter; drawn as a dashed arrow. Example: ReportGenerator uses Data.
6. *Realization* A class implements an interface; drawn as a dashed line with a hollow triangle pointing to the interface.

Generalization and realization are easily confused because both use a hollow triangle. The difference is the line: a *solid* line means that the subclass inherits the structure and behavior of a superclass; a *dashed* line means that a class promises to implement the operations declared by an interface.

Multiplicity (cardinality) is written at each end of an association to say how many objects take part: 1 (exactly one), 0..1 (zero or one), * or 0..* (any number), 1..* (at least one), or a range such as 1..4.

Figure 5.11 is a class diagram of a library system. A Library is composed of Books and Loans: if the library is deleted, its books and loan records are deleted too. Members and Librarians are aggregated by the library, because a person can exist independently of it (a student might graduate but still exist as a person). Each Loan links exactly one Book to exactly one Member, while a book or a member may have many loans over time. Faculty and Student are types of Member that have different

borrowing rules, so they are subclasses connected by generalization, each overriding `getMaxBooks()`.

Figure 5.10 Notation for relationships between classes

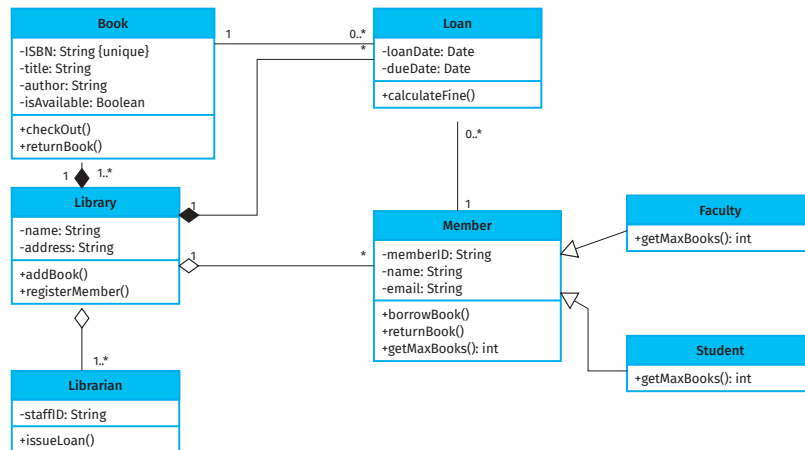
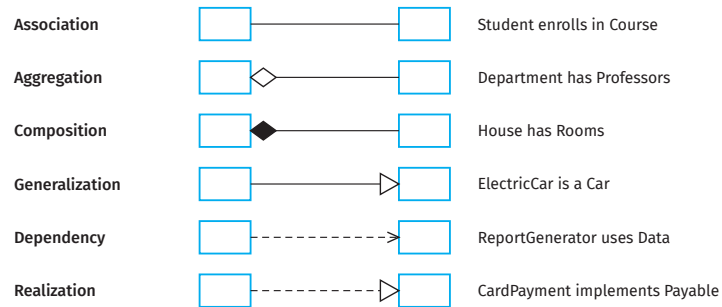


Figure 5.11 Class diagram of a library system

Example 5.3 Principal and collaborating classes of a food-delivery system

A food-delivery system processes orders placed with restaurants, takes payments, assigns delivery partners, and tracks deliveries in real time. Identify the principal classes of the system and the classes each one collaborates with.

Solution A class is identified by its responsibilities: what it knows and what it does. Its *collaborators* are the other classes whose services it must use to meet those responsibilities. This is the basis of Class–Responsibility–Collaborator (CRC) modeling.

Some of these classes are also *active classes* in the strict UML sense: an active class is one whose objects own their own thread of control and so can initiate activity, rather than only responding to calls. The Delivery Partner Allocator (continuously matching orders to free partners), the Real-Time Tracker (continuously receiving

GPS updates), and the Notification Service (sending messages asynchronously) are active; they are drawn with double vertical borders in a class diagram. Customer, by contrast, is best modeled as an actor that uses the system, together with a passive Customer class that holds the customer's data. A tempting wrong answer is to call every class that 'does something' an active class.

Answer The principal classes, their responsibilities, and their collaborators are listed in Figure 5.12. Of these, the Delivery Partner Allocator, the Real-Time Tracker, and the Notification Service are active classes. ■

Figure 5.12 Principal classes of a food-delivery system and their collaborators

Class	Responsibilities	Collaborators
Order Processing System	Create orders, compute totals, track order status	Customer, Restaurant, Payment Gateway, Delivery Partner Allocator
Restaurant Manager	Accept or reject orders, update menu and preparation time	Menu, Order, Delivery Partner Allocator
Payment Gateway (interface)	Request authorization and report payment status	Order, Customer, Bank Server
Delivery Partner Allocator	Choose the nearest free partner and assign the order	Delivery Partner, Order, Location Service
Real-Time Tracker	Receive GPS updates and compute the estimated arrival time	GPS Service, Order, Delivery Partner
Notification Service	Send status messages to customers and restaurants	Order, Customer, Restaurant

Example 5.4 Implications of multiplicity in a class model

For the food-delivery system, draw a class diagram showing the multiplicities of the main associations and explain how the multiplicities affect implementation.

Solution Figure 5.13 shows the model. A Customer places zero or more Orders; each Order belongs to one Customer and one Restaurant, has exactly one Payment, is composed of one or more OrderItems, and is assigned to zero or one DeliveryPartner (none until a partner accepts it).

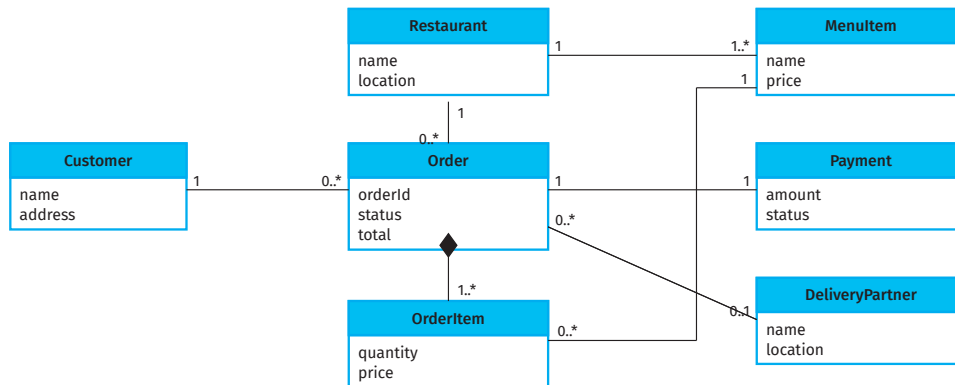


Figure 5.13 Class diagram of a food-delivery system showing multiplicities

Answer Multiplicity determines three things in the implementation:

1. *Database design* A one-to-many association (Customer 1 — 0..* Order) becomes a foreign key in the ‘many’ table (Order.customerId); a many-to-many association needs a separate link table; a composition (Order ◆— OrderItem) means that item rows are deleted with their order.
2. *Object and memory structure* An end with multiplicity 1 or 0..1 becomes a single reference (order.payment); an end with * becomes a collection (customer.orders). A lower bound of 0 means that the reference may be null and the code must handle that case.
3. *Constraint enforcement* Bounds are business rules that the code must check: exactly one payment per order, at least one item per order, and at most one delivery partner per order. ■

Example 5.5 A class model and a sequence diagram for university accounts

Each student has a University Account with an account id and a student name. Based on this account the student has a Link Account with an e-mail address and a password; the password can be reset, but the e-mail address cannot be changed. The link account is used to access Blackboard and Zoom. Blackboard holds a list of courses and supports adding courses and downloading and uploading homework. Zoom holds a list of meetings and can create and delete meetings. Each Course has a course id and name and has at least one meeting and one homework. A Meeting has a unique meeting id and an invitation link, and joining it by id requires the password. Each Homework has a course id, a homework number, and a list of questions. (a) Draw the class diagram. (b) Model the interaction among the objects when a student joins the online lecture of a course.

Solution (a) The nouns give the classes, their properties give the attributes, and the verbs give the operations (Figure 5.14). A Link Account cannot exist without its University Account, so the relationship is a composition. Blackboard and Zoom own their lists of courses and meetings, which are also compositions. Blackboard and

Zoom *use* the link account for access, which is a dependency. The multiplicity 1..* records that each course has at least one meeting and at least one homework, and {readOnly} records that the e-mail address cannot be changed.

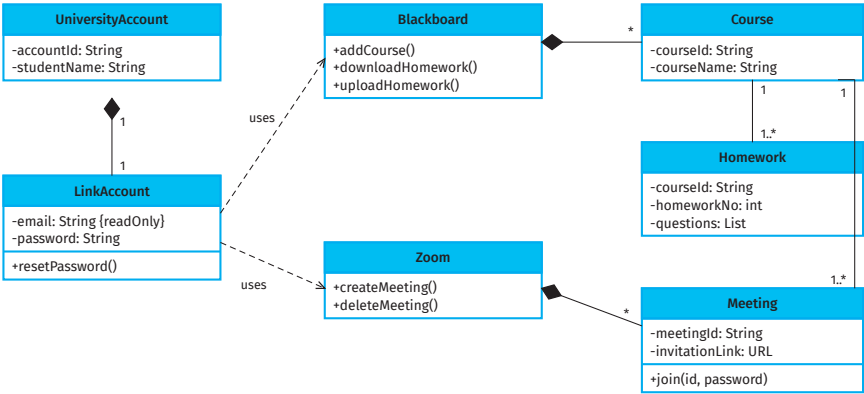


Figure 5.14 Class diagram of a university account system

(b) The interaction involves the Student, the Zoom application, the University account server, and the Zoom server (Figure 5.15). The link account is authenticated first; the meeting password is then checked by the Zoom server, and the two outcomes are shown in an *alt* fragment.

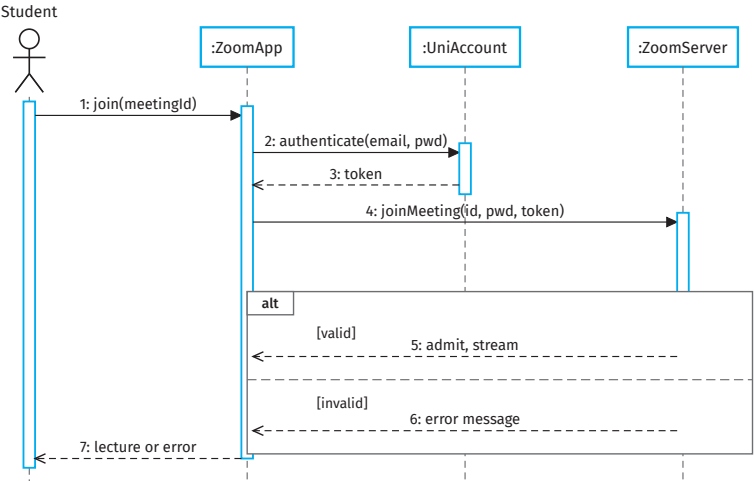


Figure 5.15 Sequence diagram for joining an online lecture

Answer (a) Classes UniversityAccount, LinkAccount, Blackboard, Zoom, Course, Meeting, and Homework with the attributes and operations shown, composition from UniversityAccount to LinkAccount, from Blackboard to Course, and from Zoom to Meeting, and Course associated with 1..* Homework and 1..* Meeting. (b) Messages: join(meetingId); authenticate(email, password); token; joinMeeting(id, password, token); then either admit and stream, or an error message. ■

5.3.2 Component and deployment diagrams

A *component diagram* shows how a system is divided into modular, replaceable components and how they depend on one another. A component is an encapsulated unit with provided and required interfaces; it may represent a physical artifact (a JAR file, a DLL, a container) or a logical service. It is drawn as a rectangle with the component icon. An *interface* is a contract that defines the operations a component offers (a *provided* interface, drawn as a 'lollipop') or needs (a *required* interface). A *frame* may enclose related components to show a subsystem. The relationships are *dependency* (a dashed arrow: a change in the target may affect the source), *interface realization* (the component implements an interface's operations), and *component realization* (the component is physically built from other artifacts, marked «build»). Figure 5.16 is a component diagram of the library system.

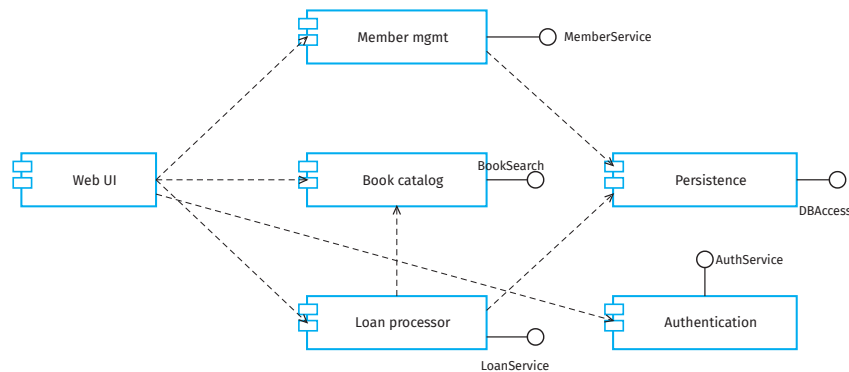


Figure 5.16
Component diagram
of a library system

A *deployment diagram* shows the physical hardware on which software is deployed. A *node* is a physical or virtual computational resource, drawn as a three-dimensional box: a *device node* is hardware such as a server or a phone, and an *execution environment node* is a software container such as a JVM, a Docker container, or an operating system. An *artifact* is a file or deliverable deployed on a node, such as library-core.jar. A *communication path*, a solid line between nodes, is labeled with the protocol used, such as HTTPS or JDBC. A *deployment specification*, a note attached to a deployment, gives configuration parameters such as memory limits. Nodes may be nested, for example a container inside a virtual machine. Figure 5.17 shows the library system deployed on four nodes.

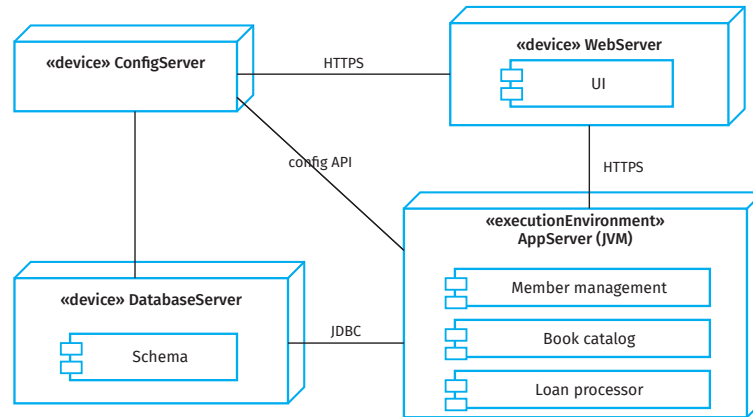


Figure 5.17
Deployment diagram
of a library system

5.4 Behavioral models

Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. Stimuli are of two types: *data*, where some data arrives that has to be processed, and *events*, which trigger system processing. Many business systems are data-processing systems that are primarily driven by data; real-time systems are often event-driven.

5.4.1 Data-driven modeling

Data-driven models show the sequence of actions involved in processing input data and generating an associated output. DFDs are the classic notation. They are simple and intuitive, and it is usually possible to explain them to potential system users who can then participate in validating the model.

DFDs are drawn as a hierarchy of levels. This book uses the convention that the context diagram is *level 0*:

1. *Level 0 (context diagram)* The system is a single process, numbered 0.0, interacting with external entities (Section 5.1).
2. *Level 1* The single process is decomposed into its main sub-processes, typically three to seven, numbered 1.0, 2.0, and so on. Data stores and detailed data flows are added, while the inputs and outputs of the context diagram are preserved. For example, a payment system might be broken into processing, invoicing, and confirmation.
3. *Level 2* Each level-1 process is refined into more detailed steps, numbered 1.1, 1.2, and so on, with its own flows and stores; for example, payment processing is refined into payment verification and funds deduction.

4. *Level 3 and below* Further detail is added for complex systems, until every input–process–output needed for implementation has been mapped.

Some texts call the context diagram the ‘context level’ and the first decomposition ‘level 0’. The diagrams are the same; only the labels differ. Whichever convention is used, it should be stated and applied consistently.

A DFD is created top-down. Start with level 0 by identifying the external entities and the high-level data exchanged with them. Then decompose iteratively: select a process, expand it in the next level, add sub-processes and data stores, and verify that the new diagram is balanced. Stop at primitive processes that need no further breakdown. The following rules keep a set of DFDs correct:

1. *Balancing rule* The inputs and outputs of a parent process must match exactly those of its child diagram, so that data flow is continuous across levels without additions or omissions.
2. *No data flow between entities* External entities interact only with processes, never directly with each other or with data stores. Entities represent the system boundary, not internal components.
3. *Process naming* Processes are named with verb–noun phrases, such as ‘Validate order’, that describe a data transformation; vague labels such as ‘Process data’ are avoided.
4. *Data-flow constraints* Flows carry data only, never control signals. They are labeled and directional, and they connect entities or processes to stores but never entity to entity. A flow out of a store is a read; a flow into a store is a write.
5. *Validity rules* Every process must have at least one input and one output. A process with inputs but no outputs is a *black hole*; a process with outputs but no inputs is a *miracle*. Data stores need flows in both directions if they are updated and read.

The lemonade stand is a simple example. The system has four activities: a customer places an order for lemonade; the stand owner makes the lemonade; the customer pays; and the owner serves the lemonade. Figure 5.18 is its context diagram. The customer, the employee, and the vendor who supplies the ingredients are the external entities.

At level 1 (Figure 5.19), the single process is decomposed into four processes: 1.0 Sale, 2.0 Production, 3.0 Procurement, and 4.0 Payroll. Every flow on the context diagram appears again, now connected to the process that handles it: the customer’s order and payment go to Sale, the served product comes from Production, and so on. New internal flows, such as the product ordered passing from Sale to Production, appear for the first time.

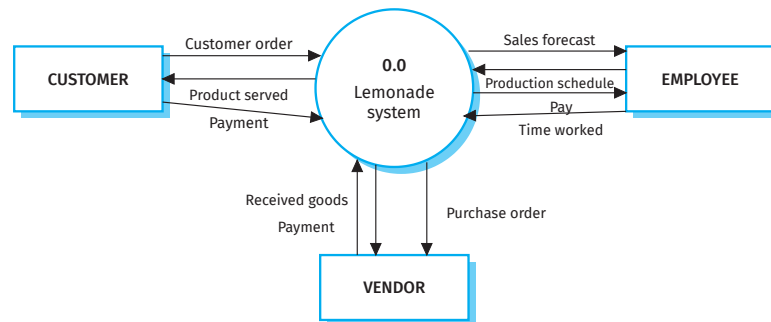


Figure 5.18 Context diagram (level 0) of the lemonade stand system

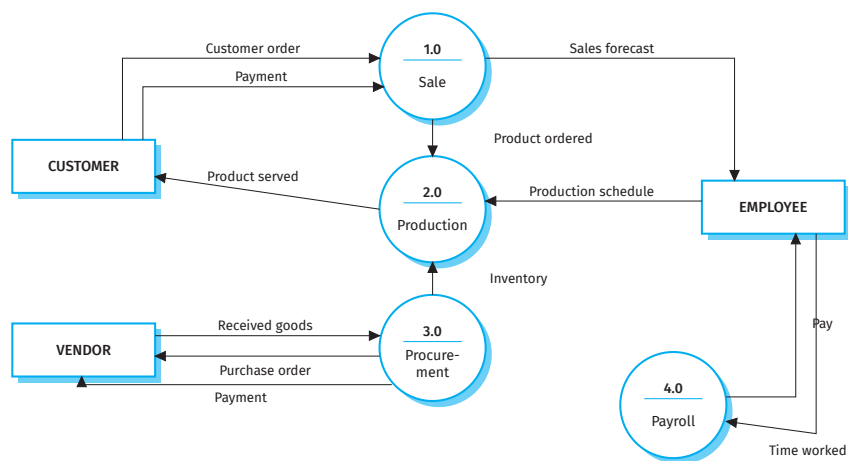


Figure 5.19 Level-1 DFD of the lemonade stand system

At level 2, each process is decomposed again. Figure 5.20 shows process 1.0, Sale, as three processes that record the order, receive the payment, and produce the sales forecast from the stored orders and payments. Its external flows (customer order and payment in; product ordered and sales forecast out) are exactly those of process 1.0 at level 1, so the diagrams balance.

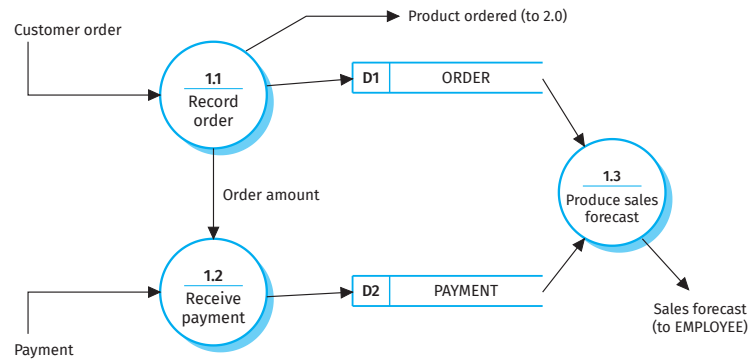


Figure 5.20 Level-2 DFD for process 1.0, Sale

Example 5.6 Data and data-flow models of an academic record system

In an academic record system, courses are created with a course number, credits, and a syllabus. Students (roll number, address, semester) are admitted and register for courses. The marks for each subject are keyed in, the semester weighted average (SWA) is calculated from the credits, the marks are combined with previous marks into a cumulative weighted average, and the marks and SWA are formatted and printed. A student with an SWA of 85 or higher is placed on the Vice-Chancellor's list; a student with an SWA below 50 is placed on conditional standing. (a) Draw an entity-relationship diagram. (b) Draw the context diagram and the level-1 DFD.

Solution (a) An entity-relationship (ER) diagram models the data perspective: entities (things about which data is kept), their attributes, and the relationships between them with their cardinalities. A student registers for many courses and a course has many students, so the many-to-many relationship is resolved by an Enrollment entity that also holds the marks. A student has one transcript *per semester*, so Student to Transcript is one-to-many (Figure 5.21).

(b) The context diagram shows the whole system as one process and the four external entities that exchange data with it (Figure 5.22). A context diagram has exactly one process; a list of the system's functions belongs to level 1, not level 0.

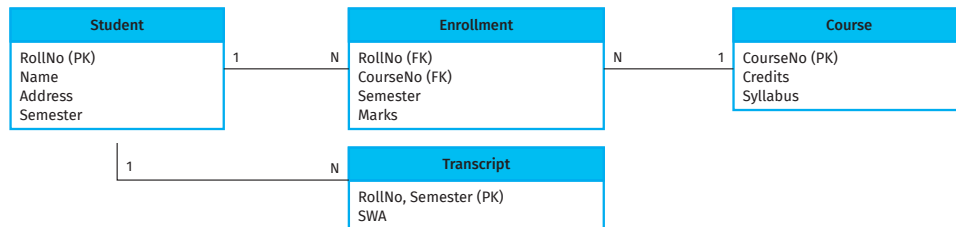


Figure 5.21 Entity-relationship diagram of an academic record system

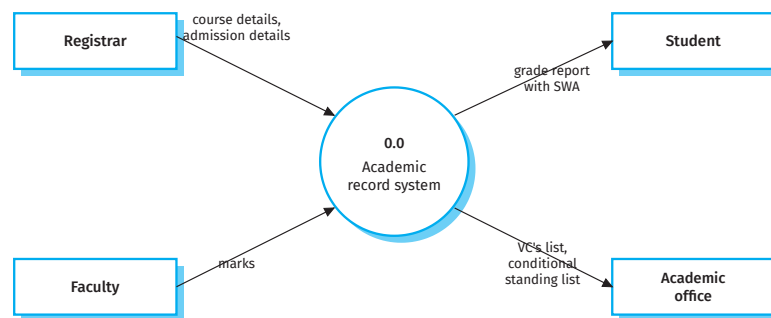


Figure 5.22 Context diagram of the academic record system

At level 1 the process is decomposed into six processes with four data stores (Figure 5.23). Every flow on the context diagram appears on the level-1 diagram, so the two diagrams balance.

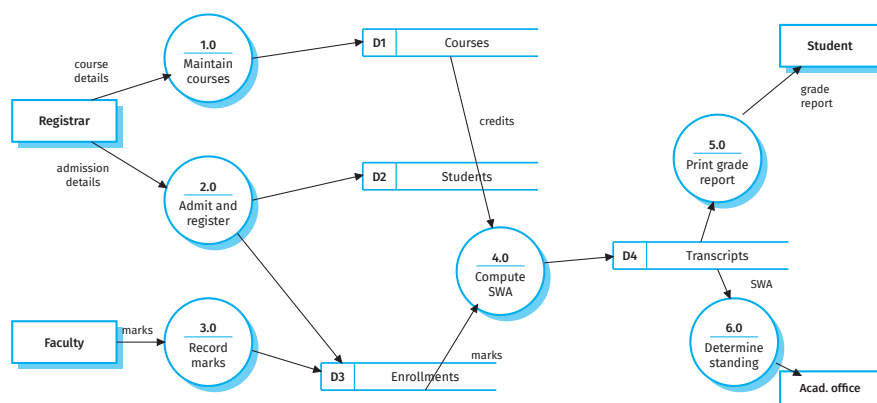


Figure 5.23 Level-1 DFD of the academic record system

Answer (a) Entities Student, Course, Enrollment (RollNo, CourseNo, Semester, Marks), and Transcript (RollNo, Semester, SWA), with Student 1–N Enrollment N–1 Course and Student 1–N Transcript. (b) Context: Registrar, Faculty, Student, and Academic office around process 0.0. Level 1: 1.0 Maintain courses, 2.0 Admit and

register, 3.0 Record marks, 4.0 Compute SWA, 5.0 Print grade report, and 6.0 Determine standing ($SWA \geq 85 \rightarrow$ Vice-Chancellor's list; $SWA < 50 \rightarrow$ conditional standing), with stores D1 Courses, D2 Students, D3 Enrollments, and D4 Transcripts. ■

Activity diagrams are the UML's notation for data-driven and workflow models. An activity diagram represents the flow of activities or actions within a system. It is used to model workflows, business processes, algorithms, and the steps of use cases, showing how tasks are performed sequentially or concurrently. Its notation is:

1. *Start node*, a solid circle marking the beginning of the activity; *end node*, a solid circle with a border marking its termination.
2. *Action*, a rounded rectangle representing one task or step.
3. *Decision node*, a diamond at which the flow branches according to guard conditions written in brackets; *merge node*, also a diamond, which combines alternative flows.
4. *Fork* and *join*, thick bars. A fork splits one flow into several concurrent flows; a join waits for all of them and merges them into one.
5. *Flow*, an arrow showing the direction of control between actions and decisions.
6. *Swimlanes*, vertical or horizontal partitions that group actions by the actor or system responsible for them.

Figure 5.24 shows borrowing a book with swimlanes for the member, the library system, and the librarian. If the book is available it is reserved and borrowing is confirmed; the book status, the member profile, and the notifications are then updated in parallel, and the join waits for all three before the book is issued. Each action is placed in the lane of whoever performs it, which is the main value of swimlanes: they show responsibilities.

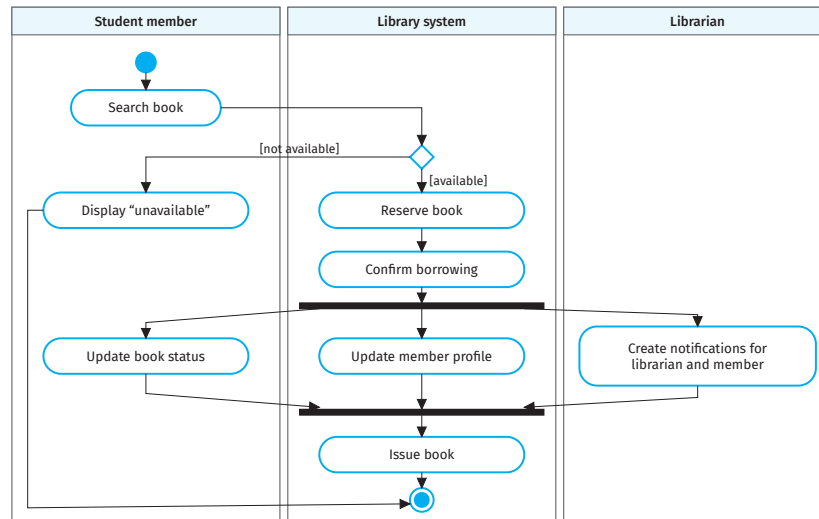


Figure 5.24 Activity diagram with swimlanes for borrowing a book

5.4.2 Event-driven modeling

Event-driven modeling shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another. A state represents a condition or situation during the lifetime of an object, and transitions represent changes from one state to another. The UML supports event-based modeling using state machine diagrams (state charts). The notation is:

1. *States*, rounded rectangles, each a specific condition of the object; the *initial state*, a solid circle; and the *final state*, a solid circle with a border.
2. *Transitions*, arrows between states, showing how the object moves from one state to another.
3. *Events*, written on transitions to name the trigger, for example *reserve* or *return*.
4. *Guard conditions*, Boolean conditions in square brackets that must be true for the transition to fire, for example *[member eligible]*.
5. *Actions*, written after a slash, performed during a transition or within a state, for example */updateStatus* or */generateFine*.
6. *Composite states*, states that contain substates and so group related states; and the *history state*, a circle containing H, which returns the machine to the last active substate when a composite state is re-entered.

Figure 5.25 models the life of a book. A new book is Available; it may be Reserved and then issued, or issued directly; while On loan it becomes Overdue if the due date passes. A returned book becomes Available again, and a fine is generated if it was overdue. Only a damaged book that is Withdrawn leaves the machine. Note that returning a book does not end its life: a state diagram whose final state follows 'Returned' would model a single loan, not the book.

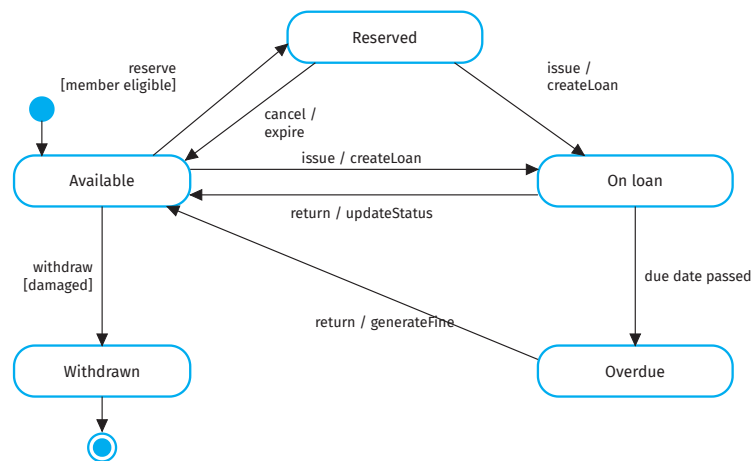


Figure 5.25 State machine diagram for a book in a library

5.5 Building a requirements model

The models of this chapter are complementary, and a complete requirements model uses several of them. It is useful to group them into four kinds of element:

1. *Scenario-based elements* describe the system from the user's point of view: use cases and their text descriptions, use case diagrams, and activity and swimlane diagrams. They are usually the first part of the model to be developed, and they drive the others.
2. *Class-based elements* describe the objects that the system manipulates, their attributes and operations, and their relationships: class diagrams, CRC models, and collaboration diagrams.
3. *Behavioral elements* describe how the system responds to external events: state diagrams and sequence diagrams.
4. *Flow-oriented elements* describe how data is transformed as it moves through the system: data-flow diagrams, control-flow diagrams, and processing narratives.

The elements are not alternatives. Each answers a different question (who uses the system and for what, what it knows, how it reacts, and how data moves through it),

and a reviewer can check them against each other: every use case should be supported by classes and operations, and every data flow should be produced and consumed by some process.

Example 5.7 A requirements model for an antivirus product

A client requires antivirus software with: signature-based, heuristic, and machine-learning detection; real-time monitoring and protection of the system and web traffic; automatic malware removal; low resource consumption; daily updates from the vendor's server; an attractive user interface; code security; and compliance with legal requirements. Build the requirements model using the elements of the requirements model.

Solution The requirements are first classified (see Example 4.1 in Chapter 4): the detection, monitoring, removal, and update requirements are functional and are modeled below, while the efficiency, usability, security, and legal requirements are non-functional and are recorded as constraints on the model.

Scenario-based element. The actors are the User and the vendor's Update Server. The use cases are Run scan, Configure real-time protection, Quarantine or remove malware, View threat report, and Update definitions. Removal happens only when a threat is found, so it extends scanning (Figure 5.26).

Class-based element. A ScanEngine uses one or more Detectors. Detector is an interface, realized by SignatureDetector, HeuristicDetector, and MLDetector, so that a new detection technique can be added without changing the engine. Infected files are passed to the Quarantine (Figure 5.27).

Behavioral element. The engine monitors file and web events, scans each one, and returns to monitoring or, if the file is malicious, quarantines it and notifies the user; updates are installed in a separate state, so scans never use half-installed definitions (Figure 5.28).

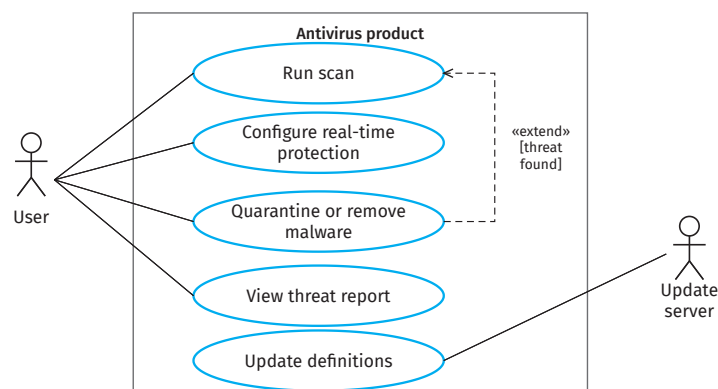


Figure 5.26 Scenario-based element: use cases of the antivirus product

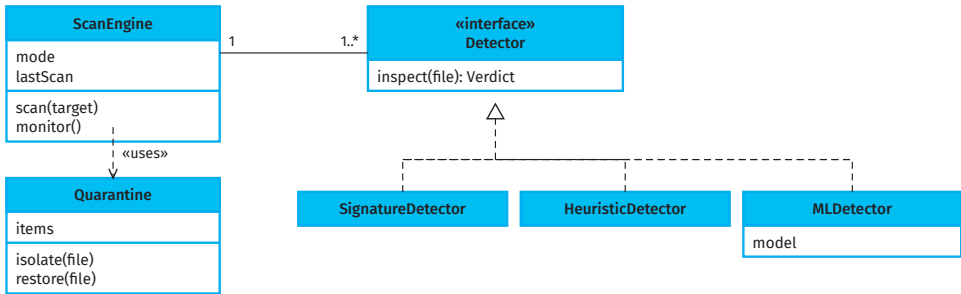


Figure 5.27 Class-based element: principal classes of the antivirus product

Flow-oriented element. Scanning is decomposed into intercepting the file, analyzing it against the signature store, and taking action, which quarantines infected files and reports to the user (Figure 5.29).

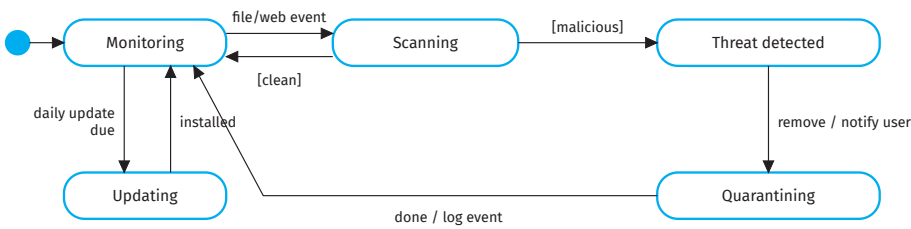


Figure 5.28 Behavioral element: state diagram of the real-time protection engine

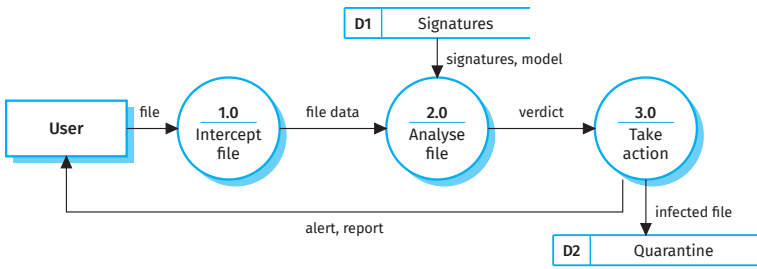
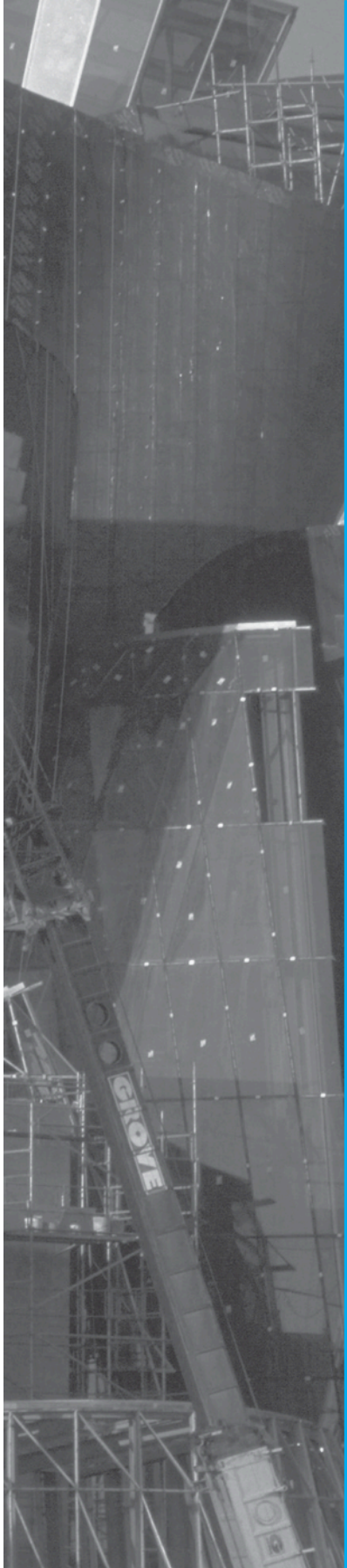


Figure 5.29 Flow-oriented element: level-1 DFD for scanning a file

Answer The model consists of the use case diagram (scenario-based), the class diagram (class-based), the state diagram of the engine (behavioral), and the level-1 DFD of scanning (flow-oriented), with the measurable non-functional constraints of Example 4.1. ■

KEY POINTS

- A model is an abstract view of a system that ignores some details. Complementary models show the system's context, interactions, structure, behavior, and data.
- The UML has structural diagrams (class, object, component, deployment, package, composite structure) and behavioral diagrams (use case, activity, state machine, and the interaction diagrams: sequence, communication, timing, and interaction overview).
- Context models show the system boundary and the external entities that exchange data with the system. The context diagram is the level-0 DFD.
- DFDs are decomposed level by level; each child diagram must balance with its parent, and entities, processes, flows, and stores must obey the DFD rules.
- Use case diagrams show actors and their goals, with «include» for mandatory and «extend» for conditional behavior. Sequence and communication diagrams show the messages exchanged between objects.
- Class diagrams show classes, their attributes and operations, and association, aggregation, composition, generalization, dependency, and realization relationships, with multiplicities that determine keys, collections, and constraints in the implementation.
- Activity diagrams show workflows with decisions, forks, joins, and swimlanes; state diagrams show how an object's state changes in response to events, guarded by conditions and accompanied by actions.
- A requirements model combines scenario-based, class-based, behavioral, and flow-oriented elements.



6

Architectural design

Objectives

The objective of this chapter is to introduce the concepts of software architecture and architectural design. When you have read the chapter, you will:

- understand what a software architecture is and the properties that an architectural design must specify;
- be able to describe a control hierarchy and partition a program structure horizontally and vertically;
- know the structural, framework, dynamic, process, and functional models of an architecture;
- understand the layered, client-server, and call-and-return architectural patterns and their trade-offs;
- be able to map a data-flow diagram into a program structure by transform mapping and transaction mapping.

Contents

- 6.1 Architectural design decisions
- 6.2 Architectural views
- 6.3 Architectural patterns
- 6.4 Architectural mapping using data flow

Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system. It is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them. The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.

Software architecture is the overall structure of the software: its components and the manner in which those components interact. It provides the framework within which detailed design is carried out. An architectural design must specify three kinds of property:

1. *Structural properties* The components of the system (modules, objects, filters) and the ways in which they are packaged and interact.
2. *Extra-functional properties* How the architecture achieves the requirements for performance, security, reliability, adaptability, and other non-functional characteristics. The architecture is the main place where these are decided, because they depend on how the whole system is organized rather than on any single component.
3. *Families of related systems* The architecture should draw on repeatable patterns that are common to families of similar systems, so that proven designs are reused rather than reinvented.

6.1 Architectural design decisions

Architectural design is a creative process, and the decisions made depend on the type of system being developed, the background and experience of the architect, and the specific requirements. A small number of structural concepts, however, recur in every architecture: the control hierarchy of the program, the way it is partitioned, and the way its data is logically organized.

6.1.1 Control hierarchy

Control hierarchy, also called program structure, is the organization of program components (modules) that shows how control flows from one module to another. Higher-level components manage and delegate tasks to lower-level components. The hierarchy is drawn as a tree-like structure chart. It does not show the sequence of processing, the order of decisions, or the repetition of operations; it shows only which module invokes which.

The following terms describe a control hierarchy:

1. *Depth* The number of levels of control.

2. *Width* The overall span of control: the largest number of modules at any one level.
3. *Fan-out* The number of modules directly controlled by a given module.
4. *Fan-in* The number of modules that directly control a given module.
5. *Superordinate module* A module that controls another module.
6. *Subordinate module* A module that is controlled by another module.

Figure 6.1 is the control hierarchy of a library management system. When a user borrows a book, the main application calls the transaction management module, which in turn uses the book inventory to check availability.

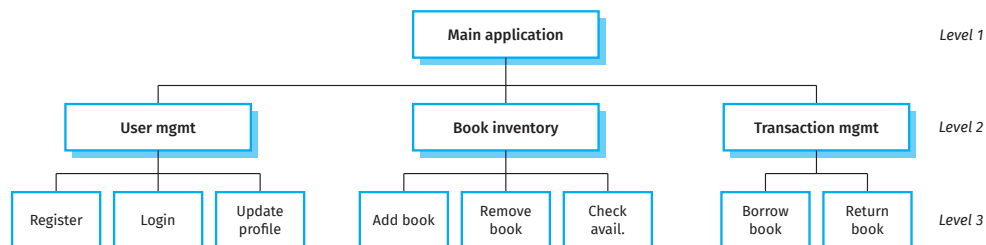


Figure 6.1 Control hierarchy of the library management system

Fan-in and fan-out are easily reversed. Fan-out is outgoing control (whom a module calls); fan-in is incoming control (who calls it). A useful check follows from the fact that both count the same arrows: the sum of all fan-outs in a structure chart equals the sum of all fan-ins. As design heuristics, fan-out should be kept moderate, since a module with very high fan-out makes too many decisions and should be factored with an intermediate control level, while fan-in should increase towards the bottom of the hierarchy, because a module with high fan-in is being reused.

Example 6.1 Measuring a control hierarchy

For the structure chart in Figure 6.2, find the depth and width of the hierarchy and the fan-in and fan-out of every module.

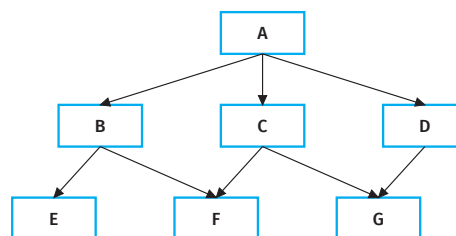


Figure 6.2 A structure chart with a shared module

Solution The levels are {A}, {B, C, D}, and {E, F, G}, so the depth is 3 and the width is $\max(1, 3, 3) = 3$. Counting arrows leaving and entering each module:

- 1. Fan-out: A = 3, B = 2, C = 2, D = 1, and E = F = G = 0.
- 2. Fan-in: A = 0, B = C = D = E = 1, and F = G = 2.

Check: the sum of the fan-outs is $3 + 2 + 2 + 1 = 8$ and the sum of the fan-ins is $1 + 1 + 1 + 1 + 2 + 2 = 8$.

Answer Depth 3, width 3. F and G have fan-in 2: they are shared utilities, which is desirable provided they are functionally cohesive. A is superordinate to B, C, and D; B and C are superordinate to F. ■

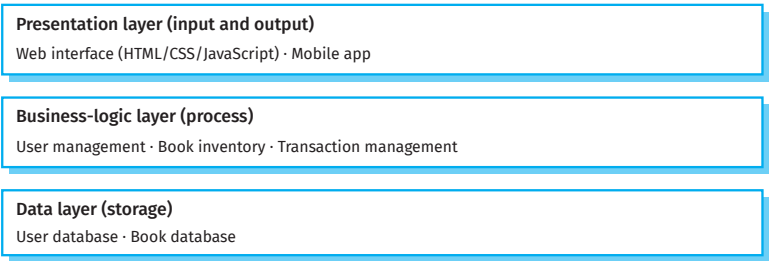
For the library system of Figure 6.1, the same measures give depth 3 and width 8; the main application has fan-out 3, user management and book inventory 3 each, transaction management 2, and every module except the main application has fan-in 1.

6.1.2 Structural partitioning

If the architectural style of a system is hierarchical, the program structure can be partitioned both horizontally and vertically.

Horizontal partitioning splits the software into separate branches of the hierarchy for each major function. In its simplest form there are three partitions: input (data collection and preprocessing), process (core computation), and output (presentation of results). Equivalently, the system is divided into layers or tiers by functionality, such as presentation, business logic, and data storage (Figure 6.3). The user interface can then be changed without affecting business logic or storage. Horizontal partitioning makes the software easier to test and maintain, gives fewer side effects because errors do not spread easily, and makes new functions easy to add. Its disadvantages are that more data must be passed across module interfaces and that the control of program flow becomes more complex.

Figure 6.3 Horizontal partitioning of the library system into layers



Vertical partitioning, also called *factoring*, organizes the program top-down so that control and decision-making reside in the upper-level modules, while the lower-level

modules are workers that perform input, computation, and output (Figure 6.4). The reason for doing this is the effect of change. A change in a control module affects all its subordinates, whereas a change in a worker module has minimal impact on other modules. Because most requirement changes affect input, computation, or output, which are done by workers, a factored structure is easier to maintain.

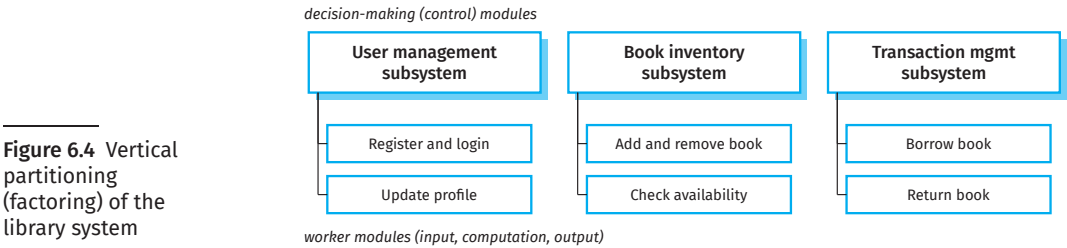


Figure 6.4 Vertical partitioning (factoring) of the library system

6.1.3 Logical data representation

Logical data representation defines how data elements relate to each other: how the data is structured and organized in terms of logical relationships between entities, independently of physical storage. It specifies the data organization, the methods of access, the associations between data items, and the processing alternatives. Figure 6.5 shows the library system. A transaction links a user to a book, recording that the user has borrowed it. The logical view is the same whether the data is kept in a relational database, a NoSQL store, or flat files.

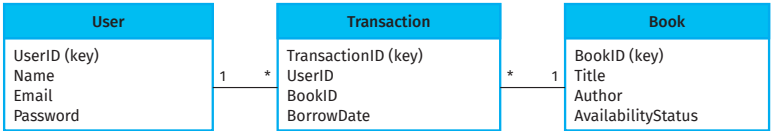


Figure 6.5 Logical data representation for the library system

6.2 Architectural views

It is impossible to represent all relevant information about a system’s architecture in a single model. An architectural design can be represented by five kinds of model:

1. *Structural model* The architecture as an organized collection of program components and their interactions. Example: a layered web application with presentation, business-logic, and data-access layers.
2. *Framework model* Raises the level of abstraction by identifying repeatable architectural frameworks found in similar applications. Example: the model–

view–controller (MVC) framework, in which the model manages data, the view handles the user interface, and the controller processes input.

- 3. *Dynamic model* The behavioral aspects of the architecture: how the structure or configuration changes in response to external events. Example: a traffic-monitoring system that adjusts signals from live traffic data.
- 4. *Process model* The business or technical processes that the system must accommodate, focusing on how tasks are executed. Example: order placement → payment → inventory check → shipping.
- 5. *Functional model* The functional hierarchy of the system: how functions are decomposed and depend on one another. Example: ‘Manage accounts’ decomposed into ‘Create account’, ‘Deposit money’, and ‘Withdraw money’.

For the library system, the structural view divides the system into user management, book inventory, and transaction management (Figure 6.6). The framework view might use a web framework such as Django or Spring Boot, which supplies authentication, object–relational mapping for the inventory, URL routing for borrow and return requests, and templates for the pages. The dynamic view is expressed as behavior over time (Figure 6.7), and the functional view simply lists what the system does: register user, log in, search books, borrow book, and return book.

Figure 6.6 Structural model of a library management system

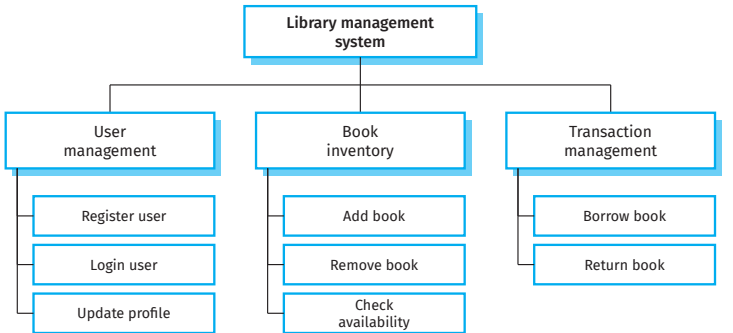
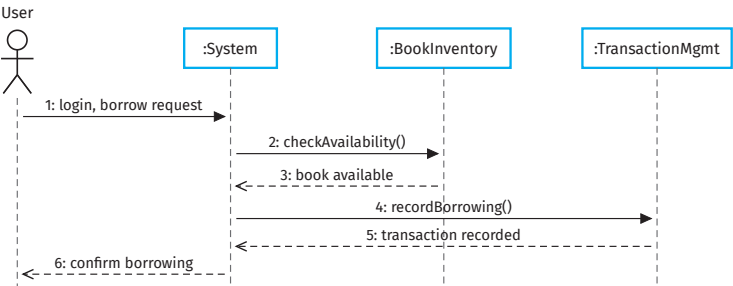


Figure 6.7 Dynamic model: borrowing a book



Example 6.2 Structural and dynamic views of an air traffic control system

An air traffic control (ATC) system is a hard real-time system in which delays or failures could be catastrophic. Radar continuously tracks the position, altitude, speed, and heading of every aircraft in an airspace. The system displays the traffic to controllers, calculates collision risks against minimum-separation thresholds, raises an alert with a recommended corrective action (a change of altitude or heading) when a conflict is detected, and sends instructions to pilots by radio or automated messaging. If hardware or the network fails, it must switch to a backup server within milliseconds. (a) Model the structural blueprint of the system: its key entities and their interconnections. (b) Model the temporal ordering of messages between the objects when a conflict is detected.

Solution The two parts are the structural and dynamic models of this section, each drawn in the UML.

(a) The structural view is a component diagram (Figure 6.8); a class diagram with the same entities is an acceptable alternative. The components are the Radar interface, which receives the radar feed through a provided interface; the Track manager, which keeps the current track of each aircraft; the Conflict detector, which compares pairs of tracks with the separation thresholds; the Alert manager, which raises alerts and recommends corrective actions; the Controller display; Pilot comms, which sends instructions; the Flight data store, which holds flight plans and tracks; and a Failover monitor. Dashed dependency arrows show which component uses which. The availability requirement is met structurally: the whole configuration is replicated on a backup server that runs as a *hot standby* and is kept up to date continuously. The failover monitor checks a heartbeat from the primary and switches to the backup as soon as the heartbeat stops, so the switch-over takes milliseconds rather than a restart.

(b) The dynamic view is a sequence diagram (Figure 6.9). Each radar update refreshes the track and the display and triggers a separation check. The alert, the recommendation, and the instruction to the pilot happen only if a conflict is found, so they are enclosed in an opt fragment with the guard [conflict]. A communication or state diagram could also show the dynamics, but a sequence diagram shows the time ordering most directly.

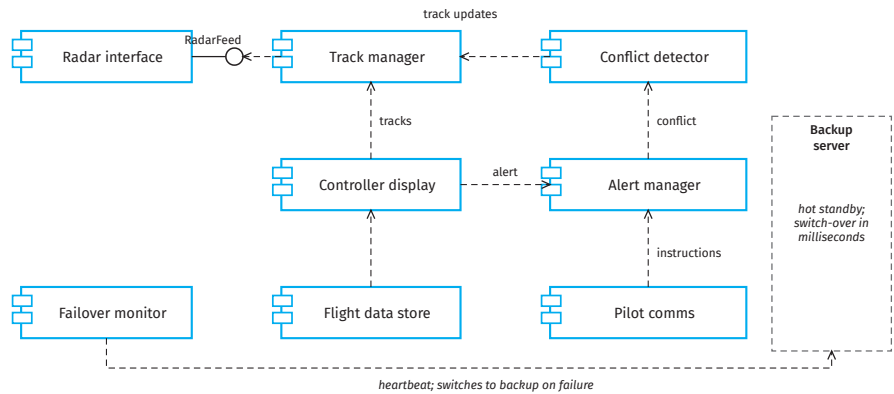


Figure 6.8
Component diagram
of an air traffic
control system

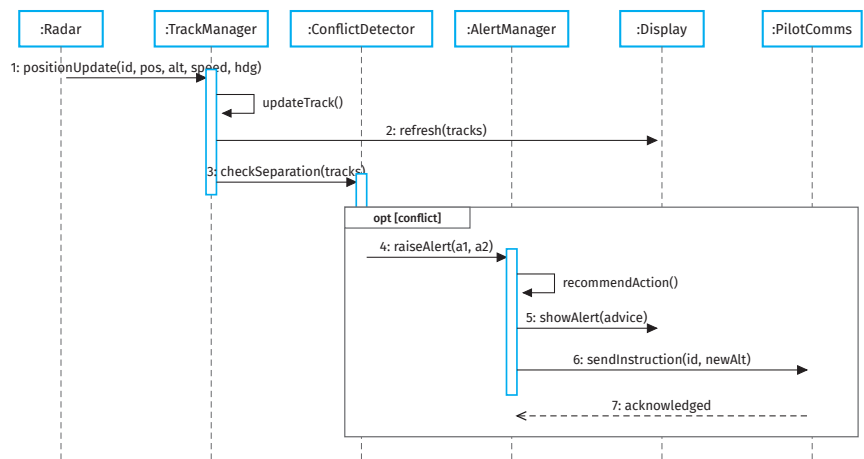


Figure 6.9 Sequence
diagram for detecting
and resolving a
conflict

Answer (a) Component diagram: Radar interface → Track manager → Conflict detector → Alert manager → Controller display and Pilot comms, with a Flight data store and a Failover monitor that switches to a hot-standby backup server. (b) Sequence diagram: positionUpdate, updateTrack, refresh, checkSeparation, then opt [conflict]: raiseAlert, recommendAction, showAlert, sendInstruction, acknowledged.

The *process model* of a system is usually drawn as an activity diagram (Section 5.4.1), which shows the order of the actions, the actions that run concurrently, and the component responsible for each.

Example 6.3 A process model of a weather sensing and alert system

A weather sensing and alert system periodically collects raw temperature, humidity, and pressure data from local sensors, and remote pressure data from an external API. Each input is preprocessed independently: temperature is calibrated, humidity normalized, and pressure values prepared and formatted. A multi-sensor synchronizer then aligns the timestamps, verifies completeness, and produces a unified fused weather record, which an alert threshold evaluator compares with the configured thresholds. If no limit is exceeded, the record is logged; if any threshold is violated, the system generates an alert, stores it, notifies users, updates the admin panel, and returns to idle. (a) Create an activity table of the events, states, and actions. (b) Draw the activity diagram for the states and transitions in the table. (c) What is the purpose of the fork nodes and swimlanes in the diagram?

Solution (a) Each row of the activity table (Figure 6.10) records a state of the system, an event that occurs in that state, the actions performed in response, and the resulting state. The system starts and ends in Idle, because it runs periodically.

Figure 6.10 Events, states, and actions of the weather sensing and alert system

State	Event	Action	Next state
Idle	Sampling timer fires	Read sensors; poll external API	Acquiring
Acquiring	Raw readings received	Calibrate temperature, normalize humidity, format pressure (in parallel)	Preprocessing
Preprocessing	All three inputs ready	Align timestamps, check completeness, fuse record	Evaluating
Evaluating	Record within limits	Log record	Idle
Evaluating	Threshold violated	Generate alert	Alerting
Alerting	Alert generated	Store alert, notify users, update admin panel (in parallel)	Idle

(b) The activity diagram (Figure 6.11) turns each action into an action node. The three preprocessing chains are independent, so a fork starts them concurrently and a join waits for all three before synchronization, which needs complete data. The decision node after evaluation has the guards [within limits] and [threshold violated]. The three alert actions are also independent, so a second fork runs them in parallel, and a join precedes the return to idle. Both branches end at the final node, which stands for the return to idle until the next sampling period.

(c) A *fork* node splits one flow into concurrent flows. Here it shows that temperature, humidity, and pressure are preprocessed at the same time rather than one after another, and that storing the alert, notifying users, and updating the admin panel also proceed together; the matching *join* marks the point at which all the parallel flows must have finished. *Swimlanes* partition the actions by the component responsible for them (sensors and API, preprocessing, synchronizer and evaluator,

and alert services), so the diagram shows who performs each action as well as the order, and each lane later becomes a component of the architecture (Example 6.5).

Answer (a) States Idle, Acquiring, Preprocessing, Evaluating, and Alerting, with the events and actions of Figure 6.10. (b) An activity diagram with four swimlanes, a fork–join pair around preprocessing, a decision on the thresholds, and a fork–join pair around the alert actions. (c) Forks show concurrency (and joins the synchronization that follows it); swimlanes show responsibility. ■

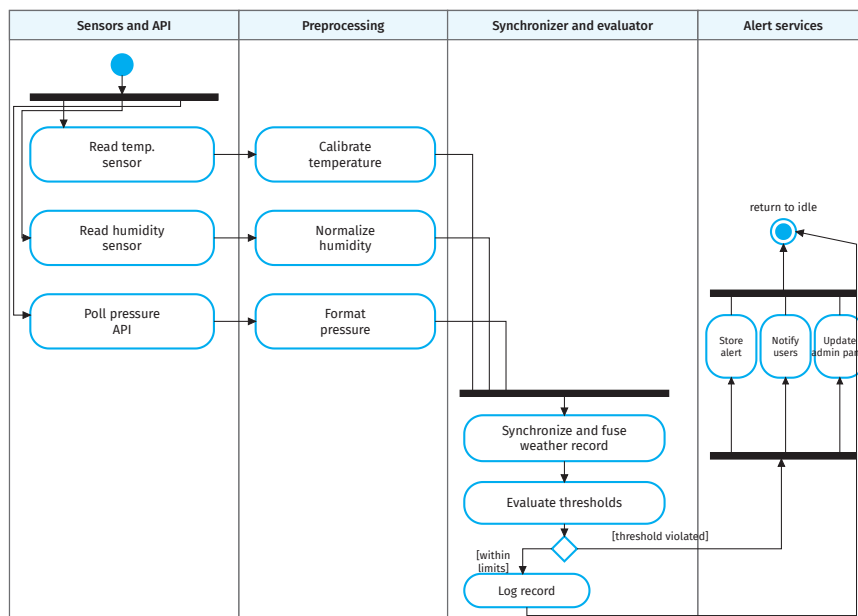


Figure 6.11 Activity diagram with swimlanes for the weather sensing and alert system

6.3 Architectural patterns

An architectural pattern (or style) is a stylized description of good design practice, which has been tried and tested in different environments. Patterns include information about when they are and when they are not useful, and a pattern gives a designer a starting point with known strengths and weaknesses. The most widely used patterns are described here.

6.3.1 Layered architecture

The layered architecture pattern organizes the system into layers, each of which provides services to the layer above it (Figure 6.12). The lowest layers represent core services, such as the operating system and database, that are used throughout the system; the top layer provides the user interface. Figure 6.13 is a generic four-layer architecture.

Figure 6.12 The layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest-level layers represent core services that are likely to be used throughout the system.
Example	A web application with presentation, business-logic, and data-access layers; the food-delivery system described in this section.
When used	When building new facilities on top of existing systems; when development is spread across several teams, each responsible for a layer; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase dependability.
Disadvantages	A clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower layers. Performance can suffer because a request is processed at each layer.

The layered approach supports the incremental development of systems. As a layer is developed, some of the services provided by that layer may be made available to users. The architecture is also changeable and portable: as long as its interface is unchanged, a layer can be replaced by another, equivalent layer, and when layer interfaces change only the adjacent layer is affected.

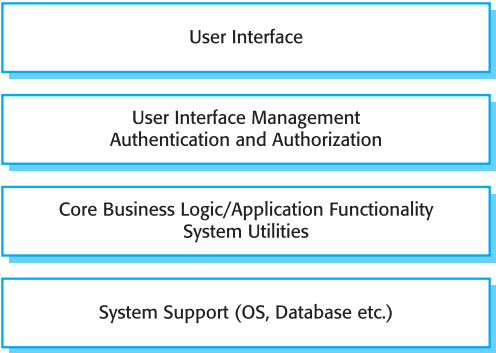


Figure 6.13 A generic layered architecture

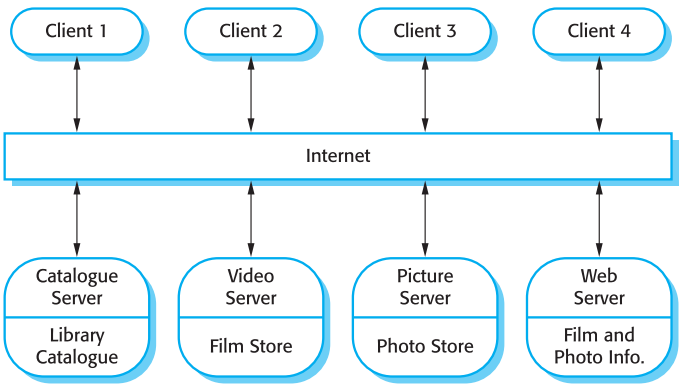
6.3.2 Client-server architecture

In a client-server architecture, the functionality of the system is organized into services, with each service delivered from a separate server, and clients use the services by making requests to the servers (Figure 6.14). The major components are a set of servers that offer services, a set of clients that call on those services, and a network that allows the clients to access the services. Figure 6.15 shows a film library in which separate servers manage the catalogue, videos, pictures, and web pages.

Figure 6.14 The client-server pattern

Name	Client-server
Description	The functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	A film and video library organized as a set of servers accessed over the Internet by clients.
When used	When data in a shared database has to be accessed from a range of locations. Because servers can be replicated, also when the load on a system is variable.
Advantages	Servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and need not be implemented by all services.
Disadvantages	Each service is a single point of failure, susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network. There may be management problems if servers are owned by different organizations.

Figure 6.15 A client-server architecture for a film library

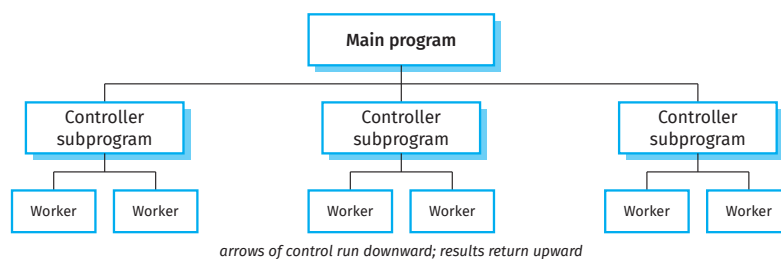


Clients have to know the names of the available servers and the services they provide, but servers do not need to know the identity of the clients or how many clients are accessing them. The principal advantage of the pattern is that servers can be distributed across a network and replicated when load is high.

6.3.3 Call-and-return architecture

A call-and-return architecture decomposes the system into a hierarchy of procedures or components in which control passes downward by calls and results pass back upward by returns. In its classic form, the *main program/subprogram* architecture, a main program invokes controller subprograms, which in turn invoke worker components (Figure 6.16). A *remote procedure call* architecture has the same structure, but the components are distributed across computers on a network. Call-and-return architectures are easy to understand and to scale: new functionality is added as new subprograms. Their weakness is that each caller waits for its callee, so a slow component delays everything above it.

Figure 6.16 A main program/subprogram (call-and-return) architecture



The program structures produced by architectural mapping (Section 6.4) are call-and-return architectures.

Example 6.4 Architectural styles for a food-delivery system

A food-delivery company runs apps for customers, restaurants, and delivery partners. Customers browse menus and place orders; restaurants accept and prepare them; delivery partners are assigned and tracked in real time; payments are processed through a payment gateway. Explain how the client-server, layered, and call-and-return architectures apply to this system, and draw the layered architecture.

Solution The three styles describe different aspects of the same system, so they are used together.

1. *Client-server* The three mobile apps are clients. They send requests (browse menus, place an order, update location) to servers that provide the services: order processing, restaurant management, delivery allocation, payment processing, and tracking. The client-server style is chosen because many clients in many places must share the same data, and because servers can be replicated to handle peak loads at meal times.
2. *Layered* Within the servers the software is organized into layers: a presentation layer (the interfaces of the three apps), an application layer (order processing, restaurant management, delivery allocation, tracking), a business-logic layer

(payment processing, validation rules, the notification service), and a data layer (order, customer, restaurant, payment, and tracking databases). Each layer uses only the services of the layer below. A change of database or payment provider is confined to one layer.

3. *Call-and-return* The flow of control for a request follows call-and-return. When a customer places an order, the client app calls the order service; the order service calls the payment service; the payment service calls the database and the payment gateway; each returns its result to its caller. Each call waits for its response before the caller continues.

Answer The system is a client-server system whose server side is layered and whose request handling is call-and-return (Figure 6.17). The client-server style gives scalability and shared data; layering gives separation of concerns and replaceable layers; call-and-return gives a simple, predictable flow of control. For this system the client-server style has four advantages: order, payment, and tracking data are managed centrally; server capacity can be increased for meal-time peaks; sensitive payment data stays on secure servers; and updates are made once, on the server. Its disadvantages are that a server failure stops every client, that all three apps need a stable network connection, that heavy traffic can make the servers a bottleneck, and that the server infrastructure is costly. ■

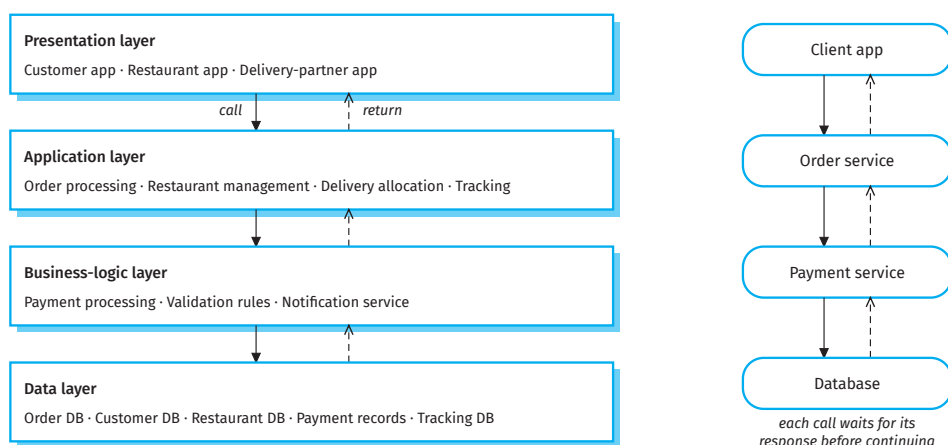


Figure 6.17 Layered architecture of a food-delivery system with call-and-return control

6.3.4 Repository architecture

In the repository pattern (Figure 6.18), all data in a system is managed in a central repository that is accessible to all system components, and components do not

interact directly, only through the repository. The majority of systems that use large amounts of data are organized around a shared database or repository, so the pattern suits applications in which data is generated by one component and used by another. The repository may be *passive*, with components reading it when they need data, or *active*, notifying the interested components when data that concerns them changes; an active repository is often called a *blackboard*.

Figure 6.18 The repository pattern

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	An integrated development environment whose tools share a repository of design information; the weather sensing system of this section.
When used	When large volumes of information are generated and must be stored for a long time; in data-driven systems where adding data to the repository triggers an action.
Advantages	Components are independent and need not know of each other. Changes made by one component are available to all. All data is managed consistently (for example, backed up together) because it is in one place.
Disadvantages	The repository is a single point of failure. Routing all communication through the repository can be inefficient. Distributing the repository across several computers may be difficult.

Example 6.5 A repository architecture for a weather sensing system

For the weather sensing and alert system of Example 6.3, (a) identify the components, (b) construct a repository-based architecture, and (c) state the disadvantages of a repository architecture for this real-time system.

Solution (a) The components are the parts of the description that perform actions or hold data. The inputs are the temperature probe, humidity sensor, and pressure transducer, and the external weather API. The processing components are the temperature collector, which calibrates temperature; the humidity and pressure packager, which normalizes humidity and formats pressure; the external API poller, which fetches remote pressure; the multi-sensor synchronizer, which fuses the three inputs into one record; the alert threshold evaluator; the notification service, which sends SMS or e-mail; and the admin panel. The shared data consists of the raw readings, the fused weather records, the configured thresholds, and the alert log.

(b) In a repository architecture (Figure 6.19) the shared data is placed in one weather data repository, and every processing component reads from and writes to it. The collectors write calibrated readings; the synchronizer reads them and writes fused records; the evaluator reads records and thresholds and writes alerts; the notification service and the admin panel read the alerts. No component calls another, so a new consumer, such as a forecasting module, can be added without changing the existing components.

(c) The disadvantages for a real-time alerting system are:

1. The repository is a *single point of failure*: if it fails, no data flows and no alerts are raised, so it must be replicated.
2. All communication passes through the repository, which adds *latency* and makes it a *bottleneck* when many sensors write at once.
3. With a passive repository, consumers learn of new data only when they next read it, so *polling delays alerts*; an active (notifying) repository is needed for timely alerts.
4. A repository is *difficult to distribute*, whereas sensing stations are naturally distributed.
5. All components must agree on one *data model*, so a change to the record format affects every component.

Answer (a) Temperature collector, humidity and pressure packager, external API poller, multi-sensor synchronizer, alert threshold evaluator, notification service, admin panel, and the shared weather data. (b) Figure 6.19. (c) Single point of failure, latency and bottleneck, delayed alerts under polling, poor distribution, and a shared data model that is hard to change. ■

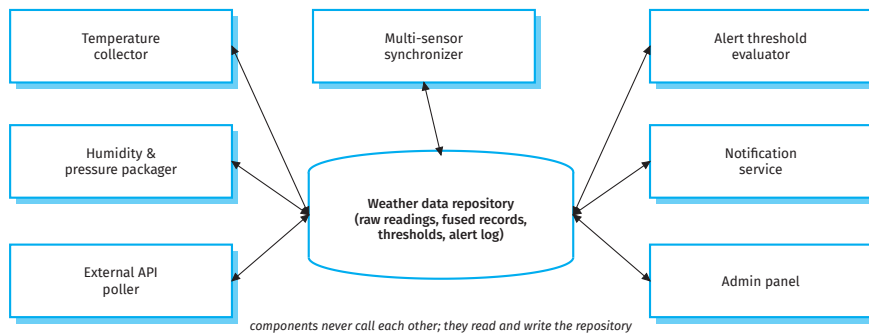


Figure 6.19
Repository
architecture for the
weather sensing and
alert system

6.3.5 Event-driven control

Call-and-return is a *centralized* control style: one component calls others and waits for them. In an *event-driven* control style, components respond to events that are generated externally and may occur at any time. There are two forms. In a *broadcast* model, an event is sent to all components, and any component that can handle it does so. In an *interrupt-driven* model, which is used in real-time and embedded systems, each type of interrupt is associated with a handler that is started as soon as the

interrupt occurs, so the system responds quickly without continuously checking for work.

Example 6.6 A control style for sensor firmware

Firmware for a microcontroller-based device monitors temperature, humidity, and air quality in a smart home. It samples the sensors continuously and sends periodic updates to a central server. It must handle asynchronous events efficiently and minimize CPU overhead. Identify the appropriate control style and justify it.

Answer An *event-driven, interrupt-driven* control style is appropriate. A timer interrupt triggers each sensor sample and each periodic upload; a sensor ‘data ready’ interrupt and a network ‘message received’ interrupt start their own handlers. Between interrupts the processor sleeps. This is justified because (1) events are asynchronous and must be handled when they happen, not when a polling loop reaches them; (2) the CPU does no work while it waits, which minimizes overhead and power consumption; (3) each handler is small and independent, so a new sensor adds a new handler without changing the others; and (4) a threshold alarm, such as poor air quality, can be given a higher interrupt priority and served at once. A centralized polling loop would waste CPU cycles and could miss or delay events. ■

6.4 Architectural mapping using data flow

Requirements models for many systems include data-flow diagrams. Architectural mapping is the method that converts a DFD into a program structure with a call-and-return architecture. It has six steps:

1. Identify the type of information flow: transform flow or transaction flow.
2. Define the flow boundaries: where data enters and leaves the processing core.
3. Map the DFD into the program structure.
4. Define the control hierarchy by factoring.
5. Refine the design using design heuristics, to improve functional independence (high cohesion and low coupling, Chapter 7).
6. Elaborate and document the architectural description.

Two kinds of information flow determine the detailed procedure (Figure 6.20). In *transform flow*, data enters the system along incoming paths, is converted from an external form to an internal representation, is processed at a *transform centre*, and leaves along outgoing paths after conversion back to an external form. The overall flow is linear. In *transaction flow*, a single data item, the transaction, triggers one of

several alternative flows of processing. A *transaction centre* evaluates the transaction and routes it to the appropriate action path.

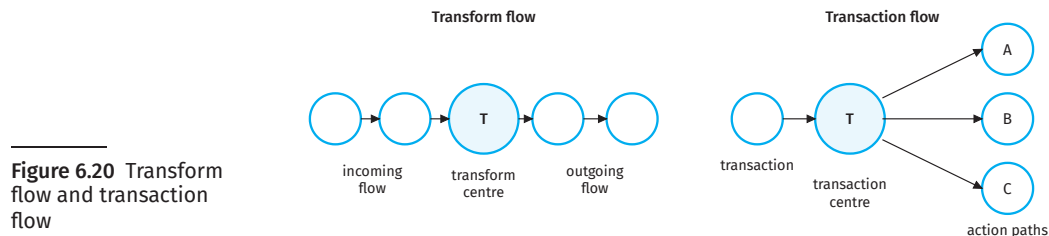


Figure 6.20 Transform flow and transaction flow

Example 6.7 Transform or transaction flow?

A food-delivery system follows this workflow: a customer places an order by selecting items from a menu; the system verifies the payment and generates an order confirmation; the order is sent to the restaurant for preparation; once it is prepared, a delivery agent is assigned; the agent picks up the order and delivers it to the customer. Does the system exhibit transaction flow or transform flow? Justify the answer by explaining how control flows through the processes.

Solution The DFD of the workflow (Figure 6.21) is a single chain. Every order enters by the same path, is converted from the customer's selection into a confirmed internal order when payment is verified (the incoming boundary), is processed by preparation and agent assignment (the transform centre), and leaves as a delivery to the customer (after the outgoing boundary). Control passes from each process to the next in a fixed sequence, and no process selects one of several alternative paths according to the type of input.

That the system handles many orders at once does not make the flow a transaction flow, and neither do the success checks in payment verification: a transaction flow requires a transaction centre that dispatches each input to a *different* action path, as the menu choice does at an ATM (Example 6.9). If the same system accepted several request types (place order, cancel order, track order, rate restaurant) through one entry point, that entry point would be a transaction centre.

Answer Transform flow. The data follows one linear path, input → transformation → output, with incoming and outgoing boundaries after payment verification and before delivery; there is no transaction centre that routes different inputs along different paths. ■

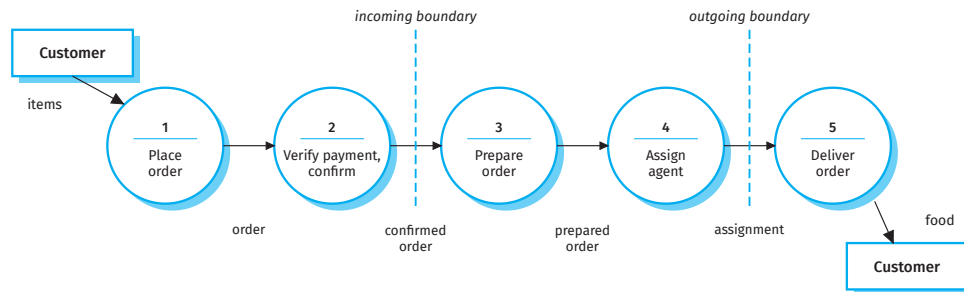


Figure 6.21 DFD of the food-delivery workflow

6.4.1 Transform mapping

Transform mapping is the set of design steps that converts a DFD exhibiting transform flow into a program structure with separate input, transform, and output branches under a main controller. The steps are:

1. Confirm that the DFD exhibits transform flow.
2. Define the flow boundaries. The *incoming boundary* is drawn where external data has become internal, valid data; the *outgoing boundary* where internal results begin to be converted to external form.
3. Identify the transform centre: the processes between the two boundaries.
4. Perform *first-level factoring*: a main controller at the top, with an input controller, a transform controller, and an output controller beneath it.
5. Perform *second-level factoring*: map each DFD process into a module beneath the controller of its domain.
6. Refine the structure to improve cohesion and reduce coupling.

Example 6.8 Transform mapping of a payroll system

A payroll system receives working hours from employees, fetches employee details from a data store, validates the data, calculates pay, generates payslips, and updates the payroll records (Figure 6.22). Map the DFD into a program structure.

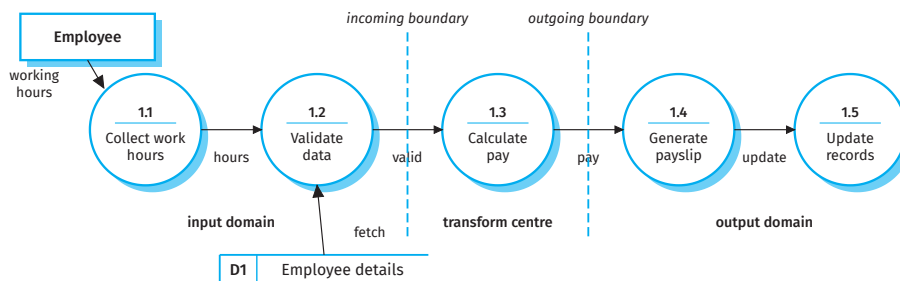


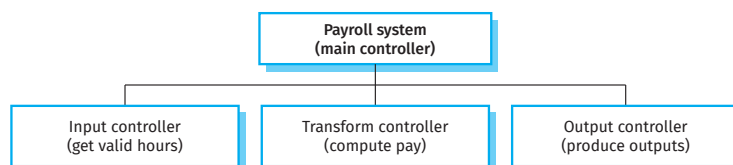
Figure 6.22 Payroll DFD with its flow boundaries

Solution Step 1. There is one input stream and one path through the system, so the flow is transform flow.

Steps 2–3. Before Validate data completes, the hours are still raw external data; afterwards they are trusted internal data, so the incoming boundary is drawn after process 1.2. Generate payslip begins converting results into an external document, so the outgoing boundary is drawn before process 1.4. The input domain is therefore {Collect work hours, Validate data}, the transform centre is {Calculate pay}, and the output domain is {Generate payslip, Update records}.

Step 4. First-level factoring places a main controller, Payroll system, over three controllers (Figure 6.23).

Figure 6.23 First-level factoring of the payroll system



Step 5. Second-level factoring places each process beneath the controller of its domain (Figure 6.24).

Step 6. Each worker module performs one task and receives only the data it needs; Calculate pay, for example, receives validated hours and the rate. If Collect work hours and Validate data are always used together and are small, they may be merged into a single 'Get valid hours' module; if Calculate pay is complex, it may be factored further into gross pay, deductions, and net pay. Controllers make decisions and workers do not call controllers, so control passes downward only.

Answer The program structure is: Payroll system → {Input controller → (Collect work hours, Validate data); Transform controller → (Calculate pay); Output controller → (Generate payslip, Update records)}. Drawing the DFD processes themselves as the final architecture, without the controllers, would not be a transform mapping. ■

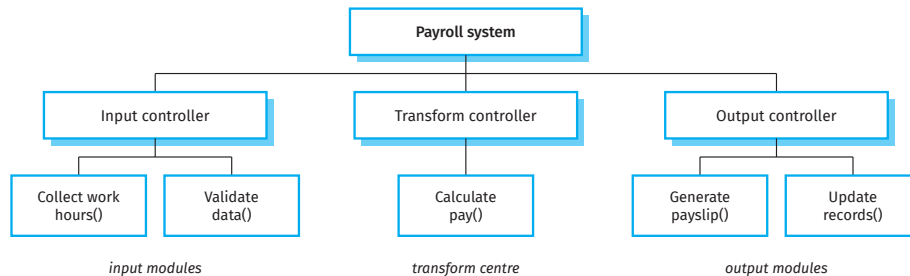


Figure 6.24 Program structure of the payroll system after second-level factoring

6.4.2 Transaction mapping

Transaction mapping converts a DFD exhibiting transaction flow into a program structure with an incoming (reception) branch and a dispatch branch in which a transaction dispatcher controls one subordinate module per action path. The steps are:

1. Identify the transaction centre: the process that controls branching, such as a request handler or menu process.
2. Identify the input transactions: the external triggers that activate the different paths.
3. Establish the processing path (action path) for each transaction type.
4. Map the DFD to a program structure: a dispatcher module with one module per transaction type.
5. Refine the architecture, factoring each action path further where needed.

Example 6.9 Transaction mapping of an ATM

At an ATM, a user's menu choice is sent to a menu selection process, which passes control to Withdraw, Deposit, or Balance enquiry (Figure 6.25). Map the DFD into a program structure.

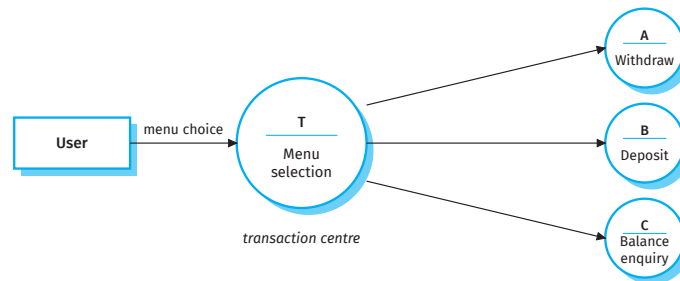


Figure 6.25 ATM DFD showing transaction flow

Solution One input triggers one of several mutually exclusive paths, so the flow is transaction flow and Menu selection is the transaction centre. The centre becomes a transaction dispatcher, each action path becomes a subordinate module, and a reception branch obtains the menu choice (Figure 6.26).

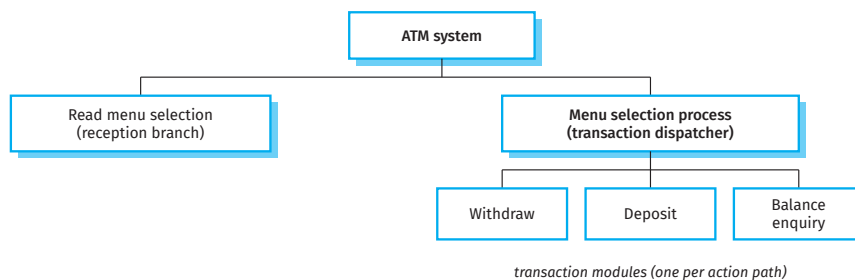


Figure 6.26 Program structure of the ATM system after transaction mapping

Each transaction module is then factored into its own action path. Withdraw controls Get amount, Check balance, Dispense cash, and Update account; Deposit controls Accept cash, Verify amount, and Update account. The shared module Update account then has fan-in 2, which is good reuse. An action path may itself have transform flow (Withdraw reads an amount, computes a new balance, and dispenses cash), in which case it is mapped by transform mapping beneath the dispatcher.

Answer ATM system → {Read menu selection (reception); Menu selection process (dispatcher) → Withdraw, Deposit, Balance enquiry}. A new transaction such as Transfer funds is added as one more subordinate of the dispatcher, without changing the existing modules. ■

Example 6.10 DFDs and a structure chart for an online shopping system

An online shopping platform lets customers browse a product catalogue, add items to a cart, place orders, pay through trusted payment gateways, and manage their accounts (personal details and order history). Administrators manage product listings, user accounts, and order processing. Products, users, and orders are kept in databases. (a) Draw the level-0 and level-1 DFDs. (b) Convert the DFD into a structure chart.

Solution (a) The context (level-0) diagram shows the three external entities and the data that crosses the boundary (Figure 6.27).

At level 1 the system is decomposed into six processes and four data stores (Figure 6.28). The customer's flows are shared among processes 1.0 to 5.0, and each flow on the context diagram reappears, so the levels balance.

(b) Each customer or administrator request (browse, update cart, order, pay, account, admin) is one input that is routed to one of several alternative processes, so the top level of the system has *transaction flow*. Transaction mapping gives a reception branch that reads and validates the request and a dispatcher with one module per transaction (Figure 6.29). The Place order path is itself a transform flow (read cart → compute total → pay → record order), so it is factored further beneath its module.

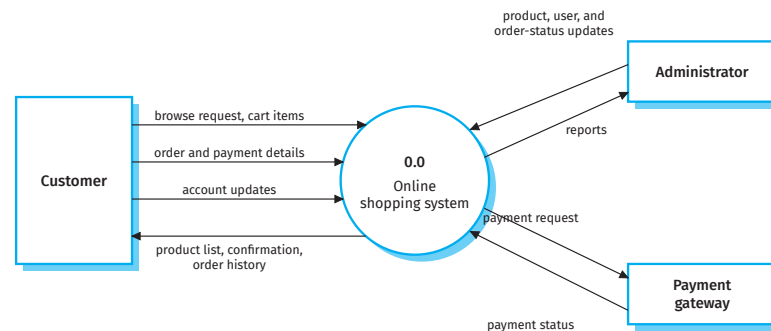


Figure 6.27 Context diagram of an online shopping system

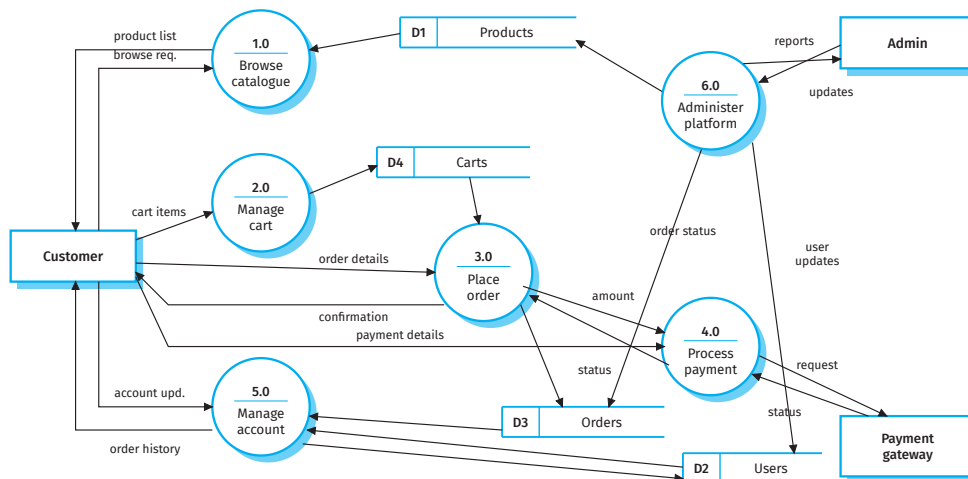


Figure 6.28 Level-1 DFD of the online shopping system

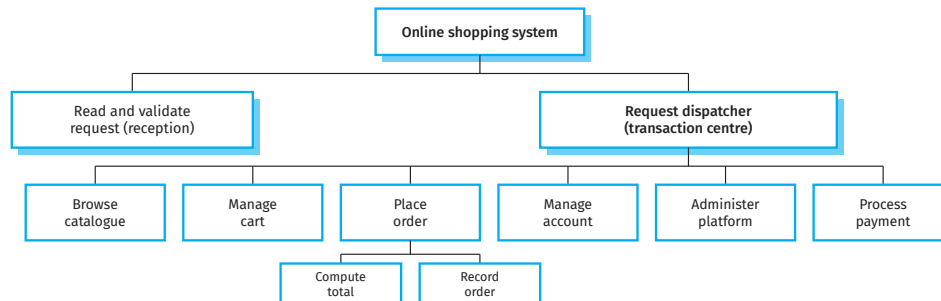


Figure 6.29 Program structure of the online shopping system after transaction mapping

Answer Level 0: Customer, Administrator, and Payment gateway around process 0.0. Level 1: 1.0 Browse catalogue, 2.0 Manage cart, 3.0 Place order, 4.0 Process payment, 5.0 Manage account, 6.0 Administer platform, with stores D1 Products, D2 Users, D3 Orders, D4 Carts. Structure chart: Online shopping system → {Read and validate request; Request dispatcher → Browse catalogue, Manage cart, Place order (→ Compute total, Record order), Manage account, Administer platform, Process payment}.

The two mappings differ in their key element (a transform centre or a transaction centre), in their top structure (input, transform, and output controllers, or a reception branch and a dispatcher), and in the way a feature is added (usually a new module in one domain, or a new module under the dispatcher). A student-records processor that reads scores, validates them, computes grades, and prints grade reports has transform flow; a library kiosk that offers Issue, Return, Renew, and Search has transaction flow.

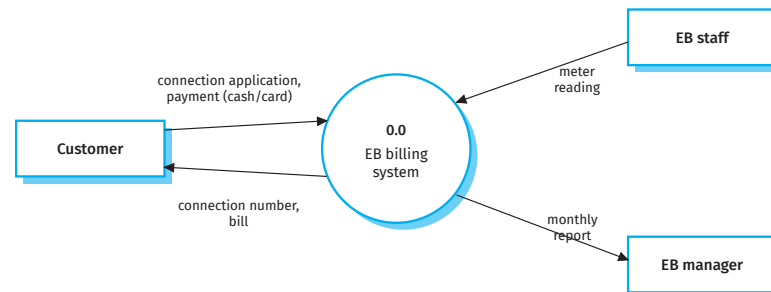
The DFDs from which a mapping starts are drawn by the rules of Section 5.4.1, usually together with a data model of the stores.

Example 6.11 DFDs and an ER diagram for an electricity billing system

An electricity board wants to automate its billing. Customers apply for a connection (domestic or commercial); EB staff take meter readings and update the system; each customer can choose to pay by cash or card; a bill is generated on payment; and monthly reports are provided to the EB manager. (a) Draw the level-0 and level-1 DFDs. (b) Design an ER diagram showing the entities Customer, Connection, Meter reading, Bill, Payment, EB staff, and Monthly report, with their primary attributes (primary keys underlined), and the relationships with their cardinalities and descriptive attributes.

Solution (a) The context (level-0) diagram (Figure 6.30) has one process, 0.0, and three external entities. The Customer sends a connection application and payments and receives a connection number and bills; EB staff send meter readings; the EB manager receives the monthly report.

Figure 6.30 Context (level-0) diagram of the electricity billing system



At level 1 (Figure 6.31) the process is decomposed into five processes with four data stores. 1.0 Register connection records the application in D1 Connections and returns the connection number; 2.0 Record meter reading checks the connection and writes the units to D2 Readings; 3.0 Receive payment reads the connection type (for the tariff) and the units consumed, and records the cash or card payment in D3 Payments; 4.0 Generate bill issues the bill once the payment is made, as the scenario requires, and stores it in D4 Bills; and 5.0 Produce monthly report summarizes the bills and payments for the manager. The five flows of the context diagram reappear, each connected to the process that handles it, so the two levels balance.

(b) In the ER diagram (Figure 6.32) a customer may hold several connections (1:N), a connection has many meter readings and many bills (1:N), and each reading is taken by one member of staff, who takes many (1:N). Each reading produces one bill (1:1), each bill is settled by one payment (1:1), and a monthly report summarizes many bills (1:N). The *descriptive attributes* belong to relationships, not to either entity: the ApplicationDate of 'applies for', the ReadingDate of 'records', and the PaidOn date of 'settled by'. The mode of payment (cash or card) is an attribute of Payment, and the type (domestic or commercial) an attribute of Connection.

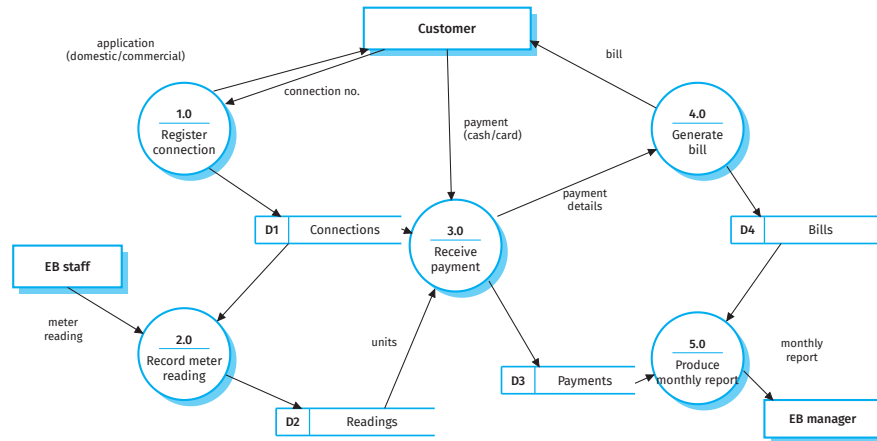


Figure 6.31 Level-1 DFD of the electricity billing system

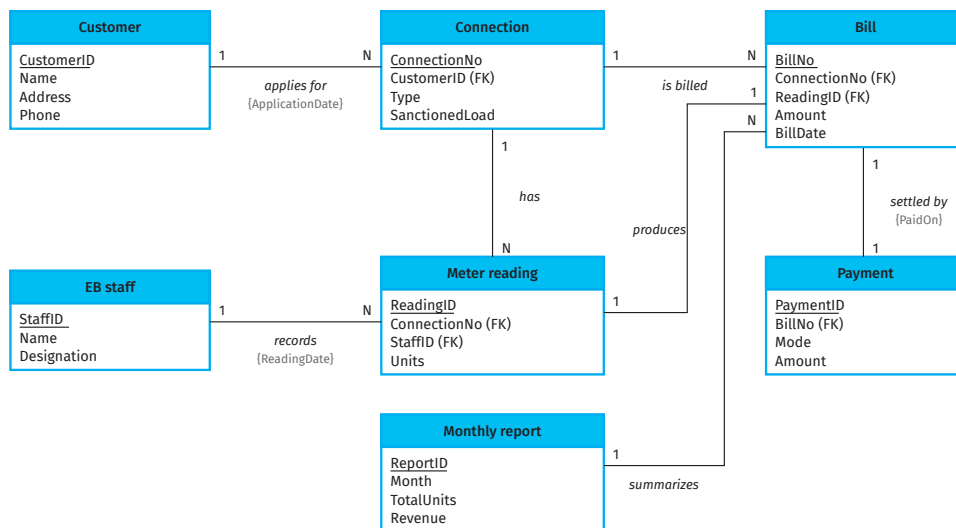
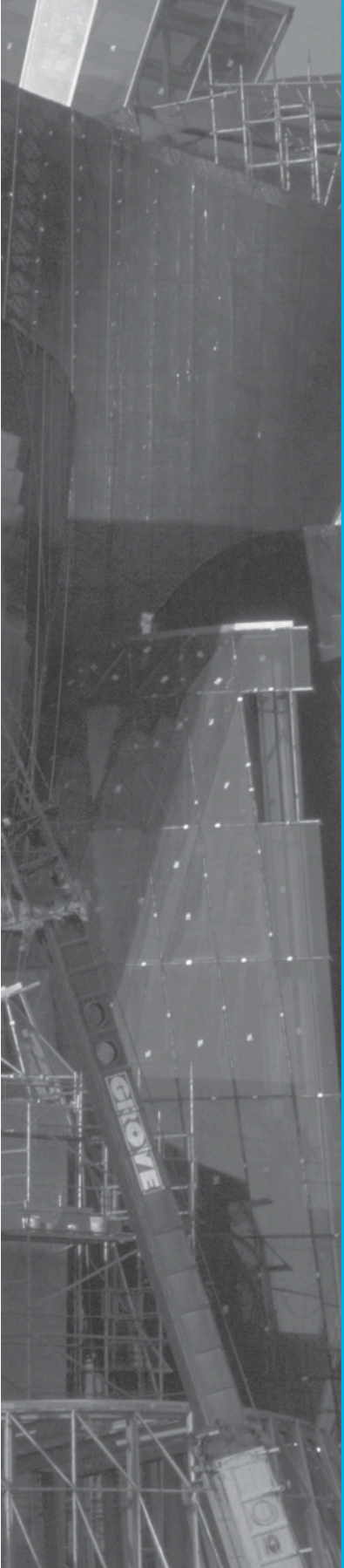


Figure 6.32 Entity-relationship diagram of the electricity billing system

Answer (a) Level 0: process 0.0 with Customer (application, payment in; connection number, bill out), EB staff (meter reading in), and EB manager (monthly report out). Level 1: 1.0 Register connection, 2.0 Record meter reading, 3.0 Receive payment, 4.0 Generate bill, 5.0 Produce monthly report, with stores D1 Connections, D2 Readings, D3 Payments, and D4 Bills. (b) Customer 1–N Connection; Connection 1–N Meter reading and 1–N Bill; EB staff 1–N Meter reading; Meter reading 1–1 Bill; Bill 1–1 Payment; Monthly report 1–N Bill. ■

KEY POINTS

- A software architecture describes the components of a system and how they interact. An architectural design must specify structural properties, extra-functional properties, and families of related systems.
- A control hierarchy is described by its depth, width, fan-in, and fan-out and by superordinate and subordinate modules. The sum of fan-outs equals the sum of fan-ins.
- Horizontal partitioning divides the program into branches or layers by major function; vertical partitioning (factoring) puts control at the top and workers at the bottom, so that most changes affect only workers.
- An architecture can be described by structural, framework, dynamic, process, and functional models.
- The layered, client-server, and call-and-return patterns are frequently combined: clients request services from servers whose software is layered and whose requests are handled by calls and returns.
- In a repository architecture, components share data only through a central repository. It decouples producers from consumers but is a single point of failure and a possible bottleneck.
- Architectural mapping converts a DFD into a program structure. Transform mapping identifies incoming and outgoing boundaries and a transform centre and factors the structure into input, transform, and output branches; transaction mapping identifies a transaction centre and builds a dispatcher with one module per action path.



7

Design and implementation

Objectives

The objective of this chapter is to introduce the principles and concepts of software design and the design of user interfaces. When you have read the chapter, you will:

- understand how the requirements model is translated into data architectural, interface, and component-level designs, and the principles that guide design;
- understand the design concepts of abstraction, separation of concerns, modularity, refinement, and refactoring;
- be able to classify the cohesion of a module and the coupling between modules and explain why high cohesion and low coupling are desirable;
- know the golden rules, models, process, and principles of user interface design and be able to lay out and justify interface screens.

Contents

- 7.1 The design model and design principles
- 7.2 Design concepts
- 7.3 Cohesion and coupling
- 7.4 User interface design

Software design is the activity in which the *what* of a system, captured in the requirements model, is converted into the *how* of a buildable solution. It is an iterative process that translates requirements into a blueprint for constructing the software. It begins with a high-level, abstract representation and becomes progressively more detailed, while every refinement remains traceable to the requirements.

Design is a core part of software engineering and is applied whatever software process is used: waterfall, incremental, spiral, or agile. It begins after the requirements have been analyzed and modeled, and it is the last modeling step before construction (code generation and testing). Design matters for several reasons. It determines how easily the software can be built and later changed; it is the place where quality is established, since quality cannot be added after coding; it produces representations that can be assessed before any code exists; and it is the foundation for coding, testing, deployment, and maintenance. Without design, the result is a system that is unstable, difficult to test, and impossible to assess.

This chapter describes the design model and the principles and concepts of good design, the measures of module quality called cohesion and coupling, and the design of user interfaces.

7.1 The design model and design principles

The requirements model of Chapter 5 contains scenario-based, class-based, flow-oriented, and behavioral elements. Design uses these elements to produce four design models (Figure 7.1):

1. *Data/class design* converts analysis classes into design classes and defines the data structures required for implementation. It forms the base of the pyramid, because every other design decision depends on the data the system manipulates.
2. *Architectural design* defines the relationships between the major structural elements of the software, using architectural styles and patterns (Chapter 6).
3. *Interface design* specifies how the software communicates with other systems and with the people who use it: the flow of information across the system boundary and the associated behavior (Section 7.4).
4. *Component-level design* converts the structural elements of the architecture into procedural descriptions of each component: its internal algorithms and data.

The correspondence is not one-to-one. Every design model draws on several analysis elements; component-level design, for example, uses class-based, flow, and behavioral models together.

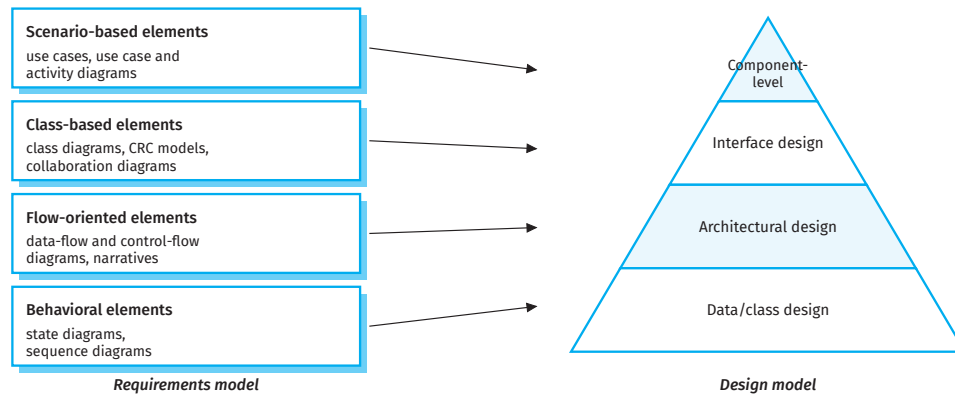


Figure 7.1 Translating the requirements model into the design model

7.1.1 Data design and entity–relationship diagrams

Data design begins from a *data model* of the information the system keeps, usually drawn as an entity–relationship (ER) diagram (Figure 7.2). An *entity* is a thing about which data is stored, such as Student or Course; a *weak entity* (double rectangle) cannot be identified without its owner, as an Installment is identified only within its Loan. *Attributes* describe entities; the *key* attribute, underlined, identifies each instance uniquely; a *derived* attribute (dashed) is computed from others, as age from date of birth; and a *multivalued* attribute (double ellipse) can hold several values, as the phone numbers of a customer. A *relationship* (diamond) associates entities, and its *cardinality* states how many instances of each take part: one-to-one (1:1), one-to-many (1:N), or many-to-many (M:N). An attribute that belongs to the relationship rather than to either entity, such as the date on which a student enrolls in a course, is a *descriptive attribute*. When the model is converted to tables, each entity becomes a table keyed by its key attribute, a 1:N relationship becomes a foreign key on the N side, and an M:N relationship becomes an associative table holding both keys and any descriptive attributes. Worked ER diagrams are given in Example 5.6 and Example 6.11. The same diagrams can be drawn in UML class notation, with entities as classes and cardinalities as multiplicities, as in this book's figures.

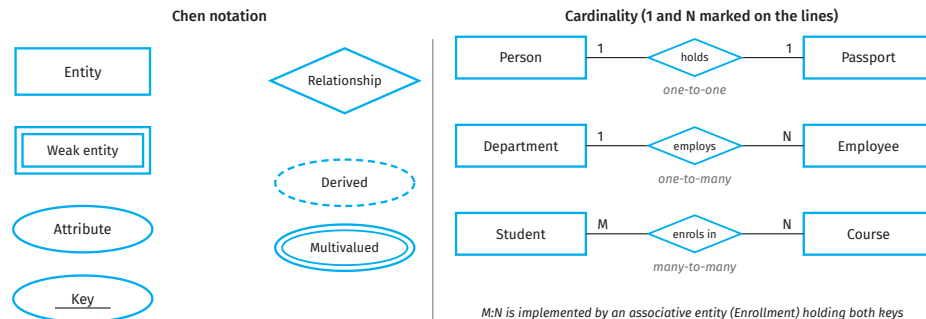


Figure 7.2 Notation of entity-relationship diagrams

7.1.2 Quality and principles of design

A design is evaluated through technical reviews. A good design must implement all the explicit requirements of the requirements model and accommodate the implicit requirements expected by stakeholders; it must be a readable, understandable guide for those who code, test, and support the software; and it must give a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Ten principles guide both the design process and the design product:

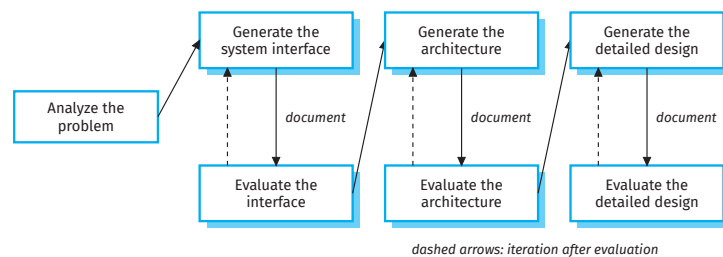
1. *Avoid tunnel vision* Consider all requirements and alternative approaches, not a single aspect such as performance.
2. *Be traceable to the analysis model* Every design element should have a justification in the requirements, so that nothing is omitted and nothing unnecessary is added.
3. *Do not reinvent the wheel* Reuse libraries, components, and proven patterns.
4. *Minimize intellectual distance* Structure the software as the real-world problem is structured; a library system whose modules are members, books, and loans is easier to understand than one organized around arbitrary technical divisions.
5. *Exhibit uniformity and integration* Keep structure, style, and logic consistent across components developed by different teams.
6. *Accommodate change* Use modular, scalable structures that absorb new requirements.
7. *Degrade gracefully* Bad data or errors must not crash the whole system; error handling and fallbacks keep it running with reduced function.
8. *Design is not coding, and coding is not design* Design plans structure at a higher level of abstraction than code, even when it includes pseudocode for complex

algorithms.

9. *Assess quality during design* Evaluate efficiency, scalability, and correctness while designing, not at the end.
10. *Review to minimize conceptual errors* Peer reviews catch logical flaws before they reach the code.

The design process itself proceeds top-down in three iterative stages (Figure 7.3). The system interface, the architecture, and the detailed design are each generated, documented, and evaluated in turn; after each evaluation the designer may return and improve the document, and the process ends with a formal review of the complete design.

Figure 7.3 The design process: generate, document, and evaluate at three levels



7.2 Design concepts

Design concepts help designers answer three questions: how the software should be divided into smaller components; how the details of functions and data structures can be separated from the overall conceptual representation of the software; and what criteria define the quality of a design. Abstraction and refinement answer the second question; modularity, separation of concerns, and architecture the first; cohesion, coupling, and the design principles the third.

7.2.1 Abstraction

Abstraction is the process of hiding implementation details and showing only the essential features of a system. At the highest level of abstraction a solution is stated in the language of the problem domain; at lower levels it is described more and more procedurally, until at the lowest level it can be implemented directly. Abstraction reduces complexity because, at any moment, the designer deals only with the details relevant at the current level. There are three types:

1. *Procedural abstraction* A process is broken into named procedures that perform specific tasks without exposing their implementation. A function

`calculateInterest(principal, rate, time)` computes interest without revealing the formula.

2. *Data abstraction* A data structure is defined so that its implementation is hidden and only the necessary operations are exposed. A Calculator class whose result attribute is private and read only through `getResult()` can later store the result differently without affecting its callers.
3. *Control abstraction* The details of control flow are hidden so that the programmer thinks about the task rather than the mechanism. A for-each loop or an iterator replaces manual management of loop counters; a menu-driven `runCalculator()` function hides the loop, the switch statement, and the termination logic.

Abstraction is related to, but not the same as, encapsulation. Abstraction is a design concept concerned with *what* is exposed; encapsulation (information hiding) is the mechanism, such as the private keyword, that prevents access to what is not exposed.

7.2.2 Design patterns and separation of concerns

A *design pattern* is a proven solution to a recurring design problem, recorded with its context and the essential structure of the solution so that it can be reused. A pattern helps a designer decide whether it applies to the current problem, whether it can be reused to save effort, and whether it can guide the development of a similar pattern. Model-view-controller, singleton, and observer are familiar examples.

Separation of concerns is the principle that a complex problem is easier to solve when it is broken into pieces that can each be solved and optimized independently. A *concern* is a feature or behavior specified in the requirements. The justification is divide and conquer: solving one large problem is harder than solving several small ones. Separation of concerns is the principle; modularity, abstraction, functional independence, and refinement are the techniques that realize it.

7.2.3 Modularity

Modularity is the practice of dividing a system into smaller, manageable, and independent components called modules, which can be combined to form the complete system. A monolithic program cannot easily be understood by one reader, because the number of control paths, variables, and interactions is too large. Dividing it into modules makes each part intellectually manageable. Modularity makes software easier to understand, improves reuse, simplifies debugging and maintenance, allows new features to be added as new modules, and allows teams to work in parallel.

Five criteria are used to evaluate how well a design method supports effective modularity:

1. *Modular decomposability* The problem can be divided systematically into subproblems, such as a banking system divided into account management, loan processing, and transactions.
2. *Modular composability* Existing modules can be assembled into new systems, such as a payment-gateway module reused across e-commerce platforms.
3. *Modular understandability* Each module can be understood as a standalone unit.
4. *Modular continuity* Small changes in requirements affect individual modules only; a change in tax rules affects only the tax module.
5. *Modular protection* An error inside one module does not propagate to others.

More modules are not always better. As the number of modules increases, the cost of developing each module falls, but the cost of integrating them (interfaces, calls, communication) rises (Figure 7.4). Too few modules give *undermodularity*, with large, complex modules approaching a monolith; too many give *overmodularity*, with high integration cost. The total cost is lowest in a region around some number M of modules, which cannot be predicted exactly but which a designer should aim for.

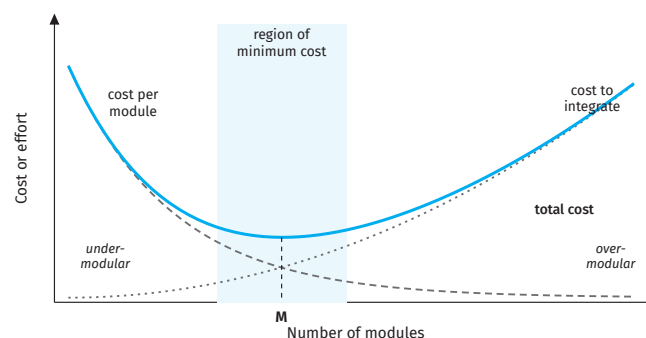
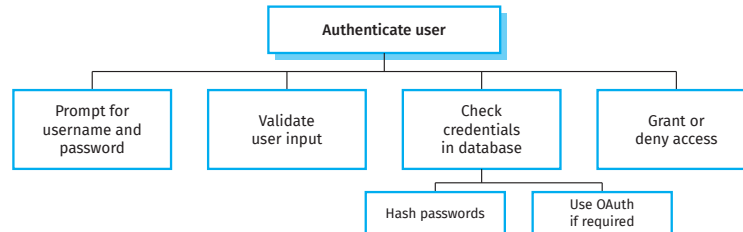


Figure 7.4 Modularity and software cost

7.2.4 Refinement and refactoring

Refinement (stepwise refinement) is a top-down strategy in which a function is elaborated level by level, from an abstract statement to detailed procedural steps that can be coded directly (Figure 7.5). Abstraction and refinement are complementary: abstraction lets a designer state a procedure while suppressing detail, and refinement reveals the detail as design proceeds. Refinement in design corresponds to partitioning in requirements analysis.

Figure 7.5 Stepwise refinement of a login function



Refactoring is a reorganization technique that improves the internal structure of a design or its code without changing its external behavior. Its purposes are to eliminate redundancy and unused elements, to improve efficiency, and to enhance readability, maintainability, and scalability. A designer looks for poorly constructed elements, low-cohesion components that perform unrelated functions, and unnecessary complexity. For example, a ReportManager that fetches sales records, computes statistics, and e-mails a PDF can be split into SalesRepository, SalesStatistics, and ReportMailer; managers receive the same report, but each component now has one responsibility. Refactoring does not fix defects, because it does not change behavior; debugging does. Section 7.3.3 shows refactoring driven by cohesion and coupling.

Example 7.1 Abstraction or refinement?

A payment system for an e-commerce website can be structured in two ways. The first creates a general Payment class, with Credit_Card, PayPal, and UPI as subclasses. The second starts with a single Payment_Processor module, which is later divided into separate handlers for each payment method. Which method uses abstraction and which uses refinement? Which is better for adding new payment methods in the future?

Solution The first method uses *abstraction*. Payment states what every payment has in common, such as validate() and process(amount), and hides how each kind of payment does it; the subclasses supply the details. The second method uses *refinement*: it begins with one abstract module and elaborates it top-down, dividing Payment_Processor into more detailed handlers.

The abstraction-based design is better for extension. A new method, such as a wallet or net banking, is added as a new subclass of Payment, and the code that calls payment.process() does not change, because it depends only on the abstraction (the design is open for extension but closed for modification). Each subclass can be tested on its own, and a fault in one does not affect the others. In the refined design, the knowledge of which handler to call stays inside Payment_Processor, so every new method means changing and re-testing that existing module.

Answer Method 1 (Payment superclass with subclasses) is abstraction; method 2 (splitting Payment_Processor into handlers) is refinement. Abstraction is better for adding payment methods: new subclasses are added without modifying existing code.

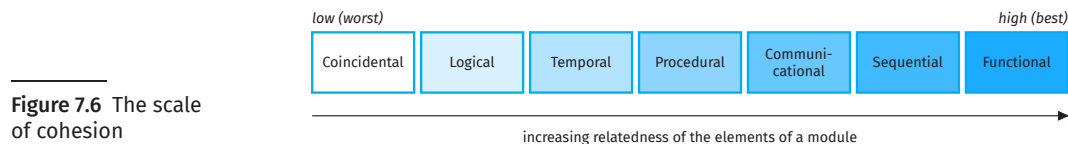
■

7.3 Cohesion and coupling

Component-level design defines the internal structure of each module identified by the architecture. Its quality is judged by two complementary measures. *Cohesion* looks inside a module and asks how strongly its elements belong together. *Coupling* looks between modules and asks how strongly they depend on one another. Together they express *functional independence*: a module is functionally independent when it has a single, well-defined purpose (high cohesion) and a minimal, simple interface to other modules (low coupling). The rule *high cohesion, low coupling* is the most important principle of modular design.

7.3.1 Cohesion

Cohesion is the degree to which the elements inside a module (its functions, data, and statements) are related to each other and belong together. A highly cohesive module performs a single, well-defined task. A module with low cohesion groups loosely related or unrelated functionality, which makes it harder to understand, test, and change. Seven levels of cohesion are distinguished, from lowest to highest (Figure 7.6). Some texts omit procedural and sequential cohesion and use a five-level scale; this book uses the full seven-level scale.



1. *Coincidental cohesion* (lowest) Unrelated functions are grouped together for no reason, usually through lack of planning; for example, a module that prints a report, clears the screen, and plays music. A change to one task risks disturbing the others that happen to live beside it.
2. *Logical cohesion* Functions that perform logically similar operations, or that belong to the same category, are grouped together, and the one executed is selected by an external control such as a flag. An `output_handler(output_type, data)` that prints to screen, printer, or file according to `output_type` is logically cohesive. Every value of the flag must be tested, and a new output type requires changing the shared module.
3. *Temporal cohesion* Functions are grouped because they are executed at the same time or in response to the same event, not because they are related: for example, a start-up module that initializes variables, opens files, and clears the screen, or a set of actions all triggered when an order is updated.
4. *Procedural cohesion* Elements are grouped because they must be executed in a fixed order, although they do not operate on the same data; for example, check

permissions, then open the file, then log the access.

5. *Communicational cohesion* Elements are grouped because they operate on the same data: get a temperature, log it, and display it. The functions share an input or output, which ties them together.
6. *Sequential cohesion* Elements are grouped because the output of one is the input of the next, like a chain: read the order, validate it, and compute its total.
7. *Functional cohesion* (highest) Every element contributes to a single, well-defined task; for example, `calculate_average(numbers)`.

A practical test is to describe the module in one sentence. If the sentence has a single verb and object ('computes the average'), the module is functionally cohesive. If it needs 'and' joining unrelated activities, cohesion is lower; 'at start-up' or 'when ... happens' suggests temporal cohesion; 'first ... then ...' suggests procedural cohesion; 'depending on the flag' or 'of this kind' suggests logical cohesion.

Higher cohesion is desirable because a module that focuses on one task is easier to understand and modify, has clear expected results and is therefore easier to test, and is self-contained and therefore easier to reuse and extend.

Example 7.2 Classifying the cohesion of two code fragments

These fragments illustrate a measure within a module that influences maintainability and reuse. Identify the type of cohesion (the bonding within the module) in each and state whether it is high or low.

```
(i) def calculate_average(numbers):
    """Calculates the average of a list of numbers."""
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)

(ii) class Utility:
    def print_message(self, message):
        print(message)
    def calculate_square(self, number):
        return number * number
    def read_file(self, filename):
        with open(filename, 'r') as file:
            return file.read()
```

Solution The measure within a module is cohesion (the measure between modules is coupling). Apply the one-sentence test to each module.

(i) 'calculate_average computes the average of a list of numbers.' One verb, one object: every statement, including the empty-list guard, contributes to that result.

(ii) 'Utility prints a message *and* squares a number *and* reads a file.' The three methods share no data, no order of execution, and no purpose; they have been placed together only because each is a 'utility'.

Answer (i) Functional cohesion, which is high and desirable: all elements contribute to a single, well-defined task. (ii) Coincidental cohesion, which is low and undesirable: the elements are grouped arbitrarily with no logical relationship. Utility should be split, for example into a messaging module, a math module, and a file module. ■

Example 7.3 Cohesion in the modules of a food-delivery system

Classify the cohesion of each of the following modules of a food-delivery order system.

1. The Order Validation module performs a single, unified purpose.
2. Order handling passes validated order details on to restaurant processing in a chained workflow, the output of each step being the input of the next.
3. The Restaurant Manager processes the order id, the items, and the restaurant id, all operating on the same data.
4. The Delivery Partner Allocator's tasks (select rider → check location → assign order) follow a fixed execution order.
5. The Real-Time Tracker groups GPS updates and ETA calculations because they fall under the same category of work.
6. Notification generation (SMS, e-mail, push) is triggered together when the order is updated.
7. The Central Order Database module groups orders, riders, restaurants, payments, and tracking because they share data.
8. Unrelated tasks such as analytics, rider assignment, and GPS tracking are placed in one module.
9. The Payment Gateway module has only one responsibility: authorizing transactions.
10. Start-up tasks such as syncing restaurant data, refreshing rider availability, and preloading menus are grouped together.

Solution Ask for each module why its elements are together: one task (functional), a data chain (sequential), shared data (communicational), fixed order (procedural), same time or event (temporal), same category chosen by a flag (logical), or no reason (coincidental).

1. Functional. 2. Sequential. 3. Communicational. 4. Procedural. 5. Logical. 6. Temporal. 7. Communicational. 8. Coincidental. 9. Functional. 10. Temporal. ■

Example 7.4 The contribution of temporal cohesion

How does temporal cohesion contribute to the organization of a module's functionality? Give an example from a real application.

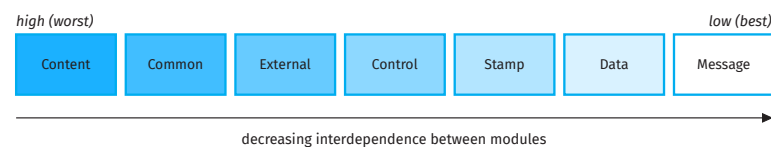
Answer Temporal cohesion groups actions that must happen at the same time or in response to the same event. Its contribution is organizational: all the actions for one moment are in one place, so they are guaranteed to run together, the sequence of

start-up (or shutdown) is easy to read and check, and nothing is forgotten when the event occurs. For example, when a mobile banking app starts, one start-up module loads the configuration, restores the session, checks for updates, and preloads the account summary. Its weakness is that the actions are otherwise unrelated, so each must be changed without disturbing the others. It lies low on the cohesion scale, and the best designs keep such a module as a thin coordinator that *calls* functionally cohesive modules (`loadConfig()`, `restoreSession()`, and so on) rather than containing their logic. ■

7.3.2 Coupling

Coupling is the degree of interdependence between modules: how closely one module is connected to another. Low coupling is desirable because it makes the system more modular and easier to maintain, test, and extend; high coupling increases complexity and makes change ripple from module to module. Seven types are distinguished, from strongest (worst) to weakest (best) (Figure 7.7):

Figure 7.7 The scale of coupling



1. *Content coupling* One module directly modifies, or relies on, the internal data or logic of another, for example by setting `module_b.internal_data = 42`. It violates information hiding; if B changes its internals, A breaks.
2. *Common coupling* Several modules share global data, such as a global configuration dictionary. A fault seen in one module may be caused by any other module that writes the global data.
3. *External coupling* Modules depend on an externally imposed format, protocol, device, or service, such as a third-party authentication API. The system is exposed to changes or failures outside its control.
4. *Control coupling* One module controls the behavior of another by passing a control flag, as in `function_b('start')`. The caller must know about the callee's internal decisions; the callee in control coupling usually has logical cohesion.
5. *Stamp coupling* A whole data structure is passed when only part of it is needed, as when `process_student(student)` uses only `student.id`. Changes to the structure ripple to the callee unnecessarily.
6. *Data coupling* Modules share only the elementary data they need, as parameters, as in `greet_user(name, age)` or `gateway.charge(99.99, "USD")`.

7. *Message coupling* (weakest) Modules communicate only by sending messages or events, asynchronously, with no direct knowledge of each other (Figure 7.8). An OrderService publishes an order_placed event to an event bus; an EmailService and an InventoryService subscribe to it. The publisher does not know that the consumers exist, so a new consumer, such as an AnalyticsService, is added without changing it, and the services fail independently: if e-mail is down, orders are still processed.

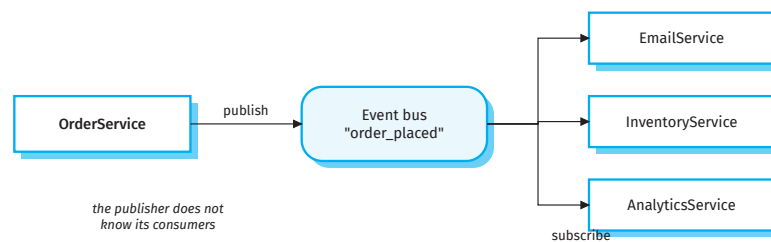


Figure 7.8 Message coupling through an event bus

Example 7.5 Classifying cohesion and coupling together

Classify each fragment. (1) `end_of_day()` closes the database, backs up the logs, and sends a summary e-mail. (2) `shipping_cost(order)` uses only `order.weight` from an Order record with twenty fields. (3) `render(report, fmt)`, with `fmt` in {"pdf", "csv", "html"} and a separate branch for each. (4) Two modules both read and write a global variable `current_user`. (5) `greet_user(name, age)` is called with just the two values it prints.

Answer (1) Temporal cohesion: the tasks are grouped only because they run at the end of the day. (2) Stamp coupling: a whole record is passed where one field is needed; passing `order.weight` would give data coupling. (3) Inside `render`, logical cohesion; between the caller and `render`, control coupling, since `fmt` selects the behavior. (4) Common coupling through shared global data. (5) Data coupling, the weakest and best form. ■

Example 7.6 Coupling and cohesion in a library management system

Identify the type of coupling in each interaction, and the type of cohesion the modules exhibit where applicable. (i) When a user logs in, the User Management module verifies the credentials, checks for pending fines, and sends the user's status (active or inactive) and fine details to the Borrowing module, which decides whether the user can borrow. (ii) When a user borrows a book, the Borrowing module directly accesses and updates the availability status in the Book Management module's database tables. (iii) The Borrowing module calls the Notification module, passing the user's contact details, the book title, and the due date as function arguments; the Notification module sends an e-mail or SMS. (iv) The Report Generation module retrieves structured data (JSON responses) from the Borrowing, Book Management, and User Management modules to report overdue and most-borrowed books, reading but never modifying it. (v) Inside the Borrowing module the steps are, in order: validate user eligibility, check book availability, update borrowing records, adjust book availability, notify the user. (vi) The Notification module checks the user's preferred channel (e-mail, SMS, or push) and selects a function with a chain of if-else conditions.

Solution (i) *Data coupling*: only the elementary values the Borrowing module needs (a status and the fine details) are passed. User Management has *functional cohesion*, because its elements serve the single task of establishing the user's standing.

(ii) *Content coupling*, the strongest form: the Borrowing module reaches into another module's internal data, so a change to the Book Management tables breaks it. The remedy is to call an operation such as `bookManager.markBorrowed(bookId)`. Book Management itself remains functionally cohesive.

(iii) *Data coupling*: each argument is an elementary value that the Notification module uses. If the whole user record were passed when only the contact details are used, the coupling would become stamp coupling.

(iv) *Stamp coupling*: whole structured records (JSON documents) are passed and the report uses only some of their fields. Because the data is only read, the coupling is weak, and it is not common coupling, since nothing is shared and written. Report Generation is functionally cohesive.

(v) *Procedural cohesion*: the steps must be performed in a fixed order, but each works on different data (the user, the book, the loan records), rather than consuming the output of the previous step. If each step passed its result to the next, as in read, validate, compute, the cohesion would be sequential.

(vi) *Logical cohesion*: the e-mail, SMS, and push functions are grouped because they are the same kind of operation, and one is selected by a condition. If the caller passed the channel as a flag, the caller and the module would also be control coupled.

Answer (i) Data coupling; functional cohesion. (ii) Content coupling. (iii) Data coupling. (iv) Stamp coupling (read-only). (v) Procedural cohesion. (vi) Logical cohesion. ■

Example 7.7 Coupling in a weather sensing system

Figure 7.9 shows the modules of the weather sensing and alert system of Example 6.3. The temperature module receives readings from the probe and passes a calibrated temperature to the synchronizer; the packager passes a packet of humidity, raw pressure count, and sensor id; the API poller passes the raw data received from a remote weather service; and the evaluator reads and writes a global alert log that also holds the thresholds set from the admin panel. Determine and justify the coupling of interactions a to e.

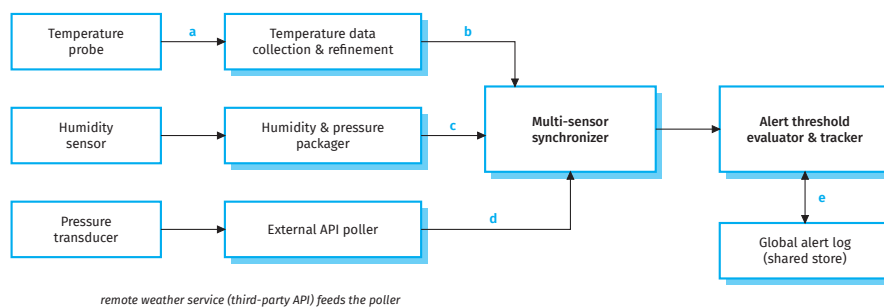


Figure 7.9 Module interactions in the weather sensing and alert system

Answer (a) Sensors → temperature data collection: *external coupling*, because the module depends on the device's externally imposed interface and signal format. (b) Temperature collection → synchronizer: *data coupling*, because one elementary value, the calibrated temperature, is passed. (c) Humidity and pressure packager → synchronizer: *stamp coupling*, because a composite packet is passed and the synchronizer depends on its structure. (d) External API poller → synchronizer: *external coupling*, because the raw data is still in the third-party service's format, so a change to the remote API affects the synchronizer; if the poller converted it to a pressure value first, this would be data coupling and the external dependency would be confined to the poller. (e) Evaluator ↔ global alert log: *common coupling*, because the log is shared global data that is read and written by the evaluator and the admin panel, so a fault in either can corrupt the other's data. ■

Example 7.8 Identifying and removing coupling in a checkout

Identify the coupling in the first version of `ShoppingCart.checkout()`, and state what it becomes in the second.

```
# Version 1
def checkout(self):
    db = DatabaseConnection("mysql://...")
    db.save(self.items)
    global current_user
    send_email(current_user.email, "Receipt")
    payment = PaymentProcessor()
    payment._balance -= self.total()

# Version 2
def checkout(self, payment_gateway, event_bus):
    transaction_id = payment_gateway.charge(self.total())
    event_bus.publish("order_completed",
        {"items": self.items, "transaction_id": transaction_id})
```

Answer Version 1 has three strong forms of coupling: *external* coupling, because `checkout` creates a connection to one specific database from a hard-coded address; *common* coupling, because it reads the global `current_user`, which any module can change; and *content* coupling, because it subtracts from the private field `_balance` of `PaymentProcessor`. Version 2 has *data* coupling with the payment gateway, which receives one number and returns an identifier, and *message* coupling with the e-mail, inventory, and database services, which react to the `order_completed` event. The gateway and the event bus are passed in (dependency injection), so either can be replaced by a test double. ■

Example 7.9 Architecture, coupling, and cohesion for a collaborative editor

A cloud-based document editor lets many users edit a document simultaneously, track changes, and chat. It must be scalable and allow later additions such as AI writing assistance and advanced versioning. (a) Identify an appropriate architectural style and justify it. (b) State design assumptions for three types of coupling and two types of cohesion that give good modularity.

Solution (a) A *layered* architecture, deployed client-server, is appropriate: a presentation layer (editor, change view, and chat in the browser), a collaboration and business-logic layer (session management, merging concurrent edits, change tracking, versioning, and chat), a service-integration layer (cloud storage and, later, AI services), and a data layer (documents, versions, messages). Each layer uses only the one below, so the user interface, the collaboration logic, and the storage can be changed or scaled independently, and a new AI assistant is added in the service layer without touching the editor. Because the modules are only data coupled (below), individual services, such as chat, can later be scaled out on their own servers.

(b) *Coupling* (choose the weak forms and control the strong ones):

1. *Data coupling* between most modules: the editor passes only an edit operation (position, inserted text, user id) to the collaboration module, and chat passes only a message and a channel id.
2. *Stamp coupling*, used deliberately only where the whole record is needed: the versioning module receives a complete ChangeSet record because it stores all its fields.
3. *External coupling* confined to adapter modules: only a StorageAdapter depends on the cloud storage API and only an AIAdapter on the future AI service, so a change of provider affects one module.

Content and common coupling are avoided: no module writes another's internal data, and there is no shared global document state.

Cohesion:

1. *Functional cohesion* for each module: Merge concurrent edits, Record version, and Deliver chat message each do one thing.
2. *Communicational cohesion* for the change-tracking module, whose functions (record change, list changes, accept or reject change) all operate on the same document-change data.

Answer (a) Layered architecture (with client-server deployment), justified by modularity, independent scaling, and easy extension. (b) Coupling: data (edit operations and messages), controlled stamp (whole change records for versioning), and external coupling isolated in adapters. Cohesion: functional (single-task modules) and communicational (change tracking on shared change data). ■

The two properties reinforce each other. When related elements are placed together (high cohesion), fewer connections are needed to other modules (low coupling); when modules communicate through a few simple parameters, each can be understood in isolation. The results are concrete: a requirement change affects one module (modular continuity); a fault is unlikely to propagate (modular protection); each module can be unit-tested with simple stubs and drivers; functionally cohesive, data-coupled modules can be reused elsewhere; and teams can work in parallel. The combinations are summarized in Figure 7.10. Note the difference between communicational cohesion (functions *inside one module* sharing data, which is acceptable) and common coupling (*different modules* sharing global data, which is not).

Figure 7.10
Combinations of
cohesion and
coupling

		High cohesion	Low cohesion
Loose coupling		Ideal focused modules that work independently	Tolerable unfocused but isolated modules
Tight coupling		Fragile focused modules with brittle dependencies	Worst unfocused and entangled: a maintenance nightmare

7.3.3 Refactoring designs

Refactoring (Section 7.2.4) is guided by cohesion and coupling. The usual signs that a design needs it are a class with several unrelated responsibilities (low cohesion); hard-coded dependencies on particular databases, providers, or file formats (external coupling); direct use of another class's internal fields (content coupling); shared global state (common coupling); and flags that switch a function's behavior (control coupling). The standard remedies are to *extract class*, giving each responsibility its own class; to *introduce an interface* and pass the implementation in (dependency injection), so that the class depends on an abstraction; to *encapsulate a field* behind operations; and to *replace a direct call by an event*. After each step the tests are run again, because the external behavior must not change.

Example 7.10 Three design refactorings

Refactor (a) a UserManager with the operations login, sendWelcomeEmail, calculateDiscount, and exportToPDF; (b) an Order that holds a PayPalPaymentProcessor and a MySQLDatabase; and (c) a CheckoutManager that validates an order, charges it through PayPal, updates stock in MySQL, and sends e-mail through Gmail, all by static calls. State the cohesion and coupling before and after.

Solution (a) UserManager has *coincidental* cohesion: authentication, e-mail, pricing, and reporting have nothing in common, so a change to discount rules risks breaking login. It is split into AuthenticationService, EmailService, PricingService, and ReportGenerator, each *functionally* cohesive and each using User (Figure 7.11).

(b) Order is tightly coupled to one payment provider and one database: neither can be replaced, and Order cannot be tested without them. This is *external* coupling to concrete classes, since the order depends on a particular provider and storage technology. Two interfaces are introduced, PaymentProcessor and OrderRepository, and the implementations are passed to the constructor (Figure 7.12). Order now knows only the interfaces and exchanges only an amount or an order with them, which is *data* coupling; PayPal can be replaced by Stripe, or MySQL by PostgreSQL, without changing Order, and tests can use mock implementations.

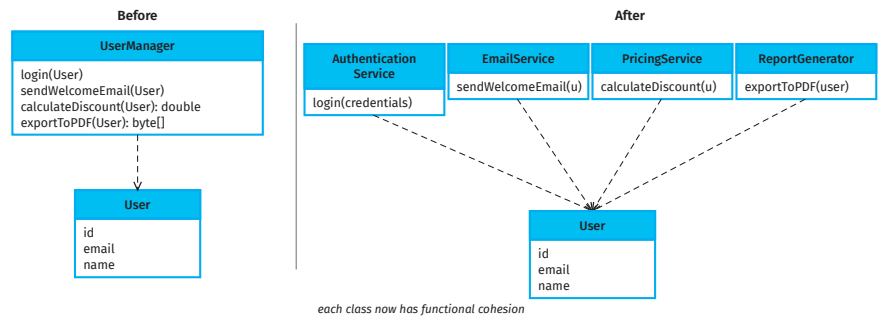


Figure 7.11
Refactoring a
coincidentally
cohesive class

(c) CheckoutManager has *coincidental* cohesion (validation, payment, inventory, and messaging) and *external* coupling to three concrete services. It becomes a CheckoutOrchestrator that performs the four steps in order through OrderValidator, a PaymentGateway interface, an InventoryService (which uses an InventoryRepository interface), and a Notifier interface, each injected (Figure 7.13).

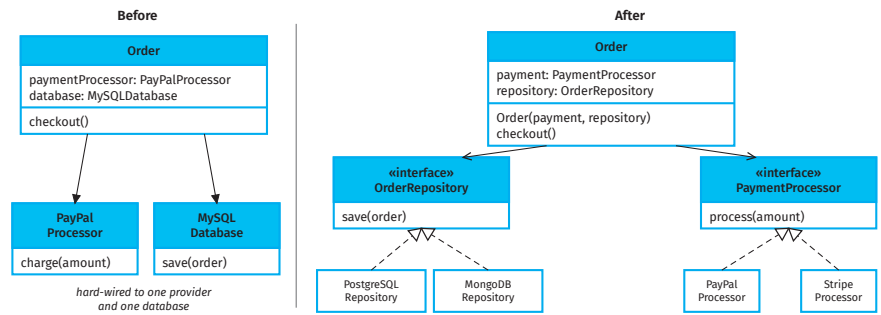


Figure 7.12 Replacing
concrete
dependencies by
interfaces

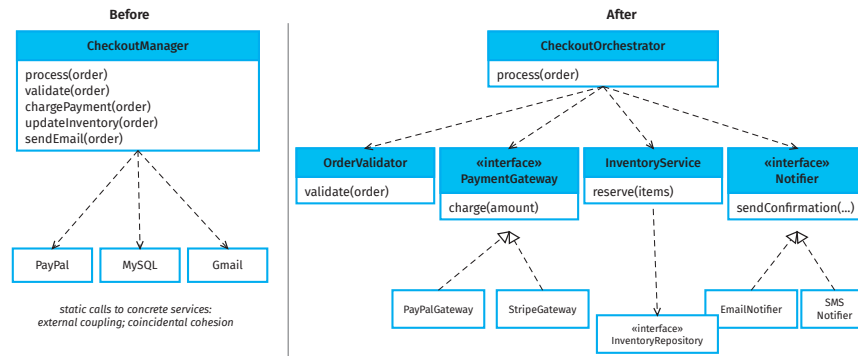


Figure 7.13
Refactoring a
checkout manager
into an orchestrator
of services

Answer (a) Coincidental → functional cohesion, by extracting four classes. (b) External coupling to concrete classes → data coupling through interfaces with dependency injection. (c) Coincidental cohesion and external coupling → an orchestrator whose steps run in a fixed order over the same order (sequential cohesion), data coupled to functionally cohesive services behind interfaces. ■

7.4 User interface design

The user interface (UI) is the part of the software that people see, hear, and touch. However sound the architecture and components, a system whose interface confuses or frustrates its users will be judged a failure. User interface design is the activity of creating an effective communication medium between a human and a computer: identifying interface objects and actions and producing screen layouts and interaction mechanisms that let users perform their tasks efficiently, intuitively, and with satisfaction.

7.4.1 The golden rules

Three golden rules form the basis of interface design principles:

1. *Place the user in control* The interface should respond to the user's needs, so that users feel in charge of the system rather than controlled by it. Interaction modes should not force users into unnecessary actions and should be easy to enter and leave (for example, turning spell-checking on and off); interaction should be flexible (keyboard, mouse, touch, voice); actions should be interruptible and undoable; skilled users should be able to streamline repetitive tasks, for example with macros; technical internals should be hidden from casual users; and users should be able to manipulate on-screen objects directly.
2. *Reduce the user's memory load* The more a user must remember, the more error-prone the interaction. Visual cues let users recognize rather than recall; defaults should be meaningful, with a way to reset them; shortcuts should be intuitive

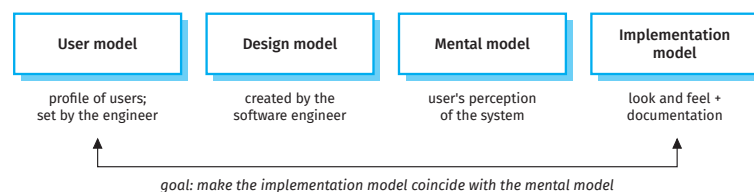
(Alt+P for print); layouts should use real-world metaphors, such as a chequebook for bill payment; and information should be disclosed progressively, overview first and detail on demand.

3. *Make the interface consistent* Consistency lets users transfer what they have learned in one part of the system to every other part. Visual information should be organized by the same rules on all screens; input mechanisms and navigation should be standardized; indicators such as titles, icons, and colors should show users where they are; applications in a family should follow the same conventions; and widely accepted interaction patterns should not be changed without a compelling reason.

7.4.2 Interface design models and users

Four models come into play when a user interface is analyzed and designed (Figure 7.14). The *user model* is a profile of the end users (their demographics, abilities, and goals), established by the software engineer. The *design model* is created by the engineer to fit the user model. The *mental model* is the user's own perception of how the system works. The *implementation model* is the system's look and feel together with its supporting documentation. The goal of interface design is to make the implementation model coincide with the users' mental model, so that the system behaves as its users expect. The key rule is 'know the user, know the tasks': designers must not substitute their own view for the user's.

Figure 7.14 The four models of interface design



Users are classified in the user model as *novices*, who have no syntactic knowledge of the system and little semantic knowledge of the application; *knowledgeable, intermittent users*, who understand the application but have low recall of the interface details; and *knowledgeable, frequent users*, who have good semantic and syntactic knowledge and look for shortcuts and abbreviated modes of interaction. A good interface serves all three: guided workflows and visible menus for novices, consistent layout and recognizable icons for intermittent users, and keyboard shortcuts and macros for frequent users.

7.4.3 The interface design process

The UI design process is iterative and is represented by a spiral with four activities (Figure 7.15):

1. *Interface analysis and modeling* Understand user profiles, skill levels, and environment; define user categories and elicit their requirements; analyze the tasks users perform; and analyze the physical work environment (location, lighting, noise). This corresponds to *user research* and to organizing the content and navigation (*information architecture*).
2. *Interface design* Define interface objects, actions, and their screen representations. Low-fidelity *wireframes* sketch the layout of each screen, and *visual design* applies typography, color, and iconography.
3. *Interface construction* Begin with a *prototype*, an interactive mock-up used to evaluate usage scenarios, then use UI toolkits to complete the interface.
4. *Interface validation* Confirm that the interface supports all user tasks and their variations, and assess ease of use, ease of learning, and user acceptance by *usability testing* with real users.

Several passes through the spiral elaborate the interface incrementally, so not every detail need be specified in the first iteration.

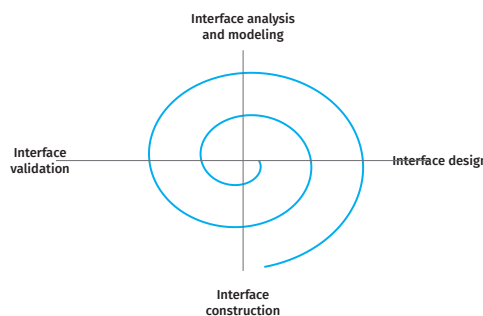


Figure 7.15 The user interface design process

The following issues arise in almost every interface design. *System response time* has two characteristics, its length and its *variability*; consistent response times are preferable to highly variable ones, even if slightly longer. *Help facilities* must decide whether help is available for all functions, how it is accessed, how it is represented and structured, and how the user returns to work. *Error messages* should use language the user understands, give constructive advice for recovery, indicate any negative consequences, be accompanied by a visual or audible cue, and never blame the user: ‘The date must be in DD/MM/YYYY format; your other entries have been kept’ is better than ‘Error 0x80070057’. *Menu and command labels* should be consistent, learnable, and customizable. *Accessibility* must allow people with visual, hearing, or mobility impairments to use the system, and *internationalization* designs a locale-independent core, supported by Unicode, that can be *localized* for particular languages and regions.

7.4.4 Principles and elements of interface design

A good interface is visually apparent and forgiving, hides system internals, and minimizes the input the user must supply. The following principles make these characteristics concrete:

1. *Clarity* The interface is easy to understand and navigate; every element has an obvious purpose.
2. *Consistency* Colors, fonts, icons, terms, and controls are used in the same way throughout.
3. *Familiarity* Familiar patterns and conventions, and real-world metaphors, reduce the learning curve.
4. *Efficiency* Users accomplish their tasks quickly and easily; the user's efficiency, not the developer's convenience, is optimized.
5. *Responsiveness and visibility of status* The interface responds quickly and always shows what the system is doing: progress, location, and the result of each action.
6. *Aesthetics* The interface is visually appealing, readable for all age groups, and uncluttered.
7. *Forgiveness* Users can recover easily from errors, with undo and clear paths back; work is saved automatically.
8. *Accessibility* The interface is usable by people with disabilities.

Other principles often listed for web and mobile applications are *anticipation* (offer what the user will need next), *controlled autonomy*, *focus* on the primary task, *Fitts's law* (the time to select a target grows with its distance and shrinks with its size, so frequent targets should be large and close), *latency reduction* (acknowledge and occupy delays), *learnability*, *work-product integrity*, *tracking state*, and *visible navigation*.

Sommerville states six principles of user interface design: *user familiarity* (the interface uses terms and concepts drawn from the experience of the people who will use the system most); *consistency* (comparable operations are activated in the same way); *minimal surprise* (users are never surprised by the behavior of the system); *recoverability* (the interface includes mechanisms, such as confirmation of destructive actions and undo, that allow users to recover from errors); *user guidance* (the interface gives meaningful feedback when errors occur and provides context-sensitive help); and *user diversity* (the interface provides appropriate interaction facilities for different types of users).

UI design is concerned with the look and feel of the product: its aesthetics, layout, typography, colors, and visual elements. *UX design* is concerned with the overall experience: how the user feels while interacting with the product, including usability, accessibility, and functionality. The key elements of a user interface are:

1. *Visual design Typography*: fonts that are readable and suited to the purpose, with a small number of sizes and weights used consistently for headings, body text, and labels. *Color palette*: colors used strategically to guide attention, convey meaning (red for errors, green for success), and maintain consistency, with enough contrast and never as the only carrier of meaning. *Imagery*: icons, illustrations, and images that match the product's tone.
2. *Layout and structure Grid systems* organize content in a balanced structure; *spacing* between elements avoids clutter and improves readability; *visual hierarchy* makes the important elements prominent through size, color, or placement.
3. *Interactivity Buttons and controls* (sliders, toggles, menus) are clear and look clickable; *animations and transitions* give feedback and make changes easy to follow; *micro-interactions*, such as a button changing color when pressed or a heart filling when an item is liked, confirm each action.
4. *Accessibility* The interface is usable by people who rely on screen readers or who are color blind, following the Web Content Accessibility Guidelines (WCAG): text alternatives, keyboard access, and sufficient contrast.
5. *Consistency* One design language across all screens and components avoids confusion and builds trust.
6. *Responsiveness* The interface adapts to desktops, tablets, and phones.

Other elements are forms (input fields for collecting data) and navigation (the system for moving between parts of the interface). A layout is usually designed first as a wireframe that places these elements, and the principles are then checked against it. In practice, UI design proceeds through research (users, their needs and pain points, competitors), wireframing, prototyping, visual design (colors, typography, and imagery in high-fidelity mock-ups), usability testing, iteration, and implementation with the developers, which are the activities of the spiral in Figure 7.15.

Shneiderman's *eight golden rules* are also widely used: strive for consistency (the same button styles on every page); enable frequent users to use shortcuts (keyboard shortcuts); offer informative feedback (loading indicators, success messages); design dialogs to yield closure (progress indicators and a final confirmation); prevent errors (input validation, confirmation dialogs); permit easy reversal of actions (undo and redo); support an internal locus of control (users initiate actions and can customize settings); and reduce short-term memory load (on-screen hints). To these, many designers add keeping interfaces simple and natural, providing clear navigation (breadcrumbs, persistent menus), being forgiving (editable forms, helpful error messages), focusing on accessibility (high contrast, alternative text), and testing and iterating (usability and A/B testing).

Example 7.11 Key aspects of user interface design

User interface design creates the visual layout and interactive elements of a product so that using it is intuitive, efficient, and enjoyable. List four key aspects of UI design, covering its principles, its design process, and its elements.

1. *Principles* Clarity (every element has an obvious purpose), consistency (the same colors, fonts, icons, and controls throughout), familiarity (known patterns and metaphors), efficiency, responsiveness, aesthetics, forgiveness (easy recovery from errors), and accessibility.
2. *Design process* User research, information architecture (organizing content and navigation), wireframing, prototyping, visual design, and usability testing, repeated in iterations (Figure 7.15).
3. *Visual and structural elements* Layout, typography, color, icons, and a visual hierarchy that makes the most important information most prominent.
4. *Interaction elements* Buttons and other controls, forms for input, navigation between parts of the interface, and feedback such as progress indicators and micro-interactions. ■

Example 7.12 A dashboard for a weather sensing system

(a) Design a user interface dashboard for the weather sensing and alert system of Example 6.3, showing its layout structure, visual hierarchy, navigation elements, widget placement, and interaction components. (b) Annotate the areas where user familiarity, consistency, recoverability, user diversity, and user guidance are applied, with a brief justification for each.

Solution (a) The dashboard (Figure 7.16) has a *header bar* with the station name, a live-data indicator and clock, settings, and help. A *navigation panel* on the left gives Dashboard, Alerts (with a count), History, Thresholds, Sensors, and Help. The *content area* follows a visual hierarchy from top to bottom: three reading tiles (temperature, humidity, pressure), each with a large current value and its status; an alert banner, shown only when a threshold is violated, with Acknowledge and Undo buttons; a 24-hour chart with the threshold drawn as a line and toggle buttons for the variable; and, at the bottom, a table of recent alerts and a form for editing a threshold. The most urgent information, the current readings and any alert, is largest and highest.

(b) The annotations A to E in the figure mark where each principle is applied:

- A *User familiarity* The tiles use the units and terms meteorologists know (°C, %, hPa) and the conventions of other dashboards: a warning triangle for alerts and a left-hand navigation menu.
- B *Consistency* Every reading tile has the same structure, red always means a violated threshold, and all buttons look and behave alike, so learning one part

teaches the rest.

- **C Recoverability** An acknowledged alert can be undone, and a threshold change takes effect only when it is saved, with the default shown so that it can be restored.
- **D User diversity** Status is shown by text as well as color, for color-blind users; the chart can be switched between variables for analysts, while casual users read the tiles; and alerts reach users by SMS or e-mail as they prefer.
- **E User guidance** The threshold form states the valid range and default, the alert banner explains what was exceeded and for how long, and Help is always available in the navigation and header.

Answer A dashboard with a header, left navigation, reading tiles, an alert banner, a trend chart, an alert table, and a threshold form, arranged by urgency, with the five principles applied as annotated in Figure 7.16. ■

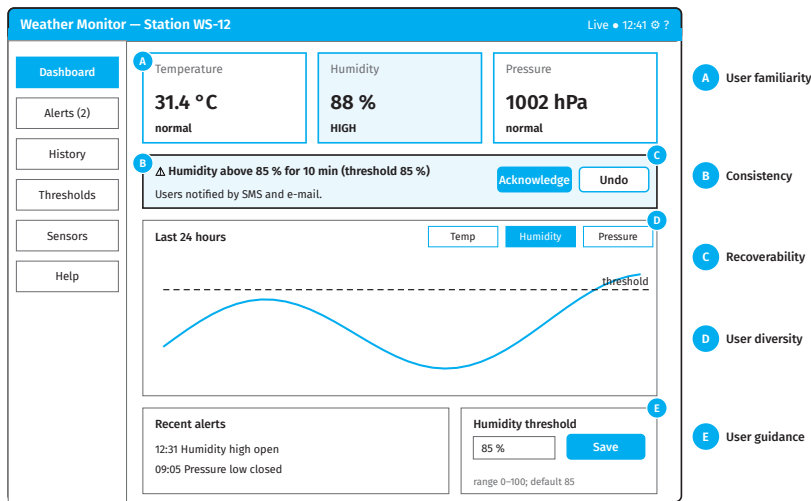


Figure 7.16 Annotated dashboard for the weather sensing and alert system

Example 7.13 Screen layouts for an order processing and delivery system

For a food-delivery company's order processing and delivery management system: (a) design user interface layouts for (i) the restaurant order management screen, (ii) the delivery partner assignment screen, and (iii) the real-time order tracking screen; (b) show in a table how key user interface design principles are applied and satisfied in these screens.

Solution Each screen is designed for the user who will use it and the task they perform most often.

(i) *Restaurant order management* (Figure 7.17) is used by busy kitchen staff on a tablet. Tabs separate New, Preparing, Ready, and Completed orders and show counts. Each new-order card shows only the items, time, and payment, with one-tap Accept and Reject and a preparation-time selector. Selecting an order opens its detail panel with Mark ready and Call rider. An out-of-stock toggle prevents orders that cannot be met, and a sound alert announces new orders.

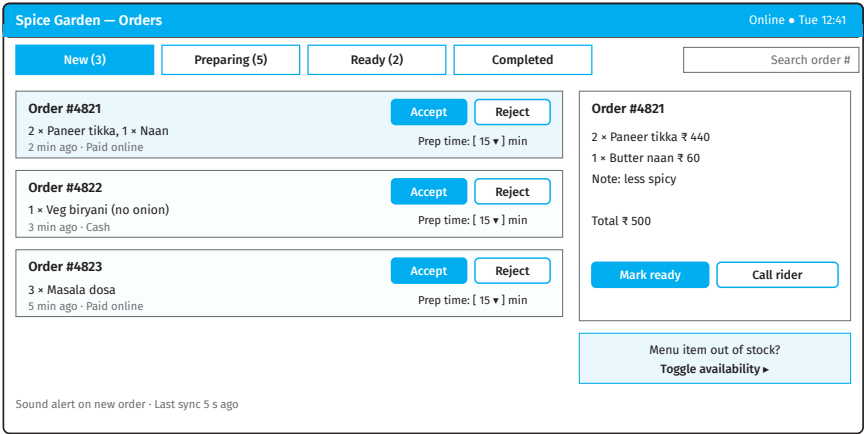


Figure 7.17 Layout of the restaurant order management screen

(ii) *Delivery partner assignment* (Figure 7.18) is used by a dispatcher. A live map shows the restaurant and nearby partners; beside it, suggested partners are ranked nearest first with distance, time, rating, and status. A partner who is still on a delivery can only be queued. Automatic assignment counts down visibly and can be overridden or undone.

(iii) *Real-time order tracking* (Figure 7.19) is used by the customer on a phone. The estimated arrival time is the most prominent element, above a five-step progress bar and a map with the rider's location, updated every ten seconds. The rider's name and rating are shown with a Call button, and delays are explained in plain language.

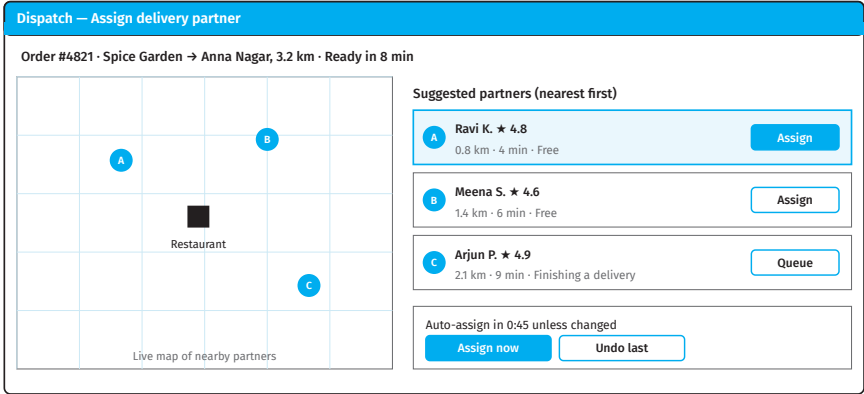


Figure 7.18 Layout of the delivery partner assignment screen

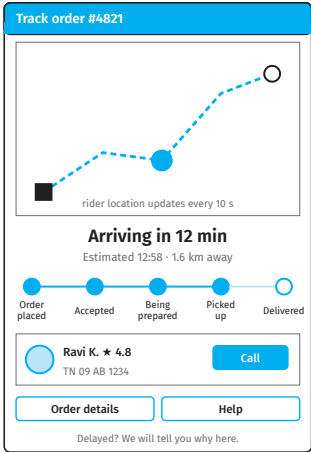


Figure 7.19 Layout of the real-time order tracking screen

Answer (b) The principles and their application are summarized in Figure 7.20. ■

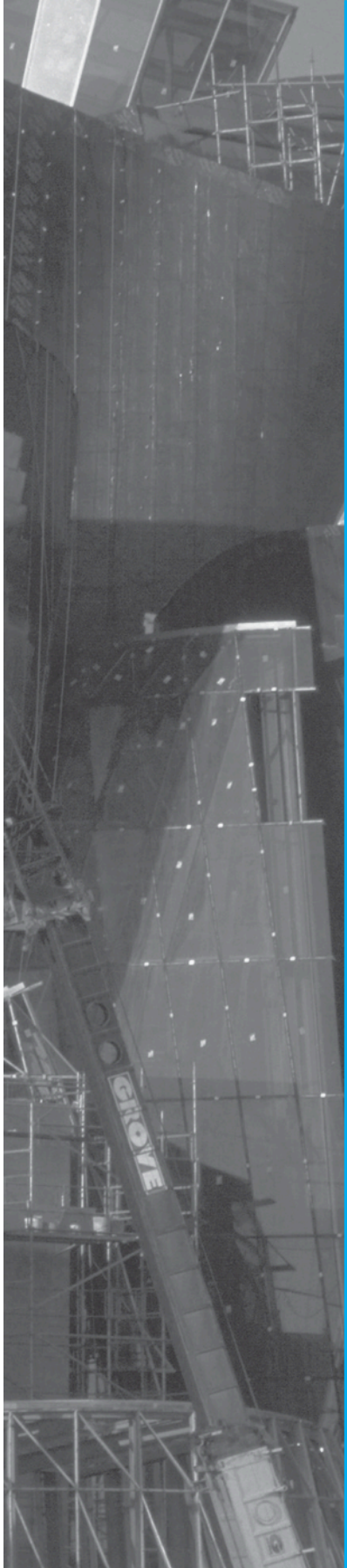
Figure 7.20 User interface design principles and how the three screens satisfy them

Principle	Meaning	How the screens apply it
Consistency	The same layout, colors, terms, and controls are used throughout.	All three screens share the title bar, button styles, primary color, and order number format; Accept, Assign, and Call are always filled primary buttons.
Visibility of system status	Users always know what the system is doing.	Order tabs with counts and “last sync 5 s ago” (restaurant); auto-assign countdown (dispatch); five-step progress bar and ETA (tracking).
Simplicity	Only what is needed for the task is shown.	Each order card shows items, time, and payment only; details appear in a side panel on selection (progressive disclosure).

Principle	Meaning	How the screens apply it
User control and freedom	Users can undo, cancel, and override.	Reject and prep-time selector; manual Assign and Undo last instead of forced auto-assignment; Help and Order details on the tracking screen.
Error prevention and handling	Designs prevent mistakes and explain problems.	Partners who are busy are shown “Queue” not “Assign”; out-of-stock toggle prevents orders that cannot be met; delays are explained on the tracking screen.
Efficiency of use	Frequent tasks need few actions.	One-tap Accept and Mark ready; partners ranked nearest first; sound alert on new orders; Call rider directly from the order.

KEY POINTS

- Design translates the requirements model into data/class, architectural, interface, and component-level designs. Ten principles, from avoiding tunnel vision to reviewing for conceptual errors, guide it.
- Abstraction may be procedural, data, or control abstraction. Separation of concerns divides a problem into independently solvable concerns and is realized by modularity.
- Modularity is judged by decomposability, composability, understandability, continuity, and protection; the total cost of a design is lowest between undermodularity and overmodularity.
- Refinement elaborates a design top-down; refactoring improves internal structure without changing external behavior.
- Cohesion, from lowest to highest, is coincidental, logical, temporal, procedural, communicational, sequential, and functional.
- Coupling, from strongest to weakest, is content, common, external, control, stamp, data, and message coupling. The goal is high cohesion and low coupling, which is functional independence.
- The golden rules of interface design are to place the user in control, reduce the user’s memory load, and make the interface consistent. The user, design, mental, and implementation models should be aligned.
- Sommerville’s interface principles are user familiarity, consistency, minimal surprise, recoverability, user guidance, and user diversity.
- The interface design process is a spiral of analysis and modeling, design, construction, and validation. Response time, help, error handling, labeling, accessibility, and internationalization must always be considered.



8

Software testing

Objectives

The objective of this chapter is to introduce software testing and the techniques used to design test cases. When you have read the chapter, you will:

- understand the test process, test plans, test cases, and the V model of verification and validation;
- understand reviews, Fagan inspections, and audits;
- understand component, integration, and system testing, and the use of stubs and drivers;
- be able to derive white-box and black-box test cases.

Contents

- 8.1 The testing process
- 8.2 Verification and validation: the V model
- 8.3 Reviews, inspections, and audits
- 8.4 Component testing
- 8.5 Integration and system testing
- 8.6 White-box testing
- 8.7 Basis path testing
- 8.8 Black-box testing
- 8.9 Regression and mutation testing
- 8.10 Testing object-oriented and web systems

Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use. Software testing is the process of evaluating a software application to ensure that it meets its specified requirements. It involves executing a program with the intent of finding errors, verifying its functionality, and ensuring that it performs as expected. The phrase *with the intent of finding errors* is essential: a good test case is one with a high probability of finding an as-yet-undiscovered error, and a test that finds nothing is not thereby a success. Testing can show the presence of errors, but not their absence.

Testing has five objectives: to detect defects before users do; to validate that the software does what its requirements specify; to improve quality (reliability, performance, and usability); to ensure user satisfaction; and to show compliance with the standards and regulations of the application domain.

It is useful to distinguish three terms. An *error* (mistake) is a human action that produces an incorrect result; it leads to a *fault* (defect, bug) in the code; executing the fault may cause a *failure*, an observable deviation from the expected behavior. Testing observes failures in order to locate faults; debugging removes them.

Testing proceeds from the small to the large. Development moves inward from system engineering through requirements and design to code; testing moves outward. *Unit testing* verifies individual components as coded; *integration testing* verifies the assembled components against the design; *validation testing* confirms the software against its requirements; and *system testing* verifies the software together with the other elements of the system (hardware, people, and databases). Other testing strategies include requirement-based, functional, performance, security, usability, regression, user acceptance, and maintenance testing.

Testing may be classified in three independent ways: by how it is executed (*manual* or *automated*), by what is tested (*functional* testing of what the system does, or *non-functional* testing of how well it does it), and by what the tester knows (*white-box* testing from the code, or *black-box* testing from the specification). A further distinction is between *static* techniques, which examine work products without executing them (reviews, walkthroughs, inspections, and static analysis), and *dynamic* techniques, which execute the software.

8.1 The testing process

Testing is the process of verifying that a system works as intended and validating that it meets its users' actual needs. It is an investigative process that gives stakeholders information about the quality of the product, and its primary goal is to deliver a reliable and secure product by uncovering defects before the software reaches its users. The fundamental test process has five groups of activities (Figure 8.1):

1. *Test planning and control* defines the approach and monitors progress: objectives, scope, strategy, resources, and entry and exit criteria are set, and metrics are monitored so that corrective action can be taken.

2. *Test analysis* identifies *what* to test: the test basis (requirements, design, code) is analyzed to identify testable features and to derive *test conditions*.
3. *Test design* defines *how* to test: test conditions are elaborated into test cases with inputs, expected results, and preconditions.
4. *Test implementation and execution* prepares the environment and runs the tests: testware and the test environment are built, test cases are prioritized and organized into suites, the tests are executed, and the results are logged.
5. *Test completion* closes testing: the exit criteria are evaluated, testware is archived, deliverables are handed over, and lessons learned are recorded.

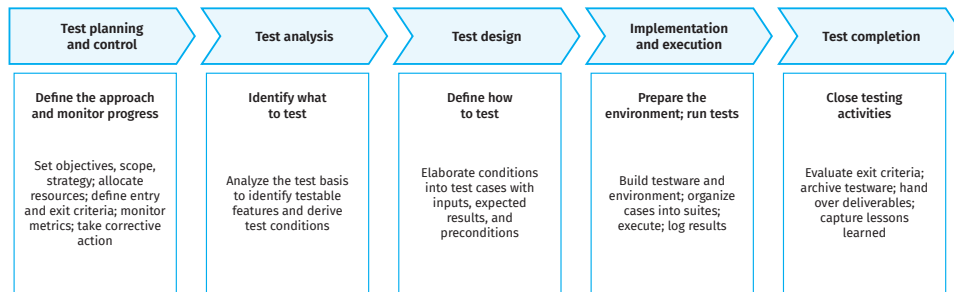


Figure 8.1 The fundamental test process

8.1.1 Test strategy and test plan

A *test strategy* is a description of how testing is to be performed to reach the test objectives under given circumstances. It provides the high-level framework for testing across an organization or program, and it is documented and implemented through *test plans*. A test plan is ‘a document describing the scope, approach, resources, and schedule of intended test activities’; it is the primary output of test planning. Its components are:

1. *Test objectives* The quality aspects that will be verified, aligned with business goals and the risk analysis.
2. *Test scope* The features in scope (to be tested) and out of scope (excluded), justified by risk assessment.
3. *Test strategy and approach* The test levels to be performed (component, integration, system, acceptance); the test types (functional, non-functional, structural); the test techniques (black-box, white-box, experience-based); the *entry criteria*, conditions that must be met before testing begins; and the *exit criteria*, measurable conditions that determine when testing is complete.

4. *Resource planning* The skills of the people needed, the specification of the test environment, and the tools.
5. *Schedule and milestones* A timeline aligned with the development milestones, with the dependencies identified.
6. *Risk management approach* How product risks (defects in the software) and project risks (constraints on testing, such as a late build) will be addressed.
7. *Metrics and reporting* What will be measured (defect density, test coverage, progress) and how often it is reported.
8. *Test deliverables* The documents and artifacts to be produced: test cases, test reports, and defect logs.

8.1.2 Test design and test cases

Test design is the process of defining the scenarios and conditions required to verify that a system meets its requirements. It is the bridge between the high-level test plan and the detailed test cases. Its objective is maximum test coverage with minimum effort; its activities are analyzing the requirements, identifying *test scenarios* (the high-level ‘what to test’), and selecting techniques. The techniques are *black-box* (without knowledge of the code: equivalence partitioning and boundary value analysis, Section 8.8), *white-box* (from the internal structure of the code: statement, branch, and path coverage, Sections 8.6 and 8.7), and *experience-based* (intuition and past knowledge: error guessing and exploratory testing).

A *test case* is a detailed, step-by-step specification of the inputs, execution conditions, and expected outcomes for one test scenario; it is the most granular level of test documentation. Its components are shown in Figure 8.2: a unique identifier, the preconditions that must be true before the test starts, the steps, the test data, the expected result, and the actual result and status, which are filled in during execution.

Figure 8.2 A test case and its components

Component	Content for TC-LOGIN-01
Test case ID	TC-LOGIN-01 (unique identifier)
Objective	Verify that a registered user can log in with valid credentials
Preconditions	User testuser1 exists and is active; the user is logged out
Test steps	1. Open the login page. 2. Enter username and password. 3. Press Log in.
Test data	Username: testuser1; Password: Pass@1234
Expected result	The dashboard is shown with "Welcome, testuser1" within 2 seconds
Actual result and status	Recorded during execution; Pass or Fail (with a defect ID if it fails)

Example 8.1 A test plan for a hospital appointment and billing system

A hospital is building a patient appointment and billing system: patients book, reschedule, and cancel appointments; reception staff register patients; the system generates bills and accepts payment through a payment gateway; and doctors view their schedules. Prepare the main components of a test plan and write one complete test case.

Solution *Objectives* Verify that appointments are never double-booked, that bills are computed correctly, that payments are recorded exactly once, and that patient data is accessible only to authorized staff.

Scope In scope: registration, appointment booking, rescheduling and cancellation, billing, payment, and doctor schedules. Out of scope: the payment gateway's internal processing (a third-party service) and the hospital's existing laboratory system, which is unchanged.

Approach Levels: component tests by developers, component and system integration tests (with the payment gateway virtualized), system tests, and user acceptance tests with reception staff. Types: functional; performance (200 concurrent bookings); security (role-based access). Techniques: equivalence partitioning and boundary values for dates and amounts, a state-transition model of an appointment (booked, rescheduled, cancelled, completed), and branch coverage of the billing rules. *Entry criteria*: the build is deployed to the test environment and smoke tests pass. *Exit criteria*: all planned tests executed; no open critical or high-severity defects; branch coverage of billing at least 90%.

Resources Two testers, one automation engineer, a test environment with anonymized patient data, and Selenium, JUnit, and a load-testing tool. *Schedule* System testing in weeks 9 and 10, acceptance testing in week 11, aligned with the development milestones. *Risks* Late delivery of the billing module (mitigated by testing booking first); unavailability of the gateway sandbox (mitigated by service virtualization). *Metrics and reporting* Tests executed and passed, defects by severity, defect density, and coverage, reported daily during system testing. *Deliverables* The test plan, test cases, defect log, and test summary report.

Answer Test case TC-APPT-03. *Objective*: a slot that is already booked cannot be booked again. *Preconditions*: Dr Rao's 10:00 slot on 5 May is booked by patient P1; receptionist R is logged in. *Steps*: 1. Search Dr Rao's slots for 5 May. 2. Select 10:00. 3. Try to book it for patient P2. *Test data*: doctor Rao, date 5 May, time 10:00, patient P2. *Expected result*: the 10:00 slot is shown as unavailable and the booking is refused with the message 'Slot already booked'; P1's booking is unchanged. *Actual result and status*: recorded during execution. ■

8.2 Verification and validation: the V model

Verification asks 'Are we building the product right?': does each work product conform to its specification? *Validation* asks 'Are we building the right product?':

does the software meet the needs of its users? The V&V model, usually called the *V model*, arranges development and testing so that every development activity has a corresponding testing activity (Figure 8.3). The left side of the V descends from requirements to code; the right side ascends from unit testing to acceptance testing; the tests at each level are *designed* when the corresponding development work product is produced, but *executed* after coding, on the way up.

The levels correspond as follows:

1. *Requirements analysis* ↔ *acceptance testing* The user requirements are the basis of the acceptance tests, which are designed as soon as the requirements are agreed and executed by or with the users on the finished system.
2. *System design* ↔ *system testing* The system requirements and system design are the basis of the system tests, which check the complete, integrated system, functional and non-functional, against its specification.
3. *Architecture design* ↔ *integration testing* The architecture (the components and their interfaces) is the basis of the integration tests, which check that the components work together.
4. *Module design* ↔ *unit testing* The detailed design of each module is the basis of its unit tests, which check each component in isolation.
5. *Coding* sits at the bottom of the V; it is verified by code reviews, inspections, and static analysis before unit testing begins.

The left side is the *verification phase*. Its activities are static: each work product is checked by reviews, walkthroughs, inspections, and audits (Section 8.3) before the next one is built on it. The right side is the *validation phase*. Its activities are dynamic: the software is executed at five levels, component (unit) testing, component integration testing, system integration testing, system testing, and acceptance testing (Figure 8.4), described in Sections 8.4 and 8.5.

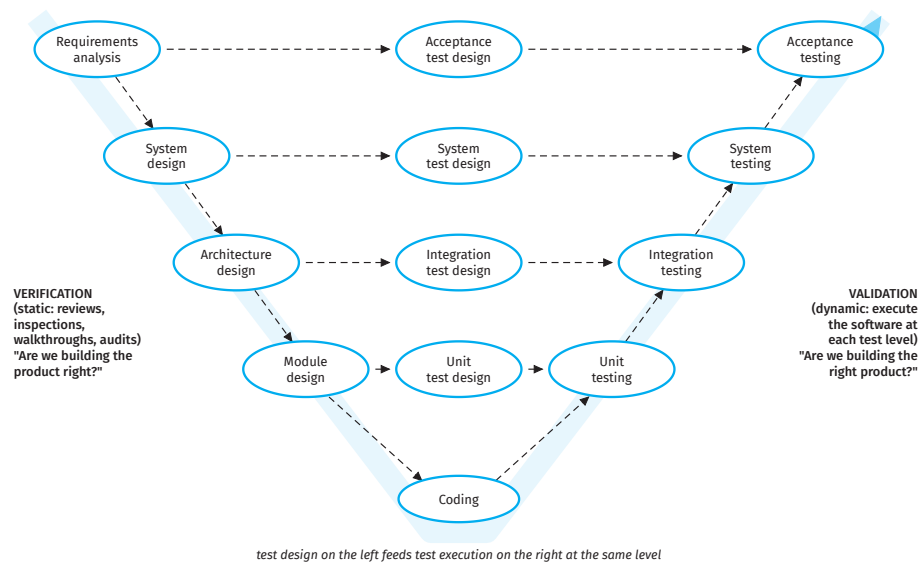


Figure 8.3 The V&V model (V model)

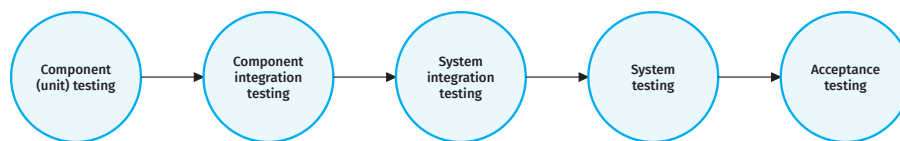


Figure 8.4 Levels of testing in the validation phase

The V model has important advantages. Testing is planned from the start, and test design begins with the requirements, so ambiguous or untestable requirements are found early, when they are cheapest to correct. Every requirement and design element has a matching test level, which gives clear traceability and clear responsibilities. Each phase has defined deliverables and review points, which makes progress easy to monitor. Its disadvantages are those of the waterfall model on which it is based: it is rigid; requirements must be stable, because a change late in the process means revising the work products and test designs of several levels; no working software is available until late; and it is poorly suited to projects whose requirements are uncertain. The V model is therefore used for systems whose requirements are well understood and fixed, and above all for safety-critical and regulated systems (medical, avionics, automotive, banking), in which verification at every stage and traceable evidence of testing are required.

Example 8.2 Applying the V model to an insulin dose system

A hospital commissions software that calculates and records insulin doses. A main controller reads the patient's data from the patient database, CalculateDose computes the dose from the patient's weight and blood glucose, and the dose is displayed to a nurse for confirmation. The software must be certified by a medical regulator. (a) Justify the choice of the V model. (b) For each level of the V, state the work product on the left, the tests designed from it, and one example test executed on the right.

Solution (a) The requirements are fixed by clinical protocol and regulation, a wrong dose is dangerous, and the regulator requires documented evidence that every requirement has been verified and tested. The V model provides exactly this: every development work product is reviewed (verification), and every one has a test level designed from it (validation), with traceability between them. Its rigidity matters little, because the requirements are stable.

(b) The four levels and coding are:

1. *Requirements analysis* → *acceptance test design* → *acceptance testing* User requirement: 'the nurse must confirm every dose before it is recorded'. Acceptance test, executed with nurses on the ward system: a calculated dose is never recorded without an explicit confirmation.
2. *System design* → *system test design* → *system testing* System requirement: 'a dose is displayed within 2 seconds; doses above 20 units require a second nurse'. System tests check the time limit under load and the second-nurse rule for 20 and 21 units (boundary values).
3. *Architecture design* → *integration test design* → *integration testing* Architecture: controller, CalculateDose, and the patient database, with the interface getPatientData() returning age and weight. Integration tests check that the units of weight passed across the interface are kilograms and that a missing record is reported, not treated as zero.
4. *Module design* → *unit test design* → *unit testing* Module design of CalculateDose: 0.5 units per kilogram, reduced by 10% for patients over 70. Unit tests for weight 70 kg at ages 70 and 71 (35.0 and 31.5 units), with a stub replacing the database (Section 8.4).
5. *Coding* The code is inspected against a checklist (Section 8.3) and statically analyzed before unit testing.

Answer The V model suits the system because its requirements are stable and safety-critical and the regulator requires traceable verification. Each left-side work product (user requirements, system design, architecture, module design) is reviewed when produced and has its tests designed at once; the tests are executed on the right at the matching level (acceptance, system, integration, unit). ■

8.3 Reviews, inspections, and audits

Software inspections and reviews analyze and check the system requirements, design models, program source code, and even proposed system tests. These are static verification techniques: you do not need to execute the software to verify it. Reviews and inspections are complementary to testing and have three advantages over it:

1. During testing, errors can mask (hide) other errors. When an error leads to unexpected outputs, you can never be sure whether later output anomalies are due to a new error or are side effects of the original error. Inspection is a static process, so errors do not interact, and a single inspection session can discover many errors.
2. Incomplete versions of a system can be inspected without additional costs. To test an incomplete program you have to develop specialized test harnesses.
3. As well as searching for program defects, an inspection can consider broader quality attributes of a program, such as compliance with standards, portability, and maintainability. It can look for inefficiencies, inappropriate algorithms, and poor programming style that make the system difficult to maintain and update.

Inspections cannot, however, check non-functional characteristics such as performance and usability, or show that the software behaves correctly when running with real data. Both inspection and testing are therefore needed.

The review process has three phases. In *pre-review activities*, the review is planned, a team is chosen, and each reviewer prepares individually, reading the work product and the relevant standards. The *review meeting* is short (two hours at most); the author ‘walks through’ the document with the team, and a chair records the problems found. In *post-review activities*, the problems raised are addressed: defects are corrected, the software may be refactored, and follow-up checks confirm that every comment has been dealt with.

An *audit* is an independent, objective examination of a work product or process, carried out by people external to the development team (often an internal audit department or a third-party organization). Instead of looking for technical defects, auditors check for compliance with standards (such as ISO 9001 or CMMI), legal regulations, and internal policies. The scope is broad: management records, security protocols, and licensing agreements are examined as well as the code. The outcome is a formal *audit report* that lists the non-compliances and recommends corrective actions. Where a review asks whether the product is correct, an audit also asks whether it was produced properly: whether it complies with legal and licensing requirements, is traceable to its requirements, is documented well enough to be maintained, and respects ethical constraints such as privacy and fairness.

A *review* is the broadest category: a systematic meeting or process in which a work product (requirements, design, or code) is examined by project personnel, managers, or users to find defects and to provide feedback. *Peer reviews* are

conducted by colleagues, as in code reviews and walkthroughs; *management reviews* are conducted by managers to monitor progress and make decisions. *Formal* reviews follow a strict agenda and a documented process; *informal* reviews, such as a ‘buddy check’, are less structured.

An *inspection* is the most formal and rigorous type of peer review. Defined by Michael Fagan at IBM, it follows six steps (Figure 8.5): *planning*, in which the moderator checks that the work product meets the entry criteria, chooses the team, and schedules the meeting; *overview*, in which the author explains the work product to the team; *preparation*, in which each inspector studies it individually against the checklist; the *inspection meeting*, in which the reader paraphrases the work product and the defects found are logged; *rework*, in which the author corrects every logged defect; and *follow-up*, in which the moderator verifies the corrections and checks the exit criteria. The roles are the *moderator*, who leads the meeting and ensures that the process is followed; the *reader*, who paraphrases the document for the group (the author may take this role in lighter-weight reviews); the *recorder* or scribe, who documents every defect; the *author*; and the *inspectors*. What distinguishes an inspection from other reviews is that it is data-driven: checklists, entry and exit criteria, and defect measurements are used to improve the development process itself, not just the code.

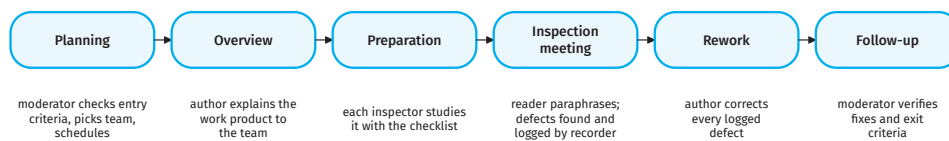


Figure 8.5 The six steps of a Fagan inspection

Program inspections are peer reviews in which team members collaborate to find bugs in the program being developed. The inspection process is driven by a *checklist* of common programming errors (Figure 8.6). The checklist should be established by discussion with experienced staff and regularly updated as more experience is gained, and it varies from one programming language to another. Different organizations may develop their own checklists based on local standards and practices.

Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size - 1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned?

Figure 8.6 An inspection checklist

Example 8.3 Manual inspections when code is generated by tools

A financial company that used to hold structured peer reviews now generates much of its code with AI tools. The generated code lacks comments, readability, and clear structure, although it works, and the company proposes to cancel all manual inspections and rely only on automated testing. (a) Give two benefits of manual inspections and four kinds of error that inspections find. (b) State the advantages and disadvantages of this decision in terms of reliability, maintainability, and long-term risk. (c) Recommend the most suitable tool for keeping inspections effective.

Solution (a) *Benefits.* Inspections improve code quality, because people can detect design flaws, poor readability, and logic that is technically correct but inappropriate, which automated tests do not look for. They also spread understanding of the system: the team learns how the code works, which reduces the risk that only its author (or no one, for generated code) understands it. *Errors found by inspection* include logical design errors; violations of coding standards; poor readability and incorrect or misleading variable names; missing or incorrect comments and incomplete documentation; and, from the checklist, data faults such as uninitialized variables and interface faults such as parameters in the wrong order.

(b) Removing inspections has advantages: development is faster, review time is saved, and automated checks run quickly and consistently on every change. Its disadvantages are more serious. Logical errors that tests do not happen to exercise

remain unnoticed; readability and structure degrade, so maintainability falls; technical debt accumulates, which raises the long-term risk; and human insight into whether the code is the right solution is lost. Generated code is particularly likely to be plausible but subtly wrong, so the loss matters more, not less.

Answer (c) A *structured inspection checklist* is the most suitable tool. It makes reviewers check systematically for coding-standard compliance, documentation, design consistency, and the classes of fault in Figure 8.6, so that quality is maintained even for generated code and even when inspections are made shorter and lighter-weight rather than removed. ■

Example 8.4 Audits for AI-generated code

Teams now generate much of their code with AI-assisted tools, yet structured audits are still used for verification. (a) List the benefits of audits for compliance gaps, ethical concerns, and maintainability, with an example use case for each. (b) A cybersecurity company develops mostly AI-generated code that is functional but poorly documented and difficult to interpret. Management cancels all audits and relies solely on test design. Propose a test design. (c) Give two reasons why testing cannot fully replace audits.

Solution (a) *Compliance gaps.* Audits check the code against regulations, security standards, and licences, which a working program may still violate. Use case: generated code that logs full card numbers breaches PCI DSS; generated code copied from a GPL-licensed source creates a licensing obligation.

Ethical concerns. Audits check how data is collected and used and whether decisions are fair and explainable. Use case: a generated loan-scoring function uses postcode as an input, which acts as a proxy for a protected attribute; generated analytics code keeps personal data longer than the privacy policy allows.

Maintainability. Audits check readability, structure, documentation, and technical debt, which no functional test measures. Use case: generated code duplicates the same validation logic in ten places with no comments, so a later change to the rule is missed in some of them.

(b) Because the code is hard to read, the test design must be driven by the requirements and threats rather than by the code:

1. *Requirements-based black-box tests* with equivalence partitioning and boundary value analysis for every input.
2. *White-box coverage* measured by a tool, with a minimum of branch coverage and basis paths for the critical functions, so that untested generated logic is visible.
3. *Security tests:* static analysis (SAST), dependency and secret scanning, fuzzing of every input parser, and penetration tests against the OWASP Top 10.
4. *Negative and abuse-case tests:* malformed, oversized, and malicious inputs, and authentication and authorization bypass attempts.

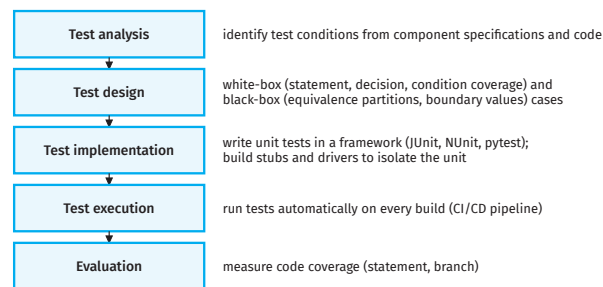
5. *Regression and mutation tests*: an automated regression suite run on every change, with mutation testing to show that the tests actually detect faults.

Answer (c) Testing cannot replace audits because (1) testing shows only that the software behaves correctly for the cases tested, while an audit examines whether it *should* behave that way and whether it meets regulations, licences, and ethical standards, which no test oracle states; and (2) testing does not assess the qualities that audits examine without executing the code, such as documentation, readability, design structure, and traceability, so poorly understood generated code would pass its tests and still be unmaintainable and insecure. ■

8.4 Component testing

Component testing (also called unit, module, or program testing) is the testing of individual software components in isolation, to detect defects and to verify the functioning of modules, classes, or functions before they are integrated. Its *test basis* is the program specifications (low-level design documents), the component design specifications, the source code itself (for white-box testing), the component requirements derived from the system requirements, and the interface specifications of the component. The process follows the general test process (Figure 8.7): test conditions are identified from the specifications and code; test cases are designed with white-box techniques (statement, decision, and condition coverage) and black-box techniques (equivalence partitions and boundary values on the component's inputs); the tests are implemented in a framework such as JUnit, NUnit, or pytest; they are executed automatically on every build in a continuous integration (CI/CD) pipeline; and the results are evaluated with code-coverage measures.

Figure 8.7 The component testing process



A component seldom works alone: it calls other components and is called by others. To test it in isolation, the missing neighbors are replaced by *test doubles* (Figure 8.8, Figure 8.9):

1. A *stub* replaces a *called* (lower-level) component that is not yet developed or must be excluded. It returns predefined responses, so the tester can check that

the component under test handles them correctly. Stubs are used in top-down integration.

2. A *driver* replaces a *calling* (higher-level) component. It invokes the component under test with test data and checks its outputs. Drivers are used in bottom-up integration.
3. A *test harness* is the whole test environment of stubs, drivers, and other utilities that executes the tests and records the results.

The stub and the driver are compared in Figure 8.9. A stub answers the question ‘does my component handle the responses it receives correctly?’; a driver answers the question ‘does my component produce the correct outputs for its inputs?’.

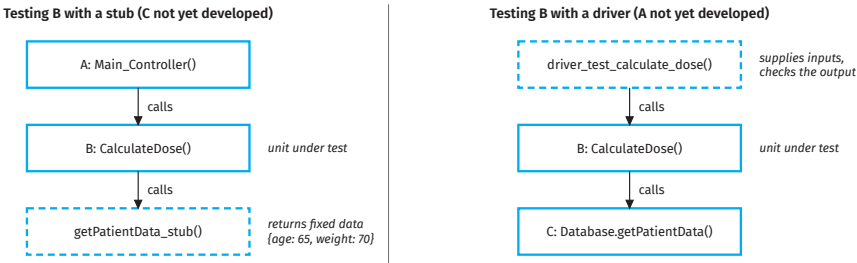


Figure 8.8 A stub replaces a called module; a driver replaces a calling module

Figure 8.9 Stubs, drivers, and test harnesses

Aspect	Stub	Driver
Replaces	A called (lower-level) component	A calling (higher-level) component
Does	Returns predefined responses to the unit under test	Invokes the unit under test with test data and checks its results
Direction	Downward dependency	Upward dependency
Test focus	Does the unit handle the responses correctly?	Does the unit produce the correct outputs?
Position	Injected into the unit under test	Wraps around the unit under test
Integration strategy	Top-down	Bottom-up
Test harness	The environment of stubs, drivers, and utilities that executes the tests; used in both strategies	

Example 8.5 A stub and a driver for a dose calculation

In the insulin dose system of Example 8.2, Main_Controller (module A) calls CalculateDose (module B), which calls Database.getPatientData (module C). The dose is 0.5 units per kilogram of body weight. Write (a) a stub that allows B to be tested before C exists, and (b) a driver that allows B to be tested before A exists.

Solution (a) The stub takes the place of the database and returns fixed test data; the test calls B, which calls the stub.

```
def getPatientData_stub():                # replaces module C
    return {"age": 65, "weight": 70}      # fixed test data

def test_calculate_dose():
    patient = getPatientData_stub()      # stub called here
    dose = calculate_dose(patient["age"], patient["weight"])
    assert dose == 35                    # 0.5 x 70
```

(b) The driver takes the place of the main controller: it supplies the inputs directly, calls B, and checks the output.

```
def driver_test_calculate_dose():          # replaces module A
    dose = calculate_dose(age=65, weight=70)
    print(f"Driver result: {dose} units")
    assert dose == 35

driver_test_calculate_dose()
```

Answer The stub sits *below* the unit and supplies the data the unit asks for; the driver sits *above* the unit and supplies the call and its arguments. Both form part of the test harness, and both are discarded when the real modules A and C are integrated. ■

8.5 Integration and system testing

Integration testing verifies the interaction between modules. Its goal is to detect interface defects, data-flow errors, and interaction failures that component tests, which test each part alone, cannot reveal. There are two levels. *Component integration testing* tests the interactions between the components of one system; *system integration testing* tests the interactions between whole systems or subsystems. The inputs to integration testing are the interface specifications (API contracts and message formats), the architectural design showing how components are related, the database schema definitions, the communication protocols (REST, SOAP, message queues), and the system and subsystem design specifications.

8.5.1 Integration strategies

In *big bang* integration, all components are combined at once and then tested. It needs no stubs or drivers, but when a test fails the fault may be anywhere, so it is practical only for very small systems. *Incremental* integration adds modules step by step, so that each failure can be traced to the module just added. There are three incremental approaches (Figure 8.10, Figure 8.11):

1. *Top-down* integration starts with the top-level (main) module and integrates downward, replacing lower modules that are not ready by stubs. Modules may be added *depth-first* (one complete branch at a time) or *breadth-first* (one level at a time). Its advantage is that a skeleton of the whole system, with its major control and decision logic, is working early; its disadvantage is that many stubs are needed and lower-level processing is tested late.
2. *Bottom-up* integration starts with the lowest-level modules and integrates upward, replacing higher modules that are not ready by drivers. Low-level modules are often combined into *clusters* (builds) that perform a sub-function, each tested with a driver. Its advantage is that the foundation services are tested thoroughly and early, and no stubs are needed; its disadvantage is that the program does not exist as a whole until the last module is added.
3. *Sandwich* (hybrid) integration combines the two: top-down for the upper levels and bottom-up for the lower levels, meeting at a middle layer. It uses both stubs and drivers and suits medium and large systems.

With the modules of Example 8.5, top-down integration first tests A with a stub for B, then replaces the stub by B and tests A with B, using a stub for C, and finally adds C. Bottom-up integration first tests C with a driver, then adds B and tests B with C through a driver that replaces A, and finally adds A.

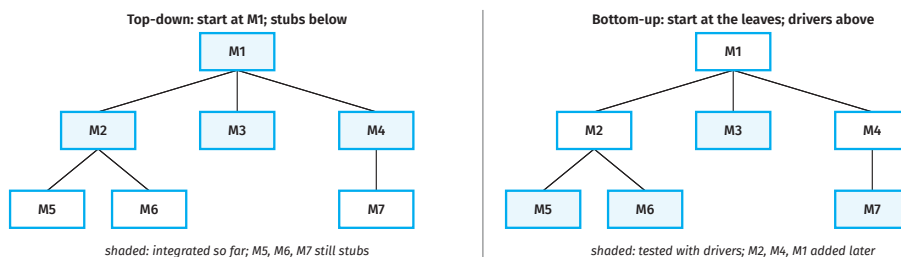


Figure 8.10 Top-down and bottom-up integration of a module hierarchy

Figure 8.11
Integration testing
strategies

Strategy	How it works	Stubs and drivers	Best for
Big bang	All components are integrated at once, then tested	None	Very small systems (fewer than about five components)
Top-down	Start with the top-level component; integrate downward	Stubs for missing lower components	Systems whose major functions are in the upper layers
Bottom-up	Start with the lowest-level components; integrate upward	Drivers for missing higher components	Systems that need stable foundation services first
Sandwich (hybrid)	Top-down and bottom-up together, meeting at a middle layer	Both stubs and drivers	Medium and large systems

Example 8.6 Integration orders, stubs, and drivers

A program has the module hierarchy of Figure 8.10: M1 calls M2, M3, and M4; M2 calls M5 and M6; M4 calls M7. Give the order of integration for top-down (depth-first and breadth-first) and bottom-up integration, and state the stubs and drivers needed. How would sandwich integration proceed?

Solution *Top-down, depth-first:* M1, M2, M5, M6, M3, M4, M7. *Top-down, breadth-first:* M1, M2, M3, M4, M5, M6, M7. In either order each module except M1 is represented by a stub until it is integrated, so six stubs are needed (for M2 to M7). Testing M1 first requires stubs for M2, M3, and M4; when M2 replaces its stub, stubs for M5 and M6 are needed; and so on.

Bottom-up: the leaves are tested first: M5 and M6 as a cluster with a driver standing in for M2, M7 with a driver for M4, and M3 with a driver for M1. Then M2 (with M5 and M6) and M4 (with M7) replace their drivers and are tested with a driver for M1. Finally M1 is added. The drivers replace the calling modules M1, M2, and M4, so three drivers are needed, and no stubs.

Sandwich: the middle layer M2, M3, M4 is chosen as the target. M1 is tested top-down with stubs for the middle layer, while the clusters {M5, M6} and {M7} are tested bottom-up with drivers; the two parts meet when the real M2, M3, and M4 are integrated.

Answer Top-down depth-first M1, M2, M5, M6, M3, M4, M7; breadth-first M1, M2, M3, M4, M5, M6, M7; six stubs. Bottom-up M5, M6, M7, M3, then M2, M4, then M1; three drivers (for M1, M2, and M4). Sandwich: top-down to the middle layer and bottom-up to it at the same time. ■

8.5.2 Interface and data-flow testing

Interface testing checks the correctness of what passes between modules: function parameters, data types, data formats, and return values. *Data-flow testing* checks that

the correct data flows between modules: that each module receives the value the specification intends, not merely a value of the right type.

Example 8.7 Finding interface and data-flow defects

Each pair of modules below passes its own component tests. Find the defect that integration testing reveals in each.

```
# (a) Module A
def get_age():
    return "25"                                # string
# Module B
def calculate_birth_year(age):
    return 2025 - age
# Integration
year = calculate_birth_year(get_age())

# (b)
def get_price():
    return 100
def apply_discount(price):
    return price * 0.9
def final_price():
    price = get_price()
    return apply_discount(50)    # wrong variable used
```

Answer (a) An *interface* defect: A returns the age as a string and B expects a number, so the subtraction fails ('cannot subtract a string from an integer'). Each module is correct by itself; the defect is in the agreement between them, and the interface specification must state the type. (b) A *data-flow* defect: `final_price` passes the constant 50 instead of the price obtained from `get_price`, so it returns 45.0 instead of 90.0. The types are correct, so only a test that checks the value flowing between the modules finds it. ■

8.5.3 System integration testing

System integration testing (SIT) tests the interaction between different systems and subsystems as a whole, including external third-party systems (payment gateways, SMS providers), databases and middleware, hardware devices (scanners, IoT sensors), and legacy systems reached through adapters. It is performed after component integration testing and before system testing, usually by integration testers or the QA team. Its inputs are the system interface specifications, the API documentation (OpenAPI or Swagger), the message formats (JSON or XML schemas), the network protocols (REST, SOAP, MQTT), and the service level agreements (SLAs) of the external systems. Its techniques are listed in Figure 8.12. Two deserve emphasis. In *contract testing*, the provider of a service publishes a contract describing its API, and each consumer is tested against the contract before the two are integrated, so that an incompatible change is found at once. In *service virtualization*, an external system that is unavailable, slow, or costly to call is

simulated; it plays the part that a stub plays in component testing, but for a complex dependency. The differences between the two levels of integration testing are summarized in Figure 8.13.

Figure 8.12
Techniques of system
integration testing

Technique	Purpose	Example
End-to-end flow testing	Validate a complete business process across systems (the primary technique)	Patient registration → lab order → results delivery across three systems
Contract testing	Verify that a consumer and a provider agree on an API (critical for microservices)	The provider publishes a contract (e.g., Pact); the consumer is tested against it before integration
Service virtualization	Simulate an unavailable or costly external system; replaces stubs for complex dependencies	A virtual payment gateway instead of one that charges for every test call
Message/queue testing	Validate asynchronous communication in event-driven systems	An order event published to a queue is consumed correctly by the inventory system
Database integration testing	Check data consistency across systems (often overlooked)	An order created in system A appears correctly in system B's reporting database

Figure 8.13
Component
integration and
system integration
testing

Aspect	Component integration testing	System integration testing
Scope	Components within one system	Multiple independent systems
Boundaries	Internal interfaces (method calls, internal APIs)	External interfaces (REST APIs, message queues, files)
Environment	Developer workstation or CI pipeline	Integration test environment that mirrors production
Dependencies	Other components, replaced by stubs and drivers	External systems, replaced by service virtualization
Defect focus	Data corruption at interface boundaries	Availability, protocol compliance, data consistency
Typical tools	JUnit with Mockito (stubs and mocks)	Postman, Pact, WireMock, Kafka test containers

8.5.4 System testing

System testing is the testing of an integrated system to evaluate its compliance with the specified requirements. It validates the complete system in an environment that mirrors production, testing both what the system does (*functional* testing) and how well it does it (*non-functional* testing) (Figure 8.14). Before it begins, the system requirements specification, the functional requirements (use cases or user stories), the non-functional requirements (performance and security specifications), the interface specifications, and the business rules must exist.

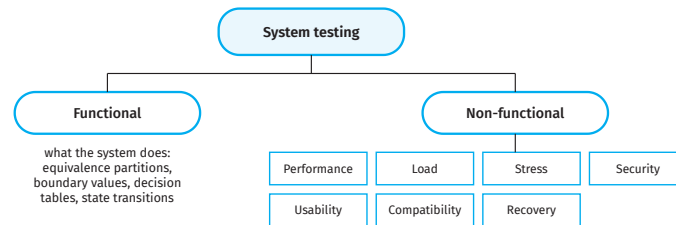


Figure 8.14 Types of system testing

Functional system tests are designed with the black-box techniques of Section 8.8: *equivalence partitioning* for input fields with valid and invalid ranges (a password of 8 to 64 characters: ‘Pass1234’ is valid, ‘short’ is rejected); *boundary value analysis* for numeric and text limits (an age of 18 to 120: test 17, 18, 19, 119, 120, and 121); *decision tables* for complex business rules (premium and standard customers, orders above and below a limit: all four combinations); and *state transitions* for workflows (an order that is CREATED, PAID, SHIPPED, and DELIVERED must not go from PAID to DELIVERED without being SHIPPED). Non-functional system tests include *performance* testing (response times under normal load), *load* testing (behavior at the expected peak), *stress* testing (behavior beyond the peak, and recovery from it), *security*, *usability*, *compatibility* (browsers, devices, and operating systems), and *recovery* testing (the system is made to fail and its recovery is checked). System testing is followed by *acceptance testing*, in which users check the system against their requirements in their own environment: *alpha* testing by users at the developer’s site, *beta* testing by users at their own sites, and formal *user acceptance testing* before the system is accepted.

Example 8.8 Integration and system tests for a hospital information system

A hospital’s patient registration system sends lab orders to a separate laboratory system, which returns results to a results-delivery portal. Registration also charges fees through an external payment gateway that charges for every call. (a) Design the system integration tests. (b) Give two functional and three non-functional system tests.

Solution (a) *End-to-end flow*: register a patient, order a blood test, record the result in the laboratory system, and check that the result appears on the portal for that patient, with the same patient identifier in all three systems. *Contract testing*: the laboratory system publishes the contract of its order API (fields, types, and error codes), and registration is tested against it, so a renamed field is found before deployment. *Service virtualization*: the payment gateway is replaced by a virtual service that returns success, decline, and timeout responses, so that all three are tested without charges. *Message testing*: if results are sent as events, check that a result event is consumed exactly once and in order. *Database integration*: a patient created in registration appears with the same data in the laboratory database.

Answer (b) Functional: a patient cannot be registered twice with the same national ID (equivalence partitions: new ID, existing ID); an appointment moves only through valid states (booked → checked in → completed). Non-functional: *performance*, a result appears on the portal within 5 seconds of being recorded; *load*, 500 concurrent registrations complete without errors; *security*, a patient can see only their own results. ■

8.6 White-box testing

White-box testing (also called clear-box, open-box, or glass-box testing) examines the internal structure, logic, and code of an application. Tests are designed from knowledge of the implementation, and the tester has access to the source code. Its objectives are to ensure that all internal operations work as the design specifies and to identify hidden errors, security vulnerabilities, and inefficiencies. It finds logical errors, broken or badly structured paths, typographical errors, and unreachable or redundant code. It is needed in addition to black-box testing because logic errors tend to hide on paths that are rarely executed, which tests derived from the specification may never reach.

Code coverage measures how much of the code is executed by a test suite. *Statement coverage* is the percentage of executable statements executed at least once; *branch coverage* is the percentage of decision outcomes (branches) taken at least once, so an if statement needs one test for its true branch and one for its false branch; and *path coverage* is the percentage of feasible paths executed. Each is computed as $(\text{number executed} \div \text{total number}) \times 100\%$. Full path coverage implies full branch coverage, which implies full statement coverage, but not the reverse.

Example 8.9 Statement, branch, and path coverage

The function below has six statements (S1–S6) and two decisions with two outcomes each. Find the coverage achieved by the test sets $\{m = 80\}$, $\{m = 30\}$, and $\{m = 80, m = 30\}$.

```
def classify(m):
    result = "Fail"           # S1
    if m >= 40:               # S2 (decision D1)
        result = "Pass"      # S3
    if m >= 75:               # S4 (decision D2)
        result = "Distinction" # S5
    return result             # S6
```

Solution $m = 80$ executes S1–S6 and takes D1-true and D2-true: statement coverage $6/6 = 100\%$, branch coverage $2/4 = 50\%$. $m = 30$ executes S1, S2, S4, S6 and takes both false branches: statement coverage $4/6 = 66.67\%$, branch coverage 50% . Together they execute all six statements and all four branches.

For paths, the four combinations of outcomes are TT, TF, FT, and FF. FT ($m < 40$ and $m \geq 75$) is infeasible, so there are three feasible paths, needing $m = 80$, $m =$

50, and $m = 30$.

Answer {80}: 100% statement, 50% branch. {30}: 66.67% statement, 50% branch. {80, 30}: 100% statement, 100% branch, but path coverage $2/3 = 66.67\%$; adding $m = 50$ gives 100% path coverage. ■

Control-structure testing supplements coverage by concentrating on the conditions inside decisions, where five kinds of error are common: Boolean operator errors (AND written for OR), Boolean variable errors (the wrong variable), parenthesis errors, relational operator errors ($>$ written for $\geq$), and arithmetic expression errors (such as division by zero inside a condition). For a relational expression such as $\text{age} > 18$, testing values just above, equal to, and just below the constant (19, 18, 17) detects most relational operator errors.

8.7 Basis path testing

Basis path testing is a white-box method that identifies a set of *independent paths* through a program's control flow graph and designs a test case for each, so that every independent path, and therefore every statement and every branch, is executed at least once.

8.7.1 Flow graphs

A control flow graph (flow graph) represents the logic of a program. *Nodes* (circles) represent one or more statements; a sequence of statements with no branching can be collapsed into one node. *Edges* (arrows) represent the transfer of control. A *predicate node* contains a condition and has two or more outgoing edges. A *region* is an area bounded by edges and nodes; the area outside the graph counts as one region. An *independent path* is one that introduces at least one new edge not traversed by any previously listed path. Structured constructs have the standard subgraphs of Figure 8.15.

To draw a flow graph, number the executable statements; give each decision (if, loop test, switch) its own node; merge purely sequential statements if desired (this removes one node and one edge together, so it does not change the result); draw a join node where branches meet; draw the back edge of each loop from the end of the body to the loop test; and split a compound condition such as $a \text{ AND } b$ into one predicate node per simple condition.

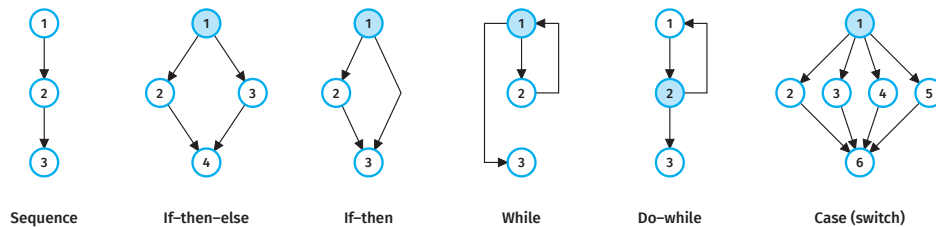


Figure 8.15 Flow-graph notation for structured constructs; shaded nodes are predicate nodes

8.7.2 Cyclomatic complexity

Cyclomatic complexity, $V(G)$, is a software metric that gives the number of linearly independent paths in a program's flow graph. It is therefore the number of test cases needed to execute every basis path, and an upper bound on the number needed to cover every edge. It can be computed in three ways, which must agree:

1. $V(G) = E - N + 2$, where E is the number of edges and N the number of nodes (for a single program; in general $E - N + 2P$ for P connected components).
2. $V(G) = D + 1$, where D is the number of predicate (decision) nodes; a multi-way decision with k outgoing edges counts as $k - 1$.
3. $V(G) = R$, the number of regions of the planar flow graph, including the outer region.

Some texts write the second formula as $V(G) = P + 1$, using P for the number of predicate nodes; this is the same formula, and P here must not be confused with the number of connected components in $E - N + 2P$.

The basis path testing procedure is: draw the flow graph; compute $V(G)$; list $V(G)$ independent paths, each adding at least one new edge; design a test case, with inputs and expected output, for each path; and execute the tests. Modules with $V(G)$ above about 10 are hard to test and are candidates for refactoring.

Example 8.10 Basis paths of a two-way decision

Find the cyclomatic complexity and the basis path tests for: 1 Read A; 2 Read B; 3 IF A > B THEN; 4 Print "A is greater"; 5 ELSE; 6 Print "B is greater"; 7 ENDIF.

Solution The flow graph is Figure 8.16. Its edges are 1→2, 2→3, 3→4, 3→5, 5→6, 4→7, 6→7, so $E = 7$ and $N = 7$.

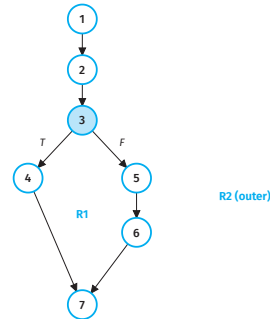


Figure 8.16 Flow graph of the larger-of-two program

$V(G) = E - N + 2 = 7 - 7 + 2 = 2$. Check: one predicate node (3), so $V(G) = 1 + 1 = 2$; two regions, R1 and the outer region.

Answer $V(G) = 2$. Path 1: 1-2-3-4-7 with $A = 10$, $B = 5$, expected 'A is greater'. Path 2: 1-2-3-5-6-7 with $A = 3$, $B = 8$, expected 'B is greater'. Note that $A = B$ also takes path 2 and prints 'B is greater', which is wrong for equal values; a boundary test (Section 8.8) is needed to find this defect. ■

Example 8.11 A nested decision

Find $V(G)$ and the basis path tests for: 1 if ($a < b$) 2 F1(); 3 else if ($a < c$) 4 F2(); 5 else F3(); 6 end.

Solution From Figure 8.17, the edges are 1→2, 1→3, 3→4, 3→5, 2→6, 4→6, 5→6: $E = 7$, $N = 6$, so $V(G) = 7 - 6 + 2 = 3$. There are two predicate nodes (1 and 3), so $D + 1 = 3$, and three regions.

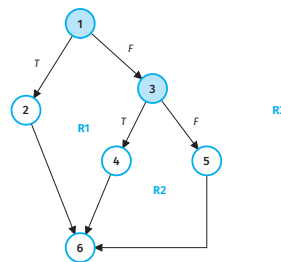


Figure 8.17 Flow graph of a nested if-else

Answer $V(G) = 3$. Path 1-2-6 ($a = 1$, $b = 5$, $c = 0$): F1 executed. Path 1-3-4-6 ($a = 5$, $b = 2$, $c = 9$): F2 executed. Path 1-3-5-6 ($a = 5$, $b = 2$, $c = 3$): F3 executed. ■

Example 8.12 Cyclomatic complexity of a factorial function

Compute the cyclomatic complexity of the following code using a flow graph.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result
```

Solution Number the nodes: 1 the test $n == 0$; 2 return 1; 3 result = 1 (with the loop initialization $i = 1$); 4 the loop test $i \leq n$; 5 result $\ast = i$ (with the increment of i); 6 return result; 7 exit. The flow graph is Figure 8.18.

The edges are 1→2, 1→3, 3→4, 4→5, 5→4 (the back edge of the loop), 4→6, 2→7, and 6→7, so $E = 8$ and $N = 7$.

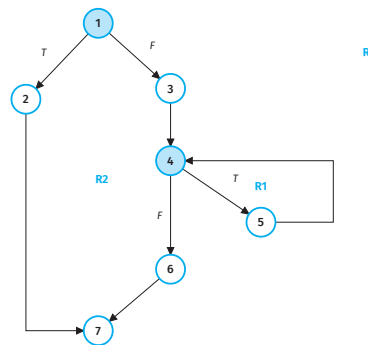


Figure 8.18 Flow graph of the factorial function

$$V(G) = E - N + 2 = 8 - 7 + 2 = 3.$$

Check by decisions: the predicate nodes are the if statement (node 1) and the for-loop test (node 4), so $D = 2$ and $V(G) = 2 + 1 = 3$. Check by regions: the loop encloses R1, the two branches of the if enclose R2, and R3 is the outer region, so $R = 3$.

A flowchart that draws only the loop, without the test $n == 0$, is a flowchart of a different program. It has one decision and gives $V(G) = 2$; the if statement must appear in the graph.

Answer $V(G) = 3$. The basis paths are 1–2–7 ($n = 0$, returns 1), 1–3–4–6–7 (loop not entered), and 1–3–4–5–4–6–7 (loop entered, for example $n = 3$, returns 6). The second path is taken only if the loop test is false at once, which for $n \neq 0$ happens only when n is negative; for $n = -1$ the function returns 1. ■

Example 8.13 A loop containing a decision

The following code counts the positive elements of an array *a* of length *n*:
 1 count = 0; 2 i = 0; 3 while i < n; 4 if a[i] > 0; 5 count = count + 1; 6 i = i + 1; 7 print(count). Find $V(G)$ and the basis path tests.

Solution The edges in Figure 8.19 are 1→2, 2→3, 3→4, 3→7, 4→5, 4→6, 5→6, and 6→3: $E = 8$ and $N = 7$, so $V(G) = 8 - 7 + 2 = 3$. The predicate nodes are 3 and 4, giving $2 + 1 = 3$. Merging the sequential nodes 1 and 2 gives $E = 7$ and $N = 6$, and $V(G)$ is still 3.

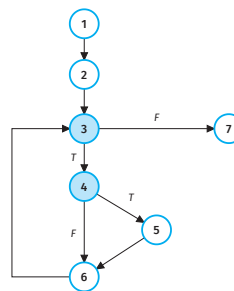


Figure 8.19 Flow graph of the count-positives loop

Answer $V(G) = 3$. Path 1–2–3–7 ($n = 0$): output 0. Path 1–2–3–4–6–3–7 ($n = 1$, $a = [-2]$): output 0. Path 1–2–3–4–5–6–3–7 ($n = 1$, $a = [5]$): output 1. ■

Example 8.14 Basis paths and equivalence classes for a leap-year function

For the following pseudocode, (a) design test cases for basis path testing and (b) apply equivalence class partitioning.

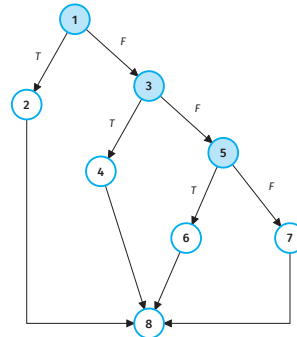
```

function isLeapYear(year):
  if (year is not divisible by 4) then return False
  else if (year is not divisible by 100) then return True
  else if (year is divisible by 400) then return True
  else return False
  
```

Solution (a) The flow graph has three predicate nodes (1, 3, and 5), one per condition (Figure 8.20). Its edges are 1→2, 1→3, 3→4, 3→5, 5→6, 5→7, and the four returns into the exit node 8, so $E = 10$ and $N = 8$: $V(G) = 10 - 8 + 2 = 4 = 3 + 1$.

Each independent path is described by the outcome of every condition it passes through. A path cannot be ‘divisible by 4, not divisible by 100, and divisible by 400’, because a year that is not divisible by 100 returns True before the third condition is tested.

Figure 8.20 Flow graph of the leap-year function



(b) Equivalence classes are formed from the *input domain*. A non-leap year is a valid input with the expected output False, so it is a valid class, not an invalid one. The valid classes follow the rules: V1 not divisible by 4; V2 divisible by 4 but not by 100; V3 divisible by 400; V4 divisible by 100 but not by 400. The invalid classes are inputs outside the specification: I1 a year ≤ 0 (if years must be positive) and I2 a non-integer value.

Answer (a) $V(G) = 4$. Path 1–2–8 (year 2023): False. Path 1–3–4–8 (2024): True. Path 1–3–5–6–8 (2000): True. Path 1–3–5–7–8 (1900): False. (b) Valid classes V1 (2023 $\rightarrow$ False), V2 (2024 $\rightarrow$ True), V3 (2000 $\rightarrow$ True), V4 (1900 $\rightarrow$ False); invalid classes I1 (–4 $\rightarrow$ rejected) and I2 ('abc' or 2020.5 $\rightarrow$ rejected). ■

Example 8.15 Cyclomatic complexity and path tests for a selection sort

Draw the flow graph of the following selection sort, find its cyclomatic complexity, and design a test suite that exercises its paths.

```

void selectionSort(int a[]) {
    int size = a.length;
    for (int i = 0; i < size; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        int temp = a[minIndex]; a[minIndex] = a[i]; a[i] = temp;
    }
}
  
```

Solution Number the nodes: 1 $\text{size} = \text{a.length}$ and $i = 0$; 2 the outer test $i < \text{size}$; 3 $\text{minIndex} = i$ and $j = i + 1$; 4 the inner test $j < \text{size}$; 5 the test $\text{a}[j] < \text{a}[\text{minIndex}]$; 6 $\text{minIndex} = j$; 7 $j++$; 8 the swap and $i++$; 9 exit (Figure 8.21).

The edges are 1 $\rightarrow$ 2, 2 $\rightarrow$ 3, 2 $\rightarrow$ 9, 3 $\rightarrow$ 4, 4 $\rightarrow$ 5, 4 $\rightarrow$ 8, 5 $\rightarrow$ 6, 5 $\rightarrow$ 7, 6 $\rightarrow$ 7, 7 $\rightarrow$ 4, and 8 $\rightarrow$ 2, so $E = 11$ and $N = 9$: $V(G) = 11 - 9 + 2 = 4$. The predicate nodes are 2, 4, and 5, giving $3 + 1 = 4$.

Because of the loops, the number of complete paths is unbounded, so path coverage is achieved in practice by the basis paths together with loop tests (zero, one,

and several iterations of each loop).

Answer $V(G) = 4$. Basis tests: P1 1–2–9 with $a = []$ (no change); P2 1–2–3–4–8–2–9 with $a = [5]$ (outer loop once, inner loop zero times); P3 ...4–5–7–4... with $a = [1, 2]$ (comparison false, already sorted); P4 ...4–5–6–7–4... with $a = [2, 1]$ (comparison true, result $[1, 2]$). Adding $a = [3, 1, 2]$ (result $[1, 2, 3]$) exercises several iterations of both loops. ■

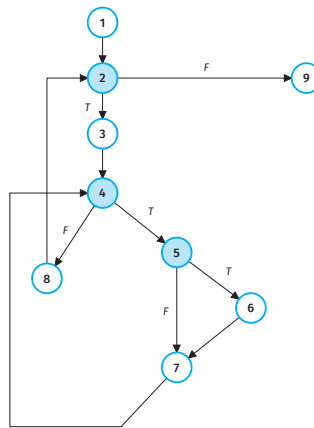


Figure 8.21 Flow graph of the selection sort

Example 8.16 Basis paths of a discount function

For the following function, (a) calculate the cyclomatic complexity, (b) identify the independent paths, and (c) write test cases that cover them.

```

float calculate_discount(float total_amount, int is_member,
                        char coupon_code[]) {
    if (total_amount <= 0) { return 0; }
    float discount = 0;
    if (is_member) { discount += total_amount * 0.1; }
    if (strcmp(coupon_code, "SAVE20") == 0)
        { discount += total_amount * 0.2; }
    else if (strcmp(coupon_code, "SAVE10") == 0)
        { discount += total_amount * 0.1; }
    float max_discount = total_amount * 0.3;
    if (discount > max_discount) { discount = max_discount; }
    float final_amount = total_amount - discount;
    return final_amount;
}
  
```

Solution Number the nodes: 1 $\text{total_amount} \leq 0$?; 2 return 0; 3 $\text{discount} = 0$ and is_member ?; 4 member discount; 5 SAVE20?; 6 add 20%; 7 SAVE10?; 8 add 10%; 9 $\text{max_discount} = 30\%$ and $\text{discount} > \text{max_discount}$?; 10 cap the discount; 11 compute and return final_amount ; 12 exit (Figure 8.22).

(a) The edges are 1→2, 1→3, 3→4, 3→5, 4→5, 5→6, 5→7, 6→9, 7→8, 7→9, 8→9, 9→10, 9→11, 10→11, 2→12, and 11→12, so $E = 16$ and $N = 12$: $V(G) = 16 - 12 + 2$

= 6. The predicate nodes are 1, 3, 5, 7, and 9 (the else-if is a separate decision), so $V(G) = 5 + 1 = 6$.

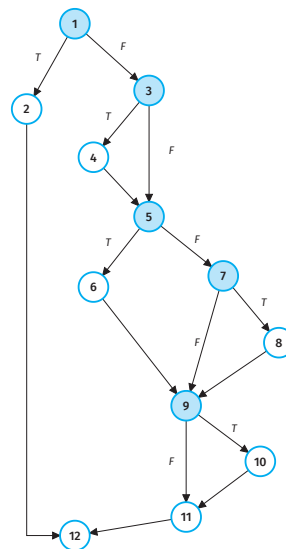
(b) Six independent paths, each adding at least one new edge:

- P1 1-2-12
- P2 1-3-5-7-9-11-12
- P3 1-3-4-5-7-9-11-12
- P4 1-3-4-5-6-9-11-12
- P5 1-3-4-5-7-8-9-11-12
- P6 1-3-...-9-10-11-12

(c) Path P6 requires $\text{discount} > 0.3 \times \text{total_amount}$, but the largest possible discount is $10\% + 20\% = 30\%$, which equals the cap and never exceeds it. P6 is therefore an *infeasible path* in exact arithmetic, and the cap is dead code. The test (200, 1, "SAVE20") gives a discount of 60 and a cap of 60, so it follows P4, not P6, and returns 140. In floating-point arithmetic, rounding can occasionally make the sum of the two discounts exceed the computed cap (for $\text{total_amount} = 3$ the cap is taken and the result is 2.1), so the path is reachable only by accident. The infeasibility is recorded, and the designer is asked whether the cap was intended for a combination that the code does not yet allow.

Answer $V(G) = 6$. Tests: P1 (0, 0, "NONE") → 0; P2 (100, 0, "NONE") → 100; P3 (100, 1, "NONE") → 90; P4 (100, 1, "SAVE20") → 70; P5 (100, 1, "SAVE10") → 80; P6 infeasible, since the combined discount can never exceed the 30% cap (the test (200, 1, "SAVE20") → 140 follows P4). ■

Figure 8.22 Flow graph of the discount function



Example 8.17 A flowchart with two decisions

In the following flowchart, b and c are inputs: Start; $a = 10$; if $a > b$ then $a = b$; otherwise, if $a > c$ then $b = c$, else $c = a$; then print a, b, c ; Stop. Compute $V(G)$ by all three methods and design the basis path tests.

Solution Number the boxes: 1 Start; 2 $a = 10$; 3 $a > b?$; 4 $a = b$; 5 $a > c?$; 6 $b = c$; 7 $c = a$; 8 print a, b, c ; 9 Stop (Figure 8.23).

Method 1. The edges are 1→2, 2→3, 3→4, 3→5, 5→6, 5→7, 4→8, 6→8, 7→8, 8→9, so $E = 10$ and $N = 9$: $V(G) = 10 - 9 + 2 = 3$.

Method 2. The predicate nodes are 3 and 5, so $V(G) = 2 + 1 = 3$.

Method 3. The regions are $R1$ (3–4–8–6–5–3), $R2$ (5–6–8–7–5), and the outer region $R3$, so $V(G) = 3$.

Answer $V(G) = 3$. Path 1–2–3–4–8–9 ($b = 5, c = 3$): prints 5, 5, 3. Path 1–2–3–5–6–8–9 ($b = 20, c = 4$): prints 10, 4, 4. Path 1–2–3–5–7–8–9 ($b = 20, c = 30$): prints 10, 20, 10. ■

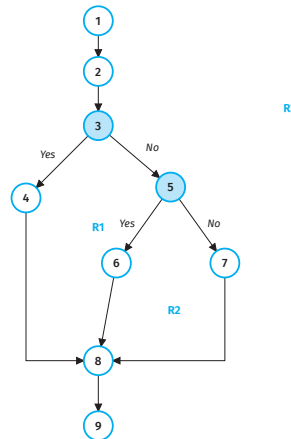


Figure 8.23 Flow graph of the three-variable flowchart

Example 8.18 A loop with constant bounds

Find $V(G)$ and the basis path tests for: 1 $\text{sum} = 0$; 2 $i = 1$; 3 while ($i \leq 5$); 4 $\text{sum} = \text{sum} + i$; 5 $i++$; 6 $\text{printf}(\text{"The sum is: \%d", sum})$; 7 return 0.

Solution From Figure 8.24, the edges are 1→2, 2→3, 3→4, 4→5, 5→3, 3→6, 6→7: $E = 7$ and $N = 7$, so $V(G) = 7 - 7 + 2 = 2$, and the single predicate node (3) gives $1 + 1 = 2$.

The basis paths are $P1$: 1–2–3–6–7 (loop not entered) and $P2$: 1–2–3–4–5–3–6–7 (loop entered). Because the bounds are constants, the test $i \leq 5$ is always true the first time, so no input can make $P1$ happen: it is an *infeasible path*. An infeasible path is recorded as such and not forced.

Answer $V(G) = 2$. $P2$ is tested by running the program: it iterates five times and prints ‘The sum is: 15’; this single run also traverses the exit edge 3→6, so all seven

edges are covered. P1 is infeasible. If the bound were an input n , P1 would be tested with $n = 0$ (expected output 'The sum is: 0'). ■

A compound condition is split into one predicate node per simple condition. For if $(x > 0 \text{ AND } y > 0)$ the graph has two predicate nodes and $V(G) = 3$, and the tests $(x = -1, y = 7)$, $(x = 2, y = -3)$, and $(x = 2, y = 3)$ exercise both simple conditions as false. Treating the compound condition as one node would give $V(G) = 2$ and would miss an error such as OR written for AND. Similarly, a switch with four outgoing edges counts as $4 - 1 = 3$ decisions, so a switch with three cases and a default has $V(G) = 4$.

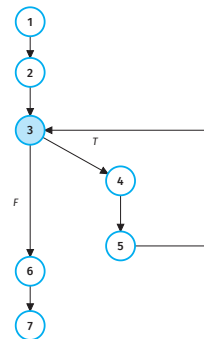


Figure 8.24 Flow graph of the summation loop

8.7.3 Graph matrices

A graph matrix is a square matrix whose rows and columns correspond to the nodes of a flow graph; the entry in row i and column j records a direct link from node i to node j . Direction matters, so the matrix is generally not symmetric. Links are usually named with letters and nodes with digits. A *connection matrix* replaces each link by a weight, in the simplest case 1 for a connection. It gives a mechanical way of computing $V(G)$: for each row, count the 1s and subtract 1; ignore empty rows; add the results; and add 1. A row with k entries is a k -way decision and contributes $k - 1$, so the procedure computes $D + 1$.

Example 8.19 Cyclomatic complexity from a connection matrix

A flow graph has the links a: $1 \rightarrow 1$ (a self-loop), b: $1 \rightarrow 2$, c: $1 \rightarrow 3$, d: $2 \rightarrow 4$, and e: $3 \rightarrow 4$. Draw its graph matrix and connection matrix, find $V(G)$, and list the paths.

Solution Each link is entered in the row of the node it leaves and the column of the node it enters (Figure 8.25). In the connection matrix, row 1 has three connections ($3 - 1 = 2$), rows 2 and 3 have one each ($1 - 1 = 0$), and row 4, the exit node, is empty and is ignored.

$V(G) = (2 + 0 + 0) + 1 = 3$. Check: $E = 5$ links and $N = 4$ nodes, so $E - N + 2 = 3$. The number of 1s in the matrix (5) equals E , and the number of rows (4) equals N ;

both are useful checks.

Answer $V(G) = 3$. Following the rows from node 1 to the empty row gives the paths 1–2–4 (links b, d) and 1–3–4 (links c, e); the third independent path uses the self-loop first, 1–1–2–4 (links a, b, d). ■

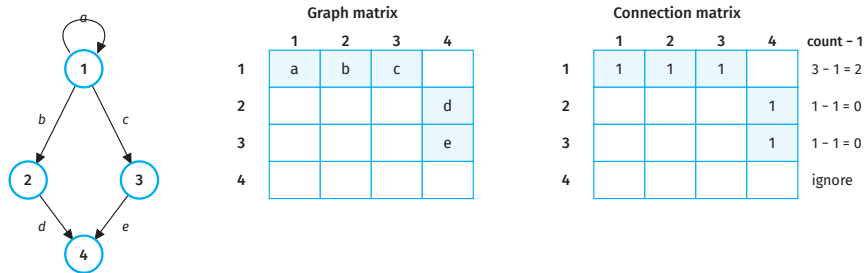


Figure 8.25 A flow graph with a self-loop, and its graph and connection matrices

When two parallel links join the same pair of nodes, the graph matrix records both in one cell (written $a + b$). This book follows the common convention of treating such a cell as a single connection when the connection matrix is built, so the cell holds 1; a graph with parallel links $a + b$ from 1 to 2, c from 1 to 3, and d from 3 to 4 therefore has $V(G) = (2 - 1) + (1 - 1) + 1 = 2$, with the paths 1–2 and 1–3–4. If each link is to be weighted separately, the cell holds 2 and the extra path via b is counted; the convention used should be stated.

8.8 Black-box testing

Black-box (behavioral) testing checks the functionality of a system without knowledge of its internal structure, by comparing the outputs produced for given inputs with the expected outputs. It is used to test the functional validity of the software, to look for interface errors, and to test behavior and performance. It finds missing or incorrect functions, which white-box testing cannot, while white-box testing finds untested internal paths, which black-box testing cannot. The two approaches are complementary.

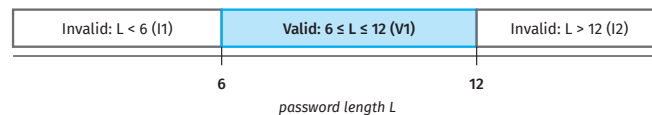
8.8.1 Equivalence partitioning

Equivalence class partitioning divides the input data into classes such that all members of a class are expected to behave in the same way. One representative from each class is tested, on the assumption that if it works, all the others will too. A *valid* class contains inputs the system should accept; an *invalid* class contains inputs that it should reject gracefully. An input range gives one valid and two invalid classes; a specific value gives one valid and two invalid classes; a member of a set or a Boolean gives one valid and one invalid class. Valid classes may be combined in one test, but

each invalid class should be tested on its own, so that the cause of a rejection is unambiguous.

A login system, for example, accepts a password only if its length L is 6 to 12 characters (Figure 8.26). The classes are $V1$ ($6 \leq L \leq 12$), $I1$ ($L < 6$), and $I2$ ($L > 12$), and representatives such as lengths 9, 3, and 16 test them.

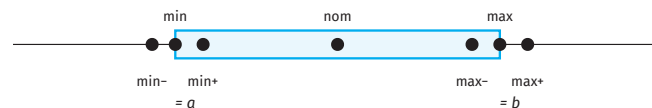
Figure 8.26
Equivalence classes
for a password length
of 6 to 12 characters



8.8.2 Boundary value analysis

Boundary value analysis (BVA) tests the edges of the input domain, selecting values just below, exactly on, and just above each boundary, because errors cluster at boundaries. Programmers often write $<$ where $\leq$ was intended, and such off-by-one errors leave typical values unaffected. For a range $[a, b]$ the values are min- , min , min+ , a nominal value, max- , max , and max+ (Figure 8.27). For the password rule the boundary tests are lengths 5, 6, 7, 11, 12, and 13; if the code were written $\text{len} > 6$ and $\text{len} < 12$, the tests at 6 and 12 would fail, whereas a mid-range value such as 9 would detect nothing. With n independent inputs, single-fault BVA needs $4n + 1$ tests and robustness testing, which adds min- and max+ , needs $6n + 1$.

Figure 8.27 Boundary
values for a range $[a, b]$



Example 8.20 Equivalence classes and boundary values for two inputs

An insurance form accepts an age from 18 to 60 and a policy type from {Gold, Silver}. Derive the equivalence classes, a minimal equivalence-partitioning test set, and the boundary value tests for age.

Solution *Classes.* Age: $V1$ ($18 \leq \text{age} \leq 60$), $I1$ ($\text{age} < 18$), $I2$ ($\text{age} > 60$). Policy: $V2$ (Gold or Silver), $I3$ (any other value).

ECP tests. The valid classes are combined in one test; each invalid class is tested alone with every other input valid: TC1 (35, Gold) $\rightarrow$ accepted; TC2 (12, Gold) $\rightarrow$ rejected, age; TC3 (70, Silver) $\rightarrow$ rejected, age; TC4 (35, Platinum) $\rightarrow$ rejected, policy.

BVA tests for age. Lower boundary 18: 17, 18, 19. Upper boundary 60: 59, 60, 61. With the nominal value 35, single-fault BVA uses 18, 19, 35, 59, 60 ($4n + 1 = 5$), and robustness testing adds 17 and 61 ($6n + 1 = 7$).

Answer Four ECP tests (one valid, three invalid) and seven boundary tests {17, 18, 19, 35, 59, 60, 61}. The values 17 and 61 also represent the invalid age classes, so the combined suite needs no separate values such as 12 and 70 for age. ■

Example 8.21 Boundary value tests for the factorial function

Identify the boundaries of the input domain of the factorial function of Example 8.12 and generate boundary value test cases for it.

Solution The specification is that the input is a non-negative integer, so the lower boundary is 0 and negative values are invalid. There is also an upper boundary, because the result must fit in the integer type used by the rest of the system. With 32-bit signed integers (maximum 2,147,483,647), $12! = 479,001,600$ fits but $13! = 6,227,020,800$ does not, so the valid domain is $0 \leq n \leq 12$ and 13 is the first invalid value above it. BVA therefore tests -1 , 0, 1, a nominal value, 11, 12, and 13.

It is tempting to take an arbitrary ‘reasonable’ maximum such as 10 and then also test 12 as a valid ‘large number’. That is inconsistent: once the maximum is fixed, a value above it is an invalid input, not a valid one. The boundary must come from the specification (here the integer range).

1. $n = -1$ (just below the minimum): expected, the input is rejected with an error. *The given code returns 1*, because $\text{range}(1, 0)$ is empty, so this test fails and reveals a defect: the function needs a guard such as `if  $n < 0$ : raise ValueError`.
2. $n = 0$ (minimum): expected 1.
3. $n = 1$ (just above the minimum): expected 1.
4. $n = 6$ (nominal): expected 720.
5. $n = 11$ (just below the maximum): expected 39,916,800.
6. $n = 12$ (maximum): expected 479,001,600.
7. $n = 13$ (just above the maximum): expected, rejected as out of range ($13!$ exceeds the 32-bit range). ■

Example 8.22 Test cases for a subscription-cost function

A function calculates a yearly subscription cost. The ‘basic’ plan costs \$100 per user and the ‘premium’ plan \$150 per user; for more than 100 users there is a 10% discount on basic and a 15% discount on premium.

```
function calculateYearlySubscriptionCost(numberOfUsers, planType) {
  let costPerUser = planType === 'basic' ? 100 : 150;
  let discount = numberOfUsers > 100 ? (planType === 'basic' ? 0.1
    : 0.15) : 0;
  return numberOfUsers * costPerUser * (1 - discount);
}
```

Write test cases covering both plans, the discount boundary, and invalid inputs, with expected outputs and rationale.

Solution The inputs are partitioned by plan (basic, premium, invalid) and by user count (invalid ≤ 0 , 1 to 100 without discount, above 100 with discount); the boundary is at 100/101. Every expected output is computed from the *specification*, not by running the code.

1. (50, 'basic') $\rightarrow$ 5,000: basic plan, no discount.
2. (100, 'basic') $\rightarrow$ 10,000: on the boundary; 100 users do not receive the discount.
3. (101, 'basic') $\rightarrow 101 \times 100 \times 0.9 = 9,090$: just above the boundary.
4. (100, 'premium') $\rightarrow$ 15,000: premium plan on the boundary.
5. (101, 'premium') $\rightarrow 101 \times 150 \times 0.85 = 12,877.50$: premium discount just above the boundary (a tempting wrong answer is 12,835).
6. (1, 'basic') $\rightarrow$ 100: minimum valid user count.
7. (0, 'basic') and (-1, 'premium') $\rightarrow$ rejected: invalid user counts. The code returns 0 and -150, so the second test fails and reveals that negative counts are not checked.
8. (50, 'gold') or (50, null) $\rightarrow$ rejected: invalid plan. The code returns 7,500, because every plan that is not 'basic' is charged as premium; this test also fails and reveals a defect. ■

Example 8.23 Black-box tests for a fund transfer

A banking application transfers funds between a user's own accounts and to other accounts in the same bank. The user enters the recipient's account number, the amount, and an optional message. Design black-box tests, stating your assumptions.

Solution *Assumptions.* Transfers are only within the bank; the amount must be from ₹1 to ₹1,00,000 per transfer; the amount cannot exceed the sender's balance; account numbers have exactly 12 digits; the message is at most 100 characters.

Equivalence classes. Valid: V1 own-account transfer; V2 transfer to another customer's account; V3 amount within limits and $\leq$ balance; V4 12-digit existing account. Invalid: I1 amount < 1 ; I2 amount $> 1,00,000$; I3 amount $>$ balance; I4 account number of wrong length; I5 non-existent account; I6 transfer to the same account as the sender; I7 message longer than 100 characters.

Boundary values for the amount. 0, 1, 2, a nominal value, 99,999, 1,00,000, and 1,00,001; and, with a balance of 5,000, amounts of 5,000 and 5,001. For the account number, lengths 11, 12, and 13.

Answer Representative tests: own-account transfer of ₹500 $\rightarrow$ success, both balances updated; transfer of ₹1 and ₹1,00,000 to a valid account $\rightarrow$ success; ₹0 and ₹1,00,001 $\rightarrow$ rejected; ₹5,001 with a balance of ₹5,000 $\rightarrow$ rejected, 'insufficient funds', and no balance changes; an 11- or 13-digit account number $\rightarrow$ rejected; a valid-format number that does not exist $\rightarrow$ rejected; a transfer to the sender's own

source account → rejected; a 101-character message → rejected or truncated as specified. ■

8.8.3 Decision tables and state transitions

Decision table testing is used for systems with complex business logic. A decision table lists the *conditions* (inputs that affect behavior), the *actions* (outputs), and the *rules*, each a unique combination of conditions with its actions. A complete table of n Boolean conditions has 2^n rules, and each rule gives a test case. For a login with the conditions ‘username correct’ and ‘password correct’, there are four rules; only T, T shows the home page, and the other three show an error message. Rules with the same actions that differ only in one condition may be merged with a ‘don’t care’ entry.

Example 8.24 A decision table for a checkout discount

At checkout, a logged-in customer whose cart total exceeds ₹1,000 receives a 10% discount; a logged-in customer with a smaller total pays full price; a customer who is not logged in is redirected to the login page, whatever the total. Build the decision table and derive the test cases.

Solution There are two Boolean conditions, C1 (logged in?) and C2 (total > ₹1,000?), so the table has $2^2 = 4$ rules (Figure 8.28).

Figure 8.28 Decision table for the checkout discount

	R1	R2	R3	R4
C1: Logged in?	T	T	F	F
C2: Cart total > ₹1,000?	T	F	T	F
A1: Apply 10% discount	x			
A2: Charge full price		x		
A3: Redirect to login			x	x

Answer R1: logged in, total ₹1,500 → pays ₹1,350. R2: logged in, total ₹800 → pays ₹800. R3: not logged in, ₹1,500 → login page. R4: not logged in, ₹800 → login page. R3 and R4 have the same action and differ only in C2, so they may be merged into one rule with C2 = ‘-’ (don’t care), giving three tests. Boundary values for C2 should be added: a total of exactly ₹1,000 must pay full price, and ₹1,000.01 must receive the discount. ■

State transition testing is used for systems whose behavior depends on their current state and so on the sequence of earlier inputs. The system is modeled by states, transitions, the events that cause them, and the outputs produced. A test is designed for each transition or sequence of events, and invalid events in each state are also tested, where the expected result is that the state does not change. Figure 8.29 models PIN entry at an ATM: the tests are a correct PIN first time; one wrong then correct;

two wrong then correct; and three wrong, which blocks the account and is the most important negative test. Figure 8.30 models a login system, whose transition table gives five tests: enter username; valid password; invalid password; retry after an error; and logout.

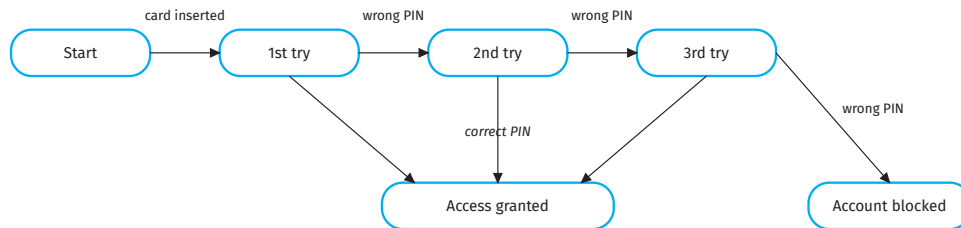


Figure 8.29 State transition diagram for PIN entry at an ATM

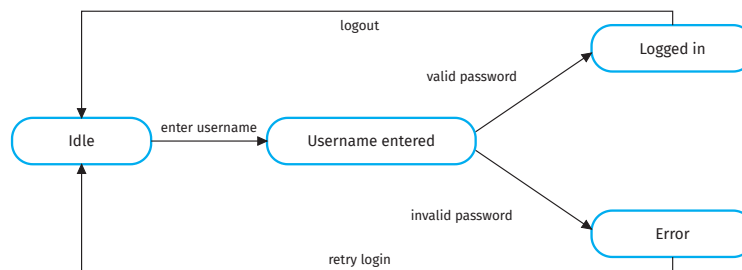


Figure 8.30 State transition diagram for a login system

8.9 Regression and mutation testing

Regression testing is testing performed to ensure that recent changes (bug fixes, new features, or refactoring) have not adversely affected existing functionality. Modules are coupled, so a change in one can break a feature that was never touched; such an unintended side effect is a *regression*. Regression testing re-runs previously passed tests after every fix, after new features are added, after refactoring, and before each release. Its steps are to identify the areas affected by a change, select the relevant test cases, execute them, and analyze the results (Figure 8.31). Because the tests are repeated often, they are usually automated. Regression testing differs from *retesting*, which re-runs the specific test that failed in order to confirm that a reported defect has been fixed.

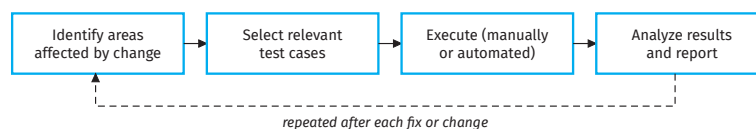


Figure 8.31 The regression testing cycle

Mutation testing evaluates the quality of a test suite. Small, deliberate changes (mutations) are made to the source code, such as replacing + with −, > with ≥, or deleting a statement, and the test suite is run against each mutant. If some test fails, the mutant is *killed*; if all tests pass, it *survives*, which reveals a weakness in the tests. An *equivalent mutant* behaves identically to the original for every input and is excluded. The mutation score is

$$\text{Mutation score} = \text{killed mutants} \div (\text{total mutants} - \text{equivalent mutants}) \times 100\%.$$

For example, if a suite kills 90 of 120 mutants and none is equivalent, the score is $90 \div 120 \times 100 = 75\%$. If 35 of 50 mutants are killed and 5 are equivalent, the score is $35 \div 45 \times 100 = 77.78\%$; if improved tests kill 7 more, it becomes $42 \div 45 \times 100 = 93.33\%$.

Example 8.25 Killing relational and arithmetic mutants

(a) The code `if (x > 5) return "High"; else return "Low";` is tested with $x = 10$ and $x = 2$. The mutants are M1: $x \geq 5$, M2: $x < 5$, and M3: $x > 6$. Find the mutation score and improve the tests. (b) The code `return a + b;` with mutants $a - b$, $a * b$, and a / b is tested with $(a, b) = (2, 2)$. Find the score and improve the test.

Solution (a) For $x = 10$ the original and M1 and M3 return High, but M2 returns Low, so M2 is killed. For $x = 2$ the original, M1, and M3 return Low. M1 and M3 survive: score $1 \div 3 \times 100 = 33.33\%$. The test $x = 5$ makes the original return Low and M1 return High, killing M1; the test $x = 6$ makes the original return High and M3 return Low, killing M3.

(b) The expected result is 4. $a - b$ gives 0 and a / b gives 1, so both are killed, but $a * b$ gives 4 and survives, because $2 + 2 = 2 \times 2$: score $2 \div 3 \times 100 = 66.67\%$. The test $(3, 1)$, expected 4, gives 3 for $a * b$ and kills it.

Answer (a) 33.33%, rising to 100% with the tests {10, 2, 5, 6}: boundary values kill relational mutants. (b) 66.67%, rising to 100% with (3, 1): test data should avoid values for which different operators give the same result. ■

8.10 Testing object-oriented and web systems

Object-oriented software needs a different approach because the natural unit of testing is the class, not the function: an operation's result depends on the object's state, operations may be inherited or overridden, and the code executed for a call may be chosen only at run time. *Class testing* exercises sequences of operations on one object, observing its state through public operations because its attributes are encapsulated. *Integration testing* checks that collaborating objects exchange messages correctly. *Inheritance testing* re-runs a base class's tests in the context of each subclass, because an overridden method works under new rules. *Polymorphism testing* supplies a test for each possible binding of a polymorphic call. *Scenario-based testing*

follows a use case across several objects. The difficulties are encapsulation, inheritance (a change in a base class can affect every subclass), polymorphism, and state dependencies. The main techniques are *random testing* (random inputs and operation sequences), *fault-based testing* (planting faults, as in mutation testing), *scenario-based testing*, and *use-case testing*. For example, an Account class is class-tested with sequences such as deposit(100), withdraw(40), and getBalance() → 60, and withdraw(10) on a new account → rejected; if SavingsAccount overrides withdraw to keep a minimum balance of 500, the inherited tests are re-run for SavingsAccount, and with a balance of 1,000 it must allow withdraw(500) but reject withdraw(501).

A web-based system runs in a browser and interacts with its users over the Internet, so it must work across many devices, browsers, and network conditions. Its testing includes functional, cross-browser, cross-platform, performance (load and stress), security (such as SQL injection and session hijacking), usability, compatibility, database, API, accessibility, localization and internationalization, and regression testing.

Example 8.26 Testing the checkout of an online store

A clothing store's website lets users browse and buy items online. Describe how you would test the checkout process to ensure a smooth and efficient user experience.

1. *Functional testing* of the whole flow: add to cart, enter shipping and billing details, choose a payment method, review, and confirm; check that the order is recorded and a confirmation e-mail is sent.
2. *Form validation and error handling*: invalid e-mail addresses, postal codes, and card numbers must give clear messages without losing the other entries.
3. *Payment testing* for every method (cards, UPI, wallets), including declined and timed-out payments.
4. *Cross-browser and cross-platform testing* on desktop, tablet, and phone.
5. *Performance and load testing* with many simultaneous users, as during a sale.
6. *Security testing*: HTTPS on every page, no storage of card data, and protection against injection and session hijacking.
7. *Usability testing* with real users, who should complete a purchase quickly without help.
8. *Regression testing* of checkout after every change to the cart, pricing, or payment code. ■

KEY POINTS

- Testing executes software with the intent of finding errors. It proceeds from unit to integration, validation, and system testing, and may be manual or automated, functional or non-functional, white-box or black-box.
- The test process is planning and control, analysis, design, implementation and execution, and completion. A test plan states objectives, scope, approach (levels, types, techniques, entry and exit criteria), resources, schedule, risks, metrics, and deliverables; a test case has an ID, preconditions, steps, data, expected result, and actual result.
- The V model pairs each development phase with a test level: requirements with acceptance testing, system design with system testing, architecture with integration testing, and module design with unit testing. The left side is verified by static reviews; the right side is validated by execution.
- A Fagan inspection has six steps (planning, overview, preparation, meeting, rework, follow-up) and the roles of moderator, reader, recorder, author, and inspectors. An audit is an independent check of compliance.
- Stubs replace called modules (top-down integration); drivers replace calling modules (bottom-up). Integration may be big bang, top-down, bottom-up, or sandwich; system integration testing uses end-to-end, contract, and service virtualization techniques.
- Reviews and inspections are static techniques. Errors do not mask one another, incomplete systems can be inspected, and broader quality attributes can be checked. Inspections are driven by checklists of common errors. Audits add independent checks of compliance, ethics, and maintainability that testing cannot provide.
- Statement, branch, and path coverage measure how much of the code is exercised; path coverage implies branch coverage, which implies statement coverage.
- Basis path testing draws a flow graph, computes $V(G) = E - N + 2 = D + 1 = R$, and designs one test for each independent path. Every decision, including loop tests and each simple part of a compound condition, is a predicate node.
- Equivalence partitioning tests one value from each valid and invalid class; boundary value analysis tests just below, on, and just above each boundary, which must be derived from the specification.
- Decision tables test combinations of conditions; state transition testing tests sequences of events.
- Regression testing checks that changes have not broken existing behavior; mutation testing measures how many deliberately planted faults a test suite detects.